

ЯЗЫК C++
ПРОГРАММИРОВАНИЯ
специальное издание

The C++ Programming Language

Special Edition

Bjarne Stroustrup

AT&T Labs
Murray Hill, New Jersey



Addison-Wesley
An Imprint of Addison Wesley Longman, Inc.

Бьери Страуструп

Язык программирования C++

Специальное издание

Перевод с английского
под редакцией
Н.Н. Мартынова



Москва
Издательство **БИНОМ**
2011

ББК 32.973.26-018.1
С83

Бьерн Страуструп

Язык программирования С++. Специальное издание. Пер. с англ. — М.: Издательство Бином, 2011 г. — 1136 с.: ил.

Книга написана Бьерном Страуструпом – автором языка программирования С++ – и является каноническим изложением возможностей этого языка. Помимо подробного описания собственно языка, на страницах книги вы найдете доказавшие свою эффективность подходы к решению разнообразных задач проектирования и программирования. Многочисленные примеры демонстрируют как хороший стиль программирования на С-совместимом ядре С++, так и современный объектно-ориентированный подход к созданию программных продуктов.

Книга адресована программистам, использующим в своей повседневной работе С++. Она также будет полезна преподавателям, студентам и всем, кто хочет ознакомиться с описанием языка «из первых рук».

Все права защищены. Никакая часть этой книги не может быть воспроизведена в любой форме или любыми средствами, электронными или механическими, включая фотографирование, магнитную запись или иные средства копирования или сохранения информации без письменного разрешения издательства

Translation copyright © 2010 by Binom Publishers.

С++ Programming Language, The: Special Edition,

First Edition by Bjarne Stroustrup, Copyright © 2000, All Rights Reserved.

Published by arrangement with the original publisher, Addison Wesley Longman, a Pearson Education Company.

ISBN 978-5-7989-0425-9 (рус.)
ISBN 0-201-70073-5 (англ.)

© Addison Wesley Longman,
a Pearson Education Company, 2000
© Издание на русском языке.
Издательство Бином, 2011

Краткое содержание

Предисловие переводчика и редактора	25
Предисловие автора к третьему русскому изданию	26
Предисловие	29
Предисловие ко второму изданию	31
Предисловие к первому изданию	33
Введение	35
1. Обращение к читателю	37
2. Обзор языка C++	59
3. Обзор стандартной библиотеки	85
Часть I. Основные средства	111
4. Типы и объявления	113
5. Указатели, массивы и структуры	133
6. Выражения и операторы	155
7. Функции	195
8. Пространства имен и исключения	219
9. Исходные файлы и программы	253
Часть II. Механизмы абстракции	281
10. Классы	283
11. Перегрузка операций	327
12. Наследование классов	371
13. Шаблоны	401
14. Обработка исключений	433
15. Иерархии классов	473
Часть III. Стандартная библиотека	515
16. Организация библиотеки и контейнеры	517
17. Стандартные контейнеры	555
18. Алгоритмы и классы функциональных объектов	607
19. Итераторы и аллокаторы	655
20. Строки	689
21. Потоки	717
22. Классы для математических вычислений	775

Часть IV. Проектирование с использованием C++	809
23. Общий взгляд на разработку программ. Проектирование	811
24. Проектирование и программирование	849
25. Роли классов	895
Приложения и предметный указатель	923
A. Грамматика	925
B. Совместимость	947
C. Технические подробности	961
D. Локализация	1007
E. Исключения и безопасность стандартной библиотеки	1077
Предметный указатель.	1117

Содержание

Предисловие переводчика и редактора	25
Предисловие автора к третьему русскому изданию	26
Предисловие	29
Предисловие ко второму изданию	31
Предисловие к первому изданию	33
Введение	35
Глава 1. Обращение к читателю	37
1.1. Структура книги	37
1.1.1. Примеры и ссылки.	39
1.1.2. Упражнения	40
1.1.3. Замечания о конкретных реализациях языка (компиляторах)	40
1.2. Как изучать C++	40
1.3. Как проектировался C++	42
1.3.1. Эффективность и структура	43
1.3.2. Философские замечания	44
1.4. Исторические замечания	45
1.5. Применение C++	47
1.6. Языки C и C++	49
1.6.1. Информация для C-программистов.	50
1.6.2. Информация для C++-программистов	50
1.7. Размышления о программировании на C++	51
1.8. Советы	53
1.8.1. Литература.	54
Глава 2. Обзор языка C++	59
2.1. Что такое язык C++?	59
2.2. Парадигмы программирования	60
2.3. Процедурное программирование	61
2.3.1. Переменные и простейшая арифметика	62
2.3.2. Операторы ветвления и циклы	63
2.3.3. Указатели и массивы.	64
2.4. Модульное программирование.	65
2.4.1. Раздельная компиляция	67
2.4.2. Обработка исключений.	68

2.5. Абстракция данных	69
2.5.1. Модули, определяющие типы	69
2.5.2. Типы, определяемые пользователем	71
2.5.3. Конкретные типы	73
2.5.4. Абстрактные типы	74
2.5.5. Виртуальные функции	77
2.6. Объектно-ориентированное программирование	77
2.6.1. Проблемы, связанные с конкретными типами	78
2.6.2. Классовые иерархии	79
2.7. Обобщенное программирование	81
2.7.1. Контейнеры	81
2.7.2. Обобщенные алгоритмы	82
2.8. Заключение	84
2.9. Советы	84
Глава 3. Обзор стандартной библиотеки.	85
3.1. Введение	85
3.2. Hello, world! (Здравствуй, мир!)	86
3.3. Пространство имен стандартной библиотеки.	87
3.4. Вывод	87
3.5. Строки	88
3.5.1. С-строки	90
3.6. Ввод	90
3.7. Контейнеры	92
3.7.1. Контейнер vector.	93
3.7.2. Проверка диапазона индексов	94
3.7.3. Контейнер list	95
3.7.4. Контейнер map	96
3.7.5. Контейнеры стандартной библиотеки.	97
3.8. Алгоритмы	98
3.8.1. Использование итераторов	99
3.8.2. Типы итераторов	101
3.8.3. Итераторы и ввод/вывод	102
3.8.4. Алгоритм for_each и предикаты	103
3.8.5. Алгоритмы, использующие функции-члены классов	105
3.8.6. Алгоритмы стандартной библиотеки.	106
3.9. Математические вычисления	107
3.9.1. Комплексные числа.	107
3.9.2. Векторная арифметика	107
3.9.3. Поддержка базовых вычислительных операций	108
3.10. Основные средства стандартной библиотеки.	108
3.11. Советы	109
Часть I. Основные средства	111
Глава 4. Типы и объявления	113
4.1. Типы	113
4.1.1. Фундаментальные типы	114
4.2. Логический тип	115

4.3. Символьные типы	116
4.3.1. Символьные литералы	117
4.4. Целые типы	118
4.4.1. Целые литералы	118
4.5. Типы с плавающей запятой	119
4.5.1. Литералы с плавающей запятой	119
4.6. Размеры	119
4.7. Тип void	121
4.8. Перечисления	122
4.9. Объявления	123
4.9.1. Структура объявления	125
4.9.2. Объявление нескольких имен	126
4.9.3. Имена	126
4.9.4. Область видимости	127
4.9.5. Инициализация	129
4.9.6. Объекты и леводопустимые выражения (lvalue)	130
4.9.7. Ключевое слово typedef	130
4.10. Советы	131
4.11. Упражнения	132
Глава 5. Указатели, массивы и структуры.	133
5.1. Указатели	133
5.1.1. Нуль	134
5.2. Массивы.	135
5.2.1. Инициализация массивов	135
5.2.2. Строковые литералы	136
5.3. Указатели на массивы	138
5.3.1. Доступ к элементам массивов	139
5.4. Константы	141
5.4.1. Указатели и константы	143
5.5. Ссылки	144
5.6. Тип void*	148
5.7. Структуры	149
5.7.1. Эквивалентность типов	152
5.8. Советы	152
5.9. Упражнения	152
Глава 6. Выражения и операторы	155
6.1. Калькулятор	155
6.1.1. Синтаксический анализатор	156
6.1.2. Функция ввода	161
6.1.3. Низкоуровневый ввод	163
6.1.4. Обработка ошибок	164
6.1.5. Управляющая программа	165
6.1.6. Заголовочные файлы	166
6.1.7. Аргументы командной строки	167
6.1.8. Замечания о стиле	169
6.2. Обзор операций языка C++	169
6.2.1. Результаты операций	171

6.2.2. Последовательность вычислений	172
6.2.3. Приоритет операций	173
6.2.4. Побитовые логические операции	174
6.2.5. Инкремент и декремент	175
6.2.6. Свободная память.	177
6.2.6.1. Массивы.	179
6.2.6.2. Исчерпание памяти	180
6.2.7. Явное приведение типов	181
6.2.8. Конструкторы	182
6.3. Обзор операторов языка C++	183
6.3.1. Объявления как операторы	184
6.3.2. Операторы выбора (условные операторы)	185
6.3.2.1. Объявления в условиях.	187
6.3.3. Операторы цикла	188
6.3.3.1. Объявления в операторах цикла for	189
6.3.4. Оператор goto.	189
6.4. Комментарии и отступы	190
6.5. Советы	192
6.6. Упражнения	192
Глава 7. Функции	195
7.1. Объявления функций	195
7.1.1. Определения функций	196
7.1.2. Статические переменные	197
7.2. Передача аргументов	197
7.2.1. Массивы в качестве аргументов	199
7.3. Возвращаемое значение	200
7.4. Перегрузка имен функций	201
7.4.1. Перегрузка и возвращаемые типы	204
7.4.2. Перегрузка и области видимости	204
7.4.3. Явное разрешение неоднозначностей	204
7.4.4. Разрешение в случае нескольких аргументов.	205
7.5. Аргументы по умолчанию	206
7.6. Неуказанное число аргументов	207
7.7. Указатели на функции	209
7.8. Макросы.	213
7.8.1. Условная компиляция.	215
7.9. Советы	216
7.10. Упражнения	217
Глава 8. Пространства имен и исключения	219
8.1. Разбиение на модули и интерфейсы.	219
8.2. Пространства имен	221
8.2.1. Квалифицированные имена	223
8.2.2. Объявления using	224
8.2.3. Директивы using	226
8.2.4. Множественные интерфейсы	227
8.2.4.1. Альтернативы интерфейсам.	229
8.2.5. Как избежать конфликта имен	231
8.2.5.1. Неименованные пространства имен.	232

8.2.6. Поиск имен	233
8.2.7. Псевдонимы пространств имен	234
8.2.8. Композиция пространств имен	235
8.2.8.1. Отбор отдельных элементов из пространства имен	236
8.2.8.2. Композиция пространств имен и отбор элементов	237
8.2.9. Пространства имен и старый код	238
8.2.9.1. Пространства имен и язык С	238
8.2.9.2. Пространства имен и перегрузка	239
8.2.9.3. Пространства имен открыты	240
8.3. Исключения	241
8.3.1. Ключевые слова throw и catch	242
8.3.2. Обработка нескольких исключений	244
8.3.3. Исключения в программе калькулятора	246
8.3.3.1. Альтернативные стратегии обработки ошибок	249
8.4. Советы	251
8.5. Упражнения	252
Глава 9. Исходные файлы и программы	253
9.1. Раздельная компиляция	253
9.2. Компоновка (linkage).	254
9.2.1. Заголовочные файлы	257
9.2.2. Заголовочные файлы стандартной библиотеки	259
9.2.3. Правило одного определения	260
9.2.4. Компоновка с кодом, написанном не на языке C++.	262
9.2.5. Компоновка и указатели на функции	264
9.3. Применяем заголовочные файлы	265
9.3.1. Единственный заголовочный файл	265
9.3.2. Несколько заголовочных файлов	268
9.3.2.1. Остальные модули калькулятора	271
9.3.2.2. Использование заголовочных файлов	273
9.3.3. Защита от повторных включений	274
9.4. Программы	275
9.4.1. Инициализация нелокальных переменных	275
9.4.1.1. Завершение выполнения программы	276
9.5. Советы	278
9.6. Упражнения	278
Часть II. Механизмы абстракции	281
Глава 10. Классы	283
10.1. Введение	283
10.2. Классы	284
10.2.1. Функции-члены	284
10.2.2. Управление режимом доступа	285
10.2.3. Конструкторы	287
10.2.4. Статические члены	288
10.2.5. Копирование объектов класса	290
10.2.6. Константные функции-члены	290
10.2.7. Ссылки на себя	291
10.2.7.1. Физическое и логическое постоянство	292
10.2.7.2. Ключевое слово mutable	294

10.2.8. Структуры и классы	295
10.2.9. Определение функций в теле определения класса.	296
10.3. Эффективные пользовательские типы	297
10.3.1. Функции-члены	300
10.3.2. Функции поддержки (helper functions)	302
10.3.3. Перегруженные операции	303
10.3.4. Роль конкретных классов	303
10.4. Объекты	304
10.4.1. Деструкторы	305
10.4.2. Конструкторы по умолчанию.	306
10.4.3. Конструирование и уничтожение объектов	307
10.4.4. Локальные объекты	307
10.4.4.1. Копирование объектов	308
10.4.5. Динамическое создание объектов в свободной памяти	309
10.4.6. Классовые объекты как члены классов	310
10.4.6.1. Обязательная инициализация членов	311
10.4.6.2. Члены-константы	312
10.4.6.3. Копирование членов	313
10.4.7. Массивы.	314
10.4.8. Локальные статические объекты	315
10.4.9. Нелокальные объекты	316
10.4.10. Временные объекты.	318
10.4.11. Размещение объектов в заданных блоках памяти	319
10.4.12. Объединения	321
10.5. Советы	322
10.6. Упражнения	323
Глава 11. Перегрузка операций	327
11.1. Введение	327
11.2. Функции-операции	329
11.2.1. Бинарные и унарные операции.	330
11.2.2. Предопределенный смысл операций	331
11.2.3. Операции и пользовательские типы	331
11.2.4. Операции и пространства имен	332
11.3. Тип комплексных чисел.	334
11.3.1. Перегрузка операций функциями-членами и глобальными функциями	334
11.3.2. Смешанная арифметика.	336
11.3.3. Инициализация	337
11.3.4. Копирование	338
11.3.5. Конструкторы и преобразования типов.	339
11.3.6. Литералы	340
11.3.7. Дополнительные функции-члены	341
11.3.8. Функции поддержки (helper functions)	341
11.4. Операции приведения типов	342
11.4.1. Неоднозначности	344
11.5. Друзья класса.	346
11.5.1. Поиск друзей	348
11.5.2. Функции-члены или друзья?	349
11.6. Объекты больших размеров	350

11.7. Важные операции	352
11.7.1. Конструктор с модификатором explicit	353
11.8. Индексирование	355
11.9. Функциональный вызов.	356
11.10. Разыменование	358
11.11. Инкремент и декремент	360
11.12. Класс строк	362
11.13. Советы	367
11.14. Упражнения	368
Глава 12. Наследование классов.	371
12.1. Введение	371
12.2. Производные классы	372
12.2.1. Функции-члены	375
12.2.2. Конструкторы и деструкторы.	376
12.2.3. Копирование	378
12.2.4. Иерархии классов	378
12.2.5. Поля типа	379
12.2.6. Виртуальные функции	381
12.3. Абстрактные классы	384
12.4. Проектирование иерархий классов	386
12.4.1. Традиционные иерархии классов.	387
12.4.1.1. Критика	389
12.4.2. Абстрактные классы	390
12.4.3. Альтернативные реализации	393
12.4.3.1. Критика	395
12.4.4. Локализация создания объектов	395
12.5. Классовые иерархии и абстрактные классы	397
12.6. Советы	397
12.7. Упражнения	398
Глава 13. Шаблоны	401
13.1. Введение	401
13.2. Простой шаблон строк	402
13.2.1. Определение шаблона	404
13.2.2. Конкретизация шаблона (template instantiation)	406
13.2.3. Параметры шаблонов	406
13.2.4. Эквивалентность типов	407
13.2.5. Проверка типов	408
13.3. Шаблоны функций	409
13.3.1. Аргументы функциональных шаблонов.	410
13.3.2. Перегрузка функциональных шаблонов	411
13.4. Применение аргументов шаблона для формирования различных вариантов поведения кода.	414
13.4.1. Параметры шаблонов по умолчанию	415
13.5. Специализация	417
13.5.1. Порядок специализаций	420
13.5.2. Специализация шаблонов функций	420
13.6. Наследование и шаблоны	422

13.6.1. Параметризация и наследование	424
13.6.2. Шаблонные члены шаблонов	424
13.6.3. Отношения наследования	425
13.6.3.1. Преобразования шаблонов	426
13.7. Организация исходного кода	427
13.8. Советы	430
13.9. Упражнения	431
Глава 14. Обработка исключений	433
14.1. Обработка ошибок	433
14.1.1. Альтернативный взгляд на исключения	436
14.2. Группировка исключений	437
14.2.1. Производные исключения	438
14.2.2. Композитные (комбинированные) исключения	440
14.3. Перехват исключений	441
14.3.1. Повторная генерация исключений	441
14.3.2. Перехват любых исключений	442
14.3.2.1. Порядок записи обработчиков	443
14.4. Управление ресурсами	444
14.4.1. Использование конструкторов и деструкторов	446
14.4.2. Auto_ptr	447
14.4.3. Предостережение	449
14.4.4. Исключения и операция new	449
14.4.5. Исчерпание ресурсов	450
14.4.6. Исключения в конструкторах	452
14.4.6.1. Исключения и инициализация членов классов	454
14.4.6.2. Исключения и копирование	454
14.4.7. Исключения в деструкторах	455
14.5. Исключения, не являющиеся ошибками	455
14.6. Спецификация исключений	457
14.6.1. Проверка спецификации исключений	458
14.6.2. Неожиданные исключения	459
14.6.3. Отображение исключений	460
14.6.3.1. Отображение исключений пользователем	460
14.6.3.2. Восстановление типа исключения	461
14.7. Неперехваченные исключения	462
14.8. Исключения и эффективность	464
14.9. Альтернативы обработке ошибок	465
14.10. Стандартные исключения	467
14.11. Советы	469
14.12. Упражнения	470
Глава 15. Иерархии классов	473
15.1. Введение и обзор	473
15.2. Множественное наследование	474
15.2.1. Разрешение неоднозначности	475
15.2.2. Наследование и using-объявление	477
15.2.3. Повторяющиеся базовые классы	478
15.2.3.1. Замещение	479
15.2.4. Виртуальные базовые классы	480

15.2.4.1. Программирование виртуальных базовых классов	482
15.2.5. Применение множественного наследования	484
15.2.5.1. Замещение функций виртуальных базовых классов.	486
15.3. Контроль доступа	487
15.3.1. Защищенные члены классов	489
15.3.1.1. Применение защищенных членов класса	490
15.3.2. Доступ к базовым классам	491
15.3.2.1. Множественное наследование и контроль доступа	492
15.3.2.2. Множественное наследование и контроль доступа	493
15.4. Механизм RTTI (Run-Time Type Information)	493
15.4.1. Операция <code>dynamic_cast</code>	495
15.4.1.1. Применение <code>dynamic_cast</code> к ссылкам	497
15.4.2. Навигация по иерархиям классов.	498
15.4.2.1. Операции <code>static_cast</code> и <code>dynamic_cast</code>	499
15.4.3. Конструирование и уничтожение классовых объектов.	501
15.4.4. Операция <code>typeid</code> и расширенная информация о типе	501
15.4.4.1. Расширенная информация о типе	502
15.4.5. Корректное и некорректное применение RTTI	504
15.5. Указатели на члены классов.	505
15.5.1. Базовые и производные классы	508
15.6. Свободная память.	509
15.6.1. Выделение памяти под массивы	511
15.6.2. «Виртуальные конструкторы»	511
15.7. Советы	513
15.8. Упражнения	514
Часть III. Стандартная библиотека	515
Глава 16. Организация библиотеки и контейнеры	517
16.1. Проектные решения стандартной библиотеки.	517
16.1.1. Проектные ограничения	518
16.1.2. Организация стандартной библиотеки	520
16.1.3. Непосредственная поддержка языка C++.	523
16.2. Дизайн контейнеров	524
16.2.1. Специализированные контейнеры и итераторы	524
16.2.2. Контейнеры с общим базовым классом.	527
16.2.3. Контейнеры STL.	531
16.3. Контейнер типа <code>vector</code>	533
16.3.1. Типы	533
16.3.2. Итераторы.	535
16.3.3. Доступ к элементам	536
16.3.4. Конструкторы	538
16.3.5. Стековые операции	541
16.3.6. Операции над векторами, характерные для списков.	543
16.3.7. Адресация элементов	546
16.3.8. Размер и емкость	547
16.3.9. Другие функции-члены	549
16.3.10. Вспомогательные функции (helper functions).	550
16.3.11. Специализация <code>vector<bool></code>	550
16.4. Советы	551
16.5. Упражнения	552

Глава 17. Стандартные контейнеры	555
17.1. Стандартные контейнеры	555
17.1.1. Обзор контейнерных операций	556
17.1.2. Краткий обзор контейнеров	559
17.1.3. Внутреннее представление	560
17.1.4. Требования к элементам контейнеров	561
17.1.4.1. Операция сравнения "<"	562
17.1.4.2. Другие операции сравнения	564
17.2. Последовательные контейнеры	565
17.2.1. Контейнер vector	565
17.2.2. Контейнер list	565
17.2.2.1. Операции splice(), sort() и merge()	566
17.2.2.2. «Головные» операции	568
17.2.2.3. Другие операции	568
17.2.3. Контейнер deque	570
17.3. Адаптеры последовательных контейнеров	570
17.3.1. Стек	571
17.3.2. Очередь	572
17.3.3. Очередь с приоритетом	574
17.4. Ассоциативные контейнеры	576
17.4.1. Ассоциативный массив map	576
17.4.1.1. Типы	577
17.4.1.2. Итераторы	577
17.4.1.3. Индексация	579
17.4.1.4. Конструкторы	581
17.4.1.5. Сравнения	581
17.4.1.6. Специфические для контейнера map операции	582
17.4.1.7. Операции, характерные для списков	584
17.4.1.8. Другие функции	586
17.4.2. Ассоциативный контейнер multimap	587
17.4.3. Ассоциативный контейнер set	588
17.4.4. Ассоциативный контейнер multiset	588
17.5. «Почти контейнеры»	589
17.5.1. Строки типа string	589
17.5.2. Массивы valarray	589
17.5.3. Битовые поля bitset	589
17.5.3.1. Конструкторы	590
17.5.3.2. Побитовые операции	591
17.5.3.3. Прочие операции	592
17.5.4. Встроенные массивы	593
17.6. Создание нового контейнера	594
17.6.1. Контейнер hash_map	594
17.6.2. Представление и конструирование	596
17.6.2.1. Поиск	598
17.6.2.2. Операции erase() и resize()	599
17.6.2.3. Хэширование	600
17.6.3. Другие хэшированные ассоциативные контейнеры	601
17.7. Советы	602
17.8. Упражнения	603

Глава 18. Алгоритмы и классы функциональных объектов	607
18.1. Введение	607
18.2. Обзор алгоритмов стандартной библиотеки	608
18.3. Диапазоны (интервалы) и контейнеры.	613
18.3.1. Входные диапазоны	614
18.4. Классы функциональных объектов	615
18.4.1. Базовые классы функциональных объектов.	616
18.4.2. Предикаты.	617
18.4.2.1. Обзор предикатов	618
18.4.3. «Арифметические» функциональные объекты.	619
18.4.4. Адаптеры («связывающие», для адаптирования функций-членов и указателей на функции, «отрицающие»)	620
18.4.4.1. «Связывающие» адаптеры	621
18.4.4.2. Адаптирование функций-членов.	623
18.4.4.3. Версии адаптеров для указателей на функции	624
18.4.4.4. «Отрицатели»	625
18.5. Немодифицирующие алгоритмы.	626
18.5.1. Алгоритм <code>for_each()</code>	626
18.5.2. Алгоритмы поиска	627
18.5.3. Алгоритмы <code>count()</code> и <code>count_if()</code>	629
18.5.4. Алгоритмы <code>equal()</code> и <code>mismatch()</code>	630
18.5.5. Поисковые алгоритмы	631
18.6. Модифицирующие алгоритмы.	632
18.6.1. Копирующие алгоритмы	632
18.6.2. Алгоритм <code>transform()</code>	634
18.6.3. Алгоритм <code>unique()</code>	636
18.6.3.1. Критерии сортировки	637
18.6.4. Алгоритмы замены элементов	638
18.6.5. Алгоритмы удаления элементов	640
18.6.6. Алгоритмы <code>fill()</code> и <code>generate()</code>	640
18.6.7. Алгоритмы <code>reverse()</code> и <code>rotate()</code>	641
18.6.8. Обмен элементов последовательностей-местами	642
18.7. Сортировка последовательностей	643
18.7.1. Сортировка	643
18.7.2. Бинарный поиск.	644
18.7.3. Слияние (алгоритмы семейства <code>merge</code>)	645
18.7.4. Разбиение элементов на две группы (алгоритмы семейства <code>partition</code>)	646
18.7.5. Операции над множествами	646
18.8. «Кучи»	648
18.9. Нахождение минимума и максимума	648
18.10. Перестановки (<code>permutations</code>)	650
18.11. Алгоритмы в C-стиле	650
18.12. Советы	651
18.13. Упражнения	652
Глава 19. Итераторы и аллокаторы	655
19.1. Введение	655
19.2. Итераторы и последовательности	656
19.2.1. Операции над итераторами.	656

19.2.2. Шаблон <code>iterator_traits</code>	658
19.2.3. Категории итераторов	660
19.2.4. Итераторы для вставок	662
19.2.5. Обратные итераторы	663
19.2.6. Поточковые итераторы	664
19.2.6.1. Буфера потоков	666
19.3. Итераторы с проверкой	668
19.3.1. Исключения, контейнеры и алгоритмы	673
19.4. Аллокаторы (распределители памяти)	674
19.4.1. Стандартный аллокатор	674
19.4.2. Пользовательский аллокатор	678
19.4.3. Обобщенные аллокаторы	680
19.4.4. Неинициализированная память	682
19.4.5. Динамическая память	684
19.4.6. Выделение памяти в стиле языка C	685
19.5. Советы	686
19.6. Упражнения	687
Глава 20. Строки	689
20.1. Введение	689
20.2. Символы	690
20.2.1. Шаблон <code>char_traits</code>	690
20.3. Стандартный строковый шаблон <code>basic_string</code>	692
20.3.1. Типы	693
20.3.2. Итераторы	694
20.3.3. Доступ к элементам (символам)	695
20.3.4. Конструкторы	695
20.3.5. Ошибки	697
20.3.6. Присваивание	698
20.3.7. Преобразование в C-строку	699
20.3.8. Сравнения	701
20.3.9. Вставка	703
20.3.10. Конкатенация	704
20.3.11. Поиск	705
20.3.12. Замена	706
20.3.13. Подстроки	707
20.3.14. Размер и емкость	708
20.3.15. Операции ввода/вывода	709
20.3.16. Обмен строк	709
20.4. Стандартная библиотека языка C	710
20.4.1. C-строки	710
20.4.2. Классификация символов	712
20.5. Советы	713
20.6. Упражнения	714
Глава 21. Потоки	717
21.1. Введение	717
21.2. Вывод	719
21.2.1. Потоки вывода	720

21.2.2. Вывод встроенных типов	722
21.2.3. Вывод пользовательских типов	724
21.2.3.1. Виртуальные функции вывода	725
21.3. Ввод	726
21.3.1. Потоки ввода	726
21.3.2. Ввод встроенных типов	727
21.3.3. Состояние потока	729
21.3.4. Ввод символов	731
21.3.5. Ввод пользовательских типов	734
21.3.6. Исключения	735
21.3.7. Связывание потоков	737
21.3.8. Часовые (sentries)	738
21.4. Форматирование	739
21.4.1. Состояние форматирования	739
21.4.1.1. Копирование состояния форматирования	741
21.4.2. Вывод целых	741
21.4.3. Вывод значений с плавающей запятой	742
21.4.4. Поля вывода	743
21.4.5. Выравнивание полей	744
21.4.6. Манипуляторы	745
21.4.6.1. Манипуляторы, принимающие аргументы	746
21.4.6.2. Стандартные манипуляторы ввода/вывода	747
21.4.6.3. Манипуляторы, определяемые пользователем	749
21.5. Файловые и строковые потоки	751
21.5.1. Файловые потоки	752
21.5.2. Заккрытие потоков	753
21.5.3. Строковые потоки	755
21.6. Буферирование	756
21.6.1. Потоки вывода и буферы	757
21.6.2. Потоки ввода и буферы	758
21.6.3. Потоки и буферы	759
21.6.4. Буфера потоков	760
21.7. Локализация (интернационализация)	764
21.7.1. Функции обратного вызова для потоков	766
21.8. Ввод/вывод языка C	766
21.9. Советы	770
21.10. Упражнения	771
Глава 22. Классы для математических вычислений	775
22.1. Введение	775
22.2. Числовые пределы	776
22.2.1. Макросы для предельных значений	778
22.3. Стандартные математические функции	778
22.4. Векторная арифметика	780
22.4.1. Конструкторы класса <code>valarray</code>	780
22.4.2. Индексирование и присваивание в классе <code>valarray</code>	782
22.4.3. Функции-члены	783
22.4.4. Внешние операции и функции	786
22.4.5. Срезы	786

22.4.6. Массив <code>slice_array</code>	789
22.4.7. Временные объекты, копирование, циклы	793
22.4.8. Обобщенные срезы	796
22.4.9. Маски (тип <code>mask_array</code>)	797
22.4.10. Тип <code>indirect_array</code>	798
22.5. Комплексная арифметика	798
22.6. Обобщенные численные алгоритмы	800
22.6.1. Алгоритм <code>accumulate()</code>	801
22.6.2. Алгоритм <code>inner_product()</code>	802
22.6.3. Приращения (<code>incremental changes</code>)	803
22.7. Случайные числа	804
22.8. Советы	806
22.9. Упражнения	807

Часть IV. Проектирование с использованием C++ 809

Глава 23. Общий взгляд на разработку программ. Проектирование 811

23.1. Обзор	811
23.2. Введение	812
23.3. Цели и средства	815
23.4. Процесс разработки	818
23.4.1. Цикл разработки	820
23.4.2. Цели проектирования	822
23.4.3. Этапы проектирования	824
23.4.3.1. Этап 1: отражение концепций классами	825
23.4.3.2. Этап 2: определение операций	828
23.4.3.3. Этап 3: выявление зависимостей	830
23.4.3.4. Этап 4: определение интерфейсов	830
23.4.3.5. Реорганизация иерархии классов	831
23.4.3.6. Использование модельных образцов	832
23.4.4. Экспериментирование и анализ	834
23.4.5. Тестирование	836
23.4.6. Сопровождение и поддержка программ	837
23.4.7. Эффективность	837
23.5. Отдельные аспекты управления проектами	838
23.5.1. Повторное использование кода	838
23.5.2. Масштаб	840
23.5.3. Личности	841
23.5.4. Гибридное проектирование	843
23.6. Аннотированная библиография	845
23.7. Советы	847

Глава 24. Проектирование и программирование 849

24.1. Обзор	849
24.2. Проектирование и язык программирования	850
24.2.1. Отказ от классов	852
24.2.2. Отказ от производных классов и виртуальных функций	854
24.2.3. Игнорирование возможностей статической проверки типов	854
24.2.4. Отказ от традиционного программирования	857
24.2.5. Применение исключительно классовых иерархий наследования	859

24.3. Классы	860
24.3.1. Что представляют собой классы	861
24.3.2. Иерархии классов	862
24.3.2.1. Зависимости внутри иерархии классов	865
24.3.3. Агрегация (отношение включения)	867
24.3.4. Агрегация и наследование	869
24.3.4.1. Альтернатива «включение/наследование»	871
24.3.4.2. Альтернатива «агрегация/наследование»	873
24.3.5. Отношение использования	874
24.3.6. Программируемые отношения	875
24.3.7. Отношения внутри класса	877
24.3.7.1. Инварианты	877
24.3.7.2. Утверждения	879
24.3.7.3. Предусловия и постусловия	882
24.3.7.4. Инкапсуляция	883
24.4. Компоненты	884
24.4.1. Шаблоны	886
24.4.2. Интерфейсы и реализации	888
24.4.3. «Жирные» интерфейсы	890
24.5. Советы	893
Глава 25. Роли классов	895
25.1. Разновидности классов	895
25.2. Конкретные классы (типы)	896
25.2.1. Многократное использование конкретных типов	899
25.3. Абстрактные типы	900
25.4. Узловые классы	903
25.4.1. Изменение интерфейсов	905
25.5. Операции	908
25.6. Интерфейсные классы	909
25.6.1. Подгонка интерфейсов	912
25.7. Дескрипторные классы (handles)	914
25.7.1. Операции в дескрипторных классах	917
25.8. Прикладные среды разработки (application frameworks)	918
25.9. Советы	920
25.10. Упражнения	921
Приложения и предметный указатель	923
Приложение А. Грамматика	925
А.1. Введение	925
А.2. Ключевые слова	926
А.3. Лексические соглашения	927
А.4. Программы	930
А.5. Выражения	930
А.6. Операторы	934
А.7. Объявления	935
А.7.1. Деклараторы	938
А.8. Классы	940
А.8.1. Производные классы	941
А.8.2. Особые функции-члены	941

A.8.3. Перегрузка	942
A.9. Шаблоны	942
A.10. Обработка исключений	943
A.11. Директивы препроцессора	944
Приложение В. Совместимость	947
В.1. Введение	947
В.2. Совместимость С и C++.	948
В.2.1. «Тихие» отличия	948
В.2.2. Код на С, не являющийся C++-кодом	948
В.2.3. Нежелательные особенности	951
В.2.4. Код на C++, не являющийся кодом на С	951
В.3. Старые реализации C++.	953
В.3.1. Заголовочные файлы	954
В.3.2. Стандартная библиотека	955
В.3.3. Пространства имен	955
В.3.4. Ошибки выделения памяти	956
В.3.5. Шаблоны.	956
В.3.6. Инициализаторы в операторах for.	958
В.4. Советы	958
В.5. Упражнения	960
Приложение С. Технические подробности	961
С.1. Введение и обзор	961
С.2. Стандарт	961
С.3. Символьные наборы.	963
С.3.1. Ограниченные наборы символов	963
С.3.2. Ескаре-символы	964
С.3.3. Расширенные символьные наборы	965
С.3.4. Знаковые и беззнаковые символы.	966
С.4. Типы целых литералов.	967
С.5. Константные выражения	967
С.6. Неявное преобразование типов	967
С.6.1. Продвижения (promotions)	968
С.6.2. Преобразования	968
С.6.2.1. Интегральные преобразования	968
С.6.2.2. Преобразования чисел с плавающей запятой	969
С.6.2.3. Преобразования указателей и ссылок.	969
С.6.2.4. Преобразования указателей на члены классов	970
С.6.2.5. Преобразования в логический тип	970
С.6.2.6. Преобразования «значение интегрального типа — значение с плавающей запятой».	970
С.6.3. Обычные арифметические преобразования	971
С.7. Многомерные массивы	971
С.7.1. Векторы	971
С.7.2. Массивы	973
С.7.3. Передача многомерных массивов в функции	974
С.8. Экономия памяти	975
С.8.1. Битовые поля	976
С.8.2. Объединения	977
С.8.3. Объединения и классы	979

С.9. Управление памятью	979
С.9.1. Автоматическая сборка мусора	980
С.9.1.1. Замаскированные указатели	980
С.9.1.2. Операция delete	981
С.9.1.3. Деструкторы	982
С.9.1.4. Фрагментация памяти	982
С.10. Пространства имен	983
С.10.1. Удобство против безопасности	983
С.10.2. Вложенные пространства имен	984
С.10.3. Пространства имен и классы	985
С.11. Управление режимами доступа	985
С.11.1. Доступ к членам класса	985
С.11.2. Доступ к базовым классам	986
С.11.3. Доступ ко вложенным классам	988
С.11.4. Отношение «дружбы»	989
С.12. Указатели на члены классов	989
С.13. Шаблоны	990
С.13.1. Статические члены	990
С.13.2. Друзья	991
С.13.3. Шаблоны в качестве параметров шаблонов	992
С.13.4. Логический вывод аргументов функциональных шаблонов	992
С.13.5. Шаблоны и ключевое слово <code>typename</code>	993
С.13.6. Ключевое слово <code>template</code> в качестве квалификатора	995
С.13.7. Конкретизация	995
С.13.8. Связывание имен	996
С.13.8.1. Зависимые имена	997
С.13.8.2. Связывание в точке определения	999
С.13.8.3. Связывание в точке конкретизации	1000
С.13.8.4. Шаблоны и пространства имен	1001
С.13.9. Когда нужны специализации	1003
С.13.9.1. Конкретизация шаблона функции	1003
С.13.10. Явная конкретизация	1004
С.14. Советы	1005
Приложение D. Локализация	1007
D.1. Национальные особенности	1007
D.1.1. Программирование национальных особенностей	1008
D.2. Класс <code>locale</code>	1011
D.2.1. Локальные контексты с заданными именами	1013
D.2.1.1. Конструирование новых объектов локализации	1015
D.2.2. Копирование и сравнение контекстов локализации	1017
D.2.3. Контексты <code>global()</code> и <code>classic()</code>	1018
D.2.4. Сравнение строк	1019
D.3. Фасеты	1020
D.3.1. Доступ к фасетам класса <code>locale</code>	1021
D.3.2. Простой пользовательский фасет	1023
D.3.3. Использование локализаций и фасетов	1026
D.4. Стандартные фасеты	1026
D.4.1. Сравнение строк	1029
D.4.1.1. Именованные фасеты сравнения	1032
D.4.2. Ввод и вывод чисел	1033
D.4.2.1. Пунктуация чисел	1033

D.4.2.2. Вывод чисел	1035
D.4.2.3. Ввод чисел	1038
D.4.3. Ввод и вывод финансовой информации	1039
D.4.3.1. Пунктуация денежных величин	1040
D.4.3.2. Вывод денежных величин	1043
D.4.3.3. Ввод денежных величин	1044
D.4.4. Ввод и вывод дат и времени	1046
D.4.4.1. Часы и таймеры	1046
D.4.4.2. Класс Date	1049
D.4.4.3. Вывод дат и времени	1049
D.4.4.4. Ввод дат и времени	1052
D.4.4.5. Более гибкий класс Date	1054
D.4.4.6. Задание формата даты	1056
D.4.4.7. Фасет ввода даты	1058
D.4.5. Классификация символов	1063
D.4.5.1. Вспомогательные шаблоны функций	1066
D.4.6. Преобразование кодов символов	1067
D.4.7. Сообщения	1071
D.4.7.1. Использование сообщений из других фасетов	1073
D.5. Советы	1075
D.6. Упражнения	1075
Приложение Е. Исключения и безопасность стандартной библиотеки	1077
E.1. Введение	1077
E.2. Исключения и безопасность	1079
C.3. Технологии реализации безопасности при исключениях	1083
E.3.1. Простой вектор	1083
E.3.2. Явное управление памятью	1087
E.3.3. Присваивание	1088
E.3.4. Метод push_back()	1091
E.3.5. Конструкторы и инварианты	1093
E.3.5.1. Применение функции init()	1095
E.3.5.2. Полагаемся на действительное состояние по умолчанию	1096
E.3.5.3. Отложенное выделение ресурсов	1097
E.4. Гарантии стандартных контейнеров	1098
E.4.1. Вставка и удаление элементов	1099
E.4.2. Гарантии и компромиссы	1102
E.4.3. Функция swap()	1105
E.4.4. Инициализация и итераторы	1106
E.4.5. Ссылки на элементы	1106
E.4.6. Предикаты	1107
E.5. Другие части стандартной библиотеки	1108
E.5.1. Строки	1108
E.5.2. Потoki	1109
E.5.3. Алгоритмы	1109
E.5.4. Типы valarray и complex	1110
E.5.5. Стандартная библиотека языка C	1110
E.6. Рекомендации пользователям стандартной библиотеки	1110
E.7. Советы	1113
E.8. Упражнения	1114
Предметный указатель	1117

Предисловие переводчика и редактора

Предыдущий перевод на русский язык книги Бьерна Страуструпа *Язык программирования C++* был выполнен в 2000 году. В течение десяти лет книга Страуструпа на русском языке неизменно пользовалась большим успехом у читателей. Книга от создателя языка C++ заслуженно считается классикой жанра во всем мире. Учитывая столь высокую роль книги, издательство Бином приняло решение о выполнении нового перевода.

Настоящий перевод 2010 года значительно отличается от перевода 2000 года в нескольких отношениях.

Во-первых, за счет объединения функций переводчика и редактора перевода, а также за счет того, что переводчик сам является автором большого печатного труда на ту же тему, удалось получить более гладкий текст на русском языке, ибо объективная сложность работы, которая стояла перед переводчиками и редакторами первого перевода на русский язык сказалась на некоторой тяжеловесности построения фраз, иногда затуманивающей исходный смысл высказываний автора.

Во-вторых, в рамках первого перевода было допущено несколько ошибок, выявить и исправить которые в переводе 2010 года не составило большого труда из-за близкого знакомства переводчика с данной предметной областью.

В-третьих, были учтены найденные за несколько лет ошибки самого автора — Бьерна Страуструпа, обнаруженные им и читателями и исправленные на сайте поддержки книги http://www.research.att.com/~bs/3rd_errata.html.

В-четвертых, за десять лет, прошедшие с момента выполнения первого русского перевода, постепенно изменялась и уточнялась система русской терминологии, причем процесс этот спонтанно отражался в учебной литературе, и его статистически среднее направление принято нами за основу при выполнении нового перевода.

В заключение, я надеюсь на то, что новый перевод позволит точнее донести содержание замечательной книги Бьерна Страуструпа до русскоязычных читателей. Свои замечания касательного перевода вы можете присылать редактору перевода 2010 года по адресу info@binom-press.ru.

Мартынов Н.Н.
10 февраля 2010 года, Москва

Предисловие автора к третьему русскому изданию

В августе 1998 года был ратифицирован стандарт языка C++ (ISO/IEC 14882 «Standard for the C++ Programming Language», результат голосования национальных комитетов по стандартизации: 22–0). Это событие знаменует новую эру стабильности и процветания C++, а также связанных с языком инструментальных средств и приемов программирования.

Лично для меня главным является то, что стандартный C++ лучше чем любая его предыдущая версия соответствует моему замыслу C++. Стандартный C++ и его стандартная библиотека позволяют мне писать лучшие, более элегантные и более эффективные программы, чем те, что я мог писать в прошлом.

Чтобы добиться этого, я и сотни других людей долго и интенсивно работали в комитетах по стандартизации ANSI и ISO. Целью этих усилий было описать язык и библиотеку, которые будут исправно служить всем пользователям языка, не предоставляя каких-либо преимуществ одной группе пользователей, компании или стране перед остальными. Процесс стандартизации был открыт, справедлив, нацелен на качество и согласие.

Усилия по стандартизации были инициированы Дмитрием Ленковым (Dmitry Lenkov) из Hewlett-Packard, который работал в качестве первого председателя комитета. Во время завершения работ над стандартом председателем был Стив Кламаг (Steve Clamage) из Sun. Джонатан Шапиро (Jonathan Shapiro) и Эндрю Кениг (Andrew Koenig) (оба из AT&T) были редакторами, которые — с помощью многих членов комитета — подготовили текст стандарта, основанный на моем исходном справочном руководстве по C++.

Открытый и демократичный процесс стандартизации потенциально таит в себе одну опасность: результат может оказаться «комитетской разработкой». В случае с C++ этого по большей части удалось избежать. Во-первых, я состоял председателем рабочей группы по расширению языка. В этом качестве я оценивал все основные предложения и составлял окончательные варианты тех из них, которые я, рабочая группа и комитет считали стоящими и в тоже время реализуемыми. Таким образом, комитет в основном обсуждал направляемые ему относительно завершённые проекты, а не занимался разработкой таковых. Во-вторых, главная из новых составляющих стандартной библиотеки — стандартная библиотека шаблонов STL, предоставляющая общую, эффективную, типобезопасную и расширяемую среду контейнеров, итераторов и алгоритмов — была, в основном, результатом работы одного человека, Александра Степанова (Alexander Stepanov) из Hewlett-Packard.

Важно, что стандарт C++ — это не просто бумажка. Он уже встроен в наиболее свежие реализации C++. Большинство основных реализаций поддерживают стандарт (с очень немногочисленными исключениями), причем все обещают полное соответствие в течение ближайших месяцев. Чтобы стимулировать честность производителей, две компании предлагают пакеты, проверяющие «стандартность» реализации C++.

Итак, код, который я пишу сейчас, использует, если это необходимо, большинство возможностей, предлагаемых стандартным C++ и описанных в данном третьем издании «Языка программирования C++».

Улучшения в языке C++ и расширение стандартной библиотеки заметно изменили то, как я пишу код. Мои теперешние программы короче, яснее и эффективнее прежних, что является прямым результатом лучшей, более ясной и систематичной поддержки абстракций в стандартном C++. Улучшенная поддержка таких средств, как шаблоны и исключения, сокращают потребность в более низкоуровневых и запутанных возможностях языка. Поэтому я считаю, что стандартный C++ проще в использовании, чем его предшествующие варианты, где мне приходилось имитировать средства, которые теперь поддерживаются непосредственно.

Основываясь на приведенных наблюдениях, я решил, что необходимо полностью переписать второе издание «Языка программирования C++». Никак иначе нельзя было должным образом отразить новые возможности, предлагаемые языком, и поддерживаемые ими приемы программирования и проектирования. В результате 80% материала третьего издания добавлено по сравнению со вторым. Изменилась и организация книги — теперь больший упор делается на технику и стиль программирования. Рассматриваемые изолированно, особенности языка скучны — они оживают лишь в контексте приемов программирования, ради поддержки которых и задумывались.

Соответственно, C++ может теперь преподаваться как язык более высокого уровня. То есть основное внимание можно уделять алгоритмам и контейнерам, а не жонглированию битами, объединениями, строками в стиле C, массивами и проч. Более низкоуровневые понятия (такие как массивы, нетривиальное использование указателей и приведения типов), естественно, также придется изучить. Но их представление теперь удастся отложить до тех пор, пока новичок в программировании на C++, читатель или студент не обретут зрелость, необходимую, чтобы воспринимать эти средства в контексте более высокоуровневых концепций, к применению которых обучаемые уже привыкли.

В частности, невозможно переусердствовать в подчеркивании преимущества статически типобезопасных строк и контейнеров перед стилями программирования, предполагающими использование множества макросов и приведений типов. В третьем издании «Языка программирования C++» мне удалось избавиться от почти всех макросов и свести использование приведений типов к немногочисленным случаям, когда они действительно существенны. Я считаю серьезным недостатком принятую в C/C++ форму макросов, которую теперь можно считать устаревшей благодаря наличию более подходящих средств языка, таких как шаблоны, пространства имен, встроенные функции и константы. Точно также, широкое использование приведений типов в любом языке сигнализирует о плохом проектировании. Как макросы, так и приведения являются частыми источниками ошибок.

Тот факт, что без них можно обойтись, делает программирование на C++ куда более безопасным и элегантным.

Стандарт C++ предназначен для того, чтобы изменить наш способ программировать на C++, проектировать программы и обучать программированию на C++. Подобные изменения не происходят за один вечер. Я призываю вас не торопясь, внимательно присмотреться к стандарту C++, к приемам проектирования и программирования, используемым в третьем издании «Языка программирования C++», — и к вашему собственному стилю программирования. Предполагаю, что вполне возможны значительные улучшения. Но пусть ваша голова остается холодной! Чудес не бывает — при написании кода опасно использовать языковые возможности и приемы, которые вы понимаете лишь частично. Настало время изучать и экспериментировать — только поняв новые концепции и приемы, можно воспользоваться по-настоящему существенными преимуществами стандартного C++.

Добро пожаловать!
Бьерн Страуструп

Предисловие

*Программирование есть понимание.
— Кристин Нюгард*

Применение языка C++ доставляет мне все большее удовольствие. За прошедшие годы средства поддержки проектирования и программирования на C++ кардинально улучшились, а также было предложено множество полезных и удобных методик их использования. Но речь идет не только об удовольствии. Практикующие программисты достигли значительного повышения производительности, уровня поддержки, гибкости и качества в проектах любого уровня сложности и масштаба. К настоящему времени C++ оправдал большую часть возлагавшихся на него надежд, а кроме того он преуспел в решении задач, о которых я даже и не мечтал.

Эта книга представляет стандарт C++¹ и ключевые методы программирования и проектирования, поддерживаемые этим языком. Стандарт C++ намного мощнее и точнее, чем то, что излагалось в первом издании данной книги. Новые языковые черты, такие как пространства имен, исключения, шаблоны и механизм динамического определения типов, позволяют реализовывать многие методики программирования более непосредственно, чем это было возможно ранее. Стандартная библиотека предоставляет программистам возможность стартовать с существенно более высокоуровневых конструкций чем те, что доступны в «голом» языке.

Всего лишь треть информации из первого издания переключалась во второе издание данной книги. Третье издание переработано еще более интенсивно. В ней есть кое-что, что будет полезно самым опытным программистам на C++. В то же время, она более доступна новичкам, чем предыдущие издания этой книги. Это стало возможным благодаря взрывному росту практического использования C++ и опыту, вытекающему из этой практики.

Стандартная библиотека позволяет по-новому представить основные концепции языка C++. Как и предыдущие издания, настоящая книга описывает язык C++ независимо от конкретных реализаций. Как и в предыдущих изданиях, все конструкции и концепции излагаются «снизу-вверх», так что к тому моменту, когда они начинают использоваться, они уже представлены и подробно описаны. Отметим, что начать использовать хорошо спроектированную библиотеку можно прежде, чем читатели смогут изучить ее внутреннее устройство. Кроме того, стандартная библиотека сама по себе является ценнейшим источником примеров программирования и методов проектирования.

В настоящей книге рассматриваются все основные средства языка и стандартной библиотеки. Ее материал организован вокруг этих основных средств, однако фокус изложения направлен на применение средств языка как инструмента проектирова-

¹ ISO/IEC 14882:2003(E), Стандарт языка программирования C++

ния и программирования, а не на формальные конструкции как таковые. Книга демонстрирует ключевые методики, делающие C++ эффективным, и излагает эти методики так, чтобы их постижение повышало мастерство программиста. За исключением глав, иллюстрирующих технические детали, практические примеры кода взяты из области системного программирования. Дополнением к данной книге может служить исчерпывающий справочник *The Annotated C++ Language Standard*.

Основной целью данной книги является стремление помочь читателю понять, как средства языка C++ непосредственно поддерживают ключевые методики программирования. Задача ставится так, чтобы подвигнуть читателя отойти как можно дальше от простого копирования примеров кода и эмулирования методик, присущих другим языкам программирования. Только отличное понимание идей, стоящих за средствами языка, ведет к настоящему мастерству. Дополненная деталями реализации информация из настоящей книги является достаточной для выполнения реальных проектов заметного масштаба. Я надеюсь, что эта книга поможет читателю по-новому взглянуть на программирование на языке C++ и стать более квалифицированным программистом и проектировщиком.

Благодарности

Помимо людей, упомянутых в соответствующих разделах предыдущих изданий книги, я хотел бы поблагодарить еще и следующих: Matt Austern, Hans Boehm, Don Caldwell, Lawrence Cowl, Alan Feuer, Andrew Forrest, David Gay, Tim Griffin, Peter Juhl, Brian Kernighan, Andrew Koenig, Mike Mowbray, Rob Murray, Lee Nackman, Joseph Newcomer, Alex Stepanov, David Vandevoorde, Peter Weinberger и Chris Van Wyk — за комментирование отдельных глав третьего издания. Без их помощи текст книги был бы менее понятен, содержал бы больше ошибок, был бы менее полным, а книга была бы тоньше.

Я также хотел бы поблагодарить добровольцев из Комитета по стандартизации C++ за их титанический труд, сделавший C++ тем, чем он сейчас является. Несправедливо здесь выделять кого-либо, но еще более несправедливо было бы не упомянуть следующих: Mike Ball, Dag Bruck, Sean Corfield, Ted Goldstein, Kim Knuttila, Andrew Koenig, Jose Lajoie, Dmitry Lenkov, Nathan Myers, Martin O'Riordan, Tom Plum, Jonathan Shopiro, John Spicer, Jerry Schwarz, Alex Stepanov и Mike Vilot — каждый из них работал со мной над отдельными частями C++ или стандартной библиотеки.

После выхода первого тиража этой книги десятки людей присылали мне исправления и предложения по ее улучшению. Многие из этих предложений я учел при подготовке более поздних тиражей, от чего они только выиграли. Переводчики этой книги на иностранные языки также высказывали свои соображения относительно пояснения отдельных моментов. Отвечая на просьбы читателей я добавил приложения D и E. Особо хотел бы при этом поблагодарить следующих людей: Dave Abrahams, Matt Austern, Jan Bielawski, Janina Mincer Daszkiewicz, Andrew Koenig, Dietmar Kuhl, Nicolai Josuttis, Nathan Myers, Paul E. Sevin, Andy Tenne-Sens, Shoichi Uchida, Ping-Fai (Mike) Yang и Dennis Yelle.

Предисловие ко второму изданию

*А дорога бежит и бежит.
— Бильбо Бэггинс*

Как было предсказано в первом издании этой книги, С++ эволюционировал в соответствии с запросами его пользователей. Эта эволюция направлялась запросами пользователей самой разной квалификации, работающих в разных предметных областях. Сообщество программистов на С++ выросло в сотни раз за шесть лет, прошедшие с момента выхода первого издания этой книги. За это время были разработаны многочисленные методики, усвоены полезные практические уроки, приобретен бесценный опыт. Часть этого опыта отражена в настоящем издании.

Главной целью развития и расширения языка за эти шесть лет было стремление сделать С++ языком абстракции данных и языком объектно-ориентированного программирования в целом, языком, пригодным для написания высококачественных библиотек, в том числе библиотек пользовательских типов. Высококачественная библиотека — это библиотека, которая предоставляет пользователю концепцию в виде одного или нескольких классов, которые удобны в применении, безопасны и эффективны. Здесь безопасность означает предоставление пользователю интерфейса к содержимому библиотеки, безопасному в отношении типов. Эффективность означает, что классовая структура библиотеки не налагает дополнительной нагрузки на процессор и память по сравнению с ручным кодом, написанным на С.

В настоящей книге приводится полное описание языка С++. В главах 1-10 сосредоточен вводный учебный материал. Главы 11-13 посвящены обсуждению вопросов проектирования и реализации; наконец, далее приведено полное справочное руководство по С++. Естественно, в настоящей книге присутствуют новые черты и свойства языка, появившиеся после выхода первого издания книги: уточненный механизм разрешения перегрузки, средства управления памятью, безопасный по типам механизм компоновки, константные и статические члены классов, абстрактные классы, множественное наследование, шаблоны, обработка исключений.

С++ является языком общего назначения; его естественной сферой применения является системное программирование в самом общем смысле этого термина. Тем не менее, С++ с успехом используется и в других предметных областях. Реализации С++ имеются и для маломощных микрокомпьютеров, и для мощнейших суперкомпьютерных систем и разных операционных сред. Поэтому настоящая книга описывает собственно язык С++ без его привязки к конкретным операционным системам и их библиотекам.

В книге имеется большое число примеров классов, которые будучи полезными должны быть охарактеризованы как «игрушечные». Такой стиль подачи общих принципов и полезных методик позволяет лучше сфокусировать на них внимание, чем это возможно в контексте больших реальных программ, где они были бы похоронены в многочисленных деталях. Большую часть полезных классов, приведенных в книге, таких как строки, связанные списки, массивы, матрицы, графические классы, ассоциативные массивы и т.д. можно найти в более совершенном виде в разных коммерческих и некоммерческих источниках. Многие из этих промышленных классов являются прямыми или косвенными потомками приведенных здесь игрушечных классов.

По сравнению с предыдущим изданием настоящее больше сфокусировано на обучении, но форма подачи материала по-прежнему ориентирована на опытных программистов и разработчиков, дабы не оскорблять их опыт и квалификацию. Более широкое обсуждение вопросов проектирования отражает спрос на информацию, не замыкающуюся лишь на изучении средств языка и их непосредственного использования. Увеличено число приведенных технических деталей, а также качество их изложения. В частности, справочное руководство по языку отражает многолетний опыт работы в этом направлении. Мне хотелось написать книгу, которую к которой большинство программистов обращалось бы не один раз. Таким образом, данная книга представляет язык C++, его фундаментальные принципы и ключевые технологии, в рамках которых язык используется. Успехов!

Благодарности

Помимо людей, упомянутых в соответствующих разделах первого издания книги, я хотел бы поблагодарить еще и следующих: Al Aho, Steve Buroff, Jim Coplien, Ted Goldstein, Tony Hansen, Lorraine Juhl, Peter Juhl, Brian Kernighan, Andrew Koenig, Bill Leggett, Warren Montgomery, Mike Mowbray, Rob Murray, Jonathan Shopiro, Mike Vilot, and Peter Weinberger — за комментирование отдельных глав второго издания.

Многие люди оказали влияние на развитие C++ с 1985 года по 1991 год. Я могу упомянуть лишь некоторых из них: Andrew Koenig, Brian Kernighan, Doug McIlroy и Jonathan Shopiro. Также большое спасибо всем участникам просмотра и рецензирования черновика справочного руководства и всем людям, пострадавшим в первый год работы комитета X3J16.

Мюррей Хилл, Нью Джерси

Бьерн Страуструп

Предисловие к первому изданию

Язык формирует наше мышление и определяет то, о чем мы можем думать.
— Б. Л. Ворф

C++ является языком общего назначения, цель которого — сделать работу серьезного программиста более приятной. За исключением некоторых деталей, C++ является надмножеством языка программирования C. Помимо возможностей языка C, C++ предоставляет гибкие и эффективные средства для определения новых типов. Программист может сегментировать приложение на фрагменты, определив для этого новые типы, отвечающие концепциям приложения. Такую технику программирования часто называют абстракцией данных. Объекты пользовательских типов содержат информацию, характерную для этих типов. Такие объекты можно с удобством и безопасностью использовать в контекстах, когда их тип неизвестен на этапе компиляции. Программы, использующие подобного рода объекты, часто называют объектными. При надлежащем применении такая техника ведет к более коротким программам, более понятным и более удобным в сопровождении.

Ключевой концепцией языка C++ является класс. Класс — это тип, определяемый пользователем. Классы обеспечивают сокрытие данных, гарантированную инициализацию данных, неявное преобразование между пользовательскими типами, динамическое определение типа, контролируемое управление памятью и механизмы перегрузки операций. C++ гораздо лучше контролирует типы, чем C, а также позволяет достичь более высокой модульности. Он также содержит улучшения, не имеющие прямого отношения к классам, такие как символические константы, встраиваемые функции, аргументы функций по умолчанию, перегрузка имен функций, операции по управлению выделением памяти, а также ссылки. C++ сохраняет возможности языка C по эффективной работе с низкоуровневыми аппаратнозависимыми базовыми типами (битами, байтами, словами, адресами и т.д.). Это позволяет реализовать пользовательские типы с высокой эффективностью.

C++ и его стандартные библиотеки построены с учетом переносимости. Текущая реализация будет работать на большинстве систем, поддерживающих язык C. Библиотеки языка C можно использовать в программах на C++, также как и большинство инструментальных средств языка C.

Данная книга предназначена в первую очередь для того, чтобы серьезные программисты могли изучить C++ и использовать его в нетривиальных разработках. Книга содержит полное описание C++, множество законченных примеров и еще большее количество фрагментов программ.

Благодарности

C++ никогда не достиг бы необходимой зрелости без интенсивного использования, советов и конструктивной критики со стороны друзей и коллег. В частности, Tom Cargill, Jim Coplien, Stu Feldman, Sandy Fraser, Steve Johnson, Brian Kernighan, Bart Locanthi, Doug McIlroy, Dennis Ritchie, Larry Rosler, Jerry Schwarz и Jon Shopiro подали важные для развития языка идеи. Dave Presotto написал текущую реализацию библиотеки потокового ввода/вывода.

Кроме того, сотни людей способствовали развитию языка C++ и его компилятора, присылая мне свои предложения по улучшениям, описания проблем, с которыми они сталкивались, а также сообщения об ошибках компилятора. Я могу отметить здесь лишь некоторых: Gary Bishop, Andrew Hume, Tom Karzes, Victor Milenkovic, Rob Murray, Leonie Rose, Brian Schmult, и Gary Walker.

Множество людей помогали с изданием данной книги. Среди них: Jon Bentley, Laura Eaves, Brian Kernighan, Ted Kowalski, Steve Mahaney, Jon Shopiro и участники семинара по C++ в Bell Labs, Columbus, штат Ohio (Огайо), 26-27 июня 1985 года.

Мюррей Хилл, Нью Джерси

Бьерн Страуструп

Введение

Во введении представлен обзор основных концепций и свойств языка программирования C++ и его стандартной библиотеки. Также поясняется структура книги и подход к изложению средств языка и методов их применения. Кроме того, во вводных главах дана базовая информация о языке C++, архитектуре и практике использования.

Главы

1. Обращение к читателю
2. Обзор языка C++
3. Обзор стандартной библиотеки

«... и ты, Маркус, ты дал мне многое. Теперь я дам тебе хороший совет. Будь многими. Брось играть в бытие одним лишь Маркусом Кокоза. Ты так сильно волновался за Маркуса Кокоза, что стал его рабом и пленником. Ты не делал ничего, не подумав сначала о его благополучии и престиже. Ты всегда боялся того, что Маркус совершит какую-либо глупость или ему станет скучно. А что в этом страшного? Во всем мире люди совершают глупости. Я хочу, чтобы тыл жил легче. Будь больше, чем одним, будь многими людьми, столь многими, сколь можешь представить себе.»

— Карен Бликсен (Karen Blixen)

(«The Dreamers» из книги «Seven Gothic Tales», написанной под псевдонимом Isak Dinesen, Random House, Inc © Isak Dinesen, 1934, обновлено в 1961)

Обращение к читателю

*И молвил Морж:
«Пришла пора поговорить о многом».
— Льюис Кэрролл*

Структура этой книги — как изучать C++ — структура C++ — эффективность и структура — философские замечания — исторические замечания — для чего используется C++ — C и C++ — рекомендации для программистов на C — рекомендации для программистов на C++ — размышления о программировании на C++ — советы — ссылки.

1.1. Структура книги

Книга состоит из шести частей:

- Введение: В главах 1–3 дается обзор языка C++, поддерживаемых им стилей программирования и стандартной библиотеки C++.
- Часть I: В главах 4–9 изучаются встроенные типы языка C++ и базовые средства построения программ.
- Часть II: Главы 10–15 содержат учебный материал по объектно-ориентированному и обобщенному программированию на C++.
- Часть III: В главах 16–22 представлена стандартная библиотека языка C++.
- Часть IV: В главах 23–25 рассматриваются проблемы, связанные с проектированием и разработкой программ.
- Приложения: Приложения A–E содержат технические детали языка C++.

В главе 1 дается обзор книги, рекомендации по ее чтению, а также общие сведения о языке и способах его применения. Вы можете бегло ознакомиться с ее содержанием, задержавшись лишь на интересных местах, а позднее вернуться к ней снова, после прочтения других частей данной книги.

Главы 2 и 3 содержат обзор основных концепций и свойств языка C++ и его стандартной библиотеки. Цель этих глав — обратить ваше внимание на важность

понимания фундаментальных концепций и базовых свойств языка, демонстрируя то, что может быть выражено полным набором языковых средств. По крайней мере, эти главы должны убедить читателя в том, что язык C++ — это не C, и что он прошел большой путь со времени первого и второго изданий этой книги. Глава 2 посвящена знакомству с высокоуровневыми чертами языка C++: поддержкой абстракции данных, объектно-ориентированным и обобщенным программированием. В главе 3 представлены базовые принципы и основные средства стандартной библиотеки. Это позволит мне применять стандартную библиотеку в последующих главах, а вам — использовать в упражнениях, вместо того, чтобы полагаться исключительно на встроенные низкоуровневые средства языка.

Вводные главы демонстрируют основной способ представления материала, принятый в данной книге: для того, чтобы обеспечить предметное и реалистичное изучение конкретного вопроса, я вначале коротко излагаю лишь основы концепции, а подробное и углубленное рассмотрение выполняю позже. Такой подход позволяет мне обращаться к конкретным примерам до того, как будут изучены все детали. Можно сказать, организация книги соответствует положению, что мы лучше обучаемся, если продвигаемся от конкретного к абстрактному — даже там, где абстрактные идеи кажутся в ретроспективе простыми и очевидными.

В части I рассматривается подмножество C++, поддерживающее стили программирования, характерные для языков C или Pascal. Эта часть книги охватывает фундаментальные типы, выражения и управляющие конструкции программ на C++. Модульность в той части, что поддерживается пространствами имен, исходными файлами и обработкой исключений, также рассматривается. Я предполагаю, что вы знакомы с фундаментальными основами программирования, излагаемыми в части I. Поэтому, например, хоть я и рассматриваю рекурсию и итерацию в этой части книги, я не трачу много времени на прояснение вопроса о реальной пользе этих концепций.

Часть II посвящена средствам C++ для определения и использования новых типов. Здесь представлены (глава 10, глава 12) конкретные и абстрактные классы (интерфейсы), а также перегрузка операций (глава 11), полиморфизм и иерархии классов (глава 12, глава 15). В главе 13 представлены шаблоны, то есть средства для определения семейств типов и функций. Она также демонстрирует способы создания контейнеров (например, списков) и приемы обобщенного программирования. В главе 14 рассматриваются обработка исключений, методы обработки ошибок и общая стратегия создания устойчивых и надежных программ. Я предполагаю, что вы или не слишком хорошо знакомы с объектно-ориентированным и обобщенным программированием, или вам не хватает подробных объяснений того, как именно эти базовые абстракции поддерживаются средствами языка C++. Поэтому я рассматриваю не только сами абстракции, но также и технику их применения. В части IV эта тема развивается далее.

Часть III посвящена стандартной библиотеке языка C++. В ней объясняется, как пользоваться библиотекой, каков ее дизайн и техника применения, а также показано, как расширить ее возможности. Библиотека предоставляет контейнеры (такие как *list*, *vector* и *map*; глава 16, глава 17), стандартные алгоритмы (такие как *sort*, *find* и *merge*; глава 18, глава 19), строки (глава 20), ввод/вывод (глава 21) и поддержку вычислений (глава 22).

Часть IV затрагивает вопросы, актуальные для ситуаций, когда C++ используется в проектировании и реализации больших программных систем. В главе 23 рассматрива-

ются вопросы проектирования и управления ходом выполнения проектов. В главе 24 обсуждается связь между языком программирования и приемами проектирования. Глава 25 демонстрирует некоторые способы применения классов в проектировании.

Приложение А содержит грамматику C++ с некоторыми комментариями. В приложении В обсуждается связь между C++ и C, а также между стандартным C++ (называемым также ISO C++ или ANSI C++) и предыдущими его версиями. В приложении С представлены некоторые технические детали языка C++. В приложении D рассматриваются средства стандартной библиотеки, предназначенные для локализации программного обеспечения. В приложении Е обсуждаются вопросы гарантии возбуждения исключений и соответствующие требования стандартной библиотеки.

1.1.1. Примеры и ссылки

В данной книге акцент сделан скорее на организации программ, а не на реализации конкретных алгоритмов. Как следствие, я избегаю «слишком умных» или просто трудных для понимания алгоритмов. Тривиальный алгоритм лучше подходит для иллюстрации языковых элементов или структуры программы. Например, я использую сортировку Шелла, тогда как в реальном коде лучше применить быструю сортировку. Часто в качестве упражнений предлагается переработать учебный код под более подходящий алгоритм. И, наконец, в реальной практике лучше использовать библиотечные функции вместо приведенного в книге иллюстративного кода.

Учебные примеры неизбежно искажают представление о разработке программ, так как упрощение кода примеров одновременно приводит к исчезновению сложностей, связанных с масштабом проблем. Я не представляю другого способа почувствовать, что такое реальное программирование и каков в действительности язык программирования, иначе как в процессе практической реализации программных систем «настоящего размера». В данной книге основное внимание уделяется элементам языка и технике программирования, на основе которых и строятся все программы.

Отбор примеров отражает мой собственный опыт разработки компиляторов, базовых библиотек и программ-симуляторов (моделирование; программы-тренажеры). Примеры являются упрощенными вариантами того, что встречается в реальной практике. Упрощение необходимо, иначе язык программирования и вопросы проектирования затеряются во множестве мелких деталей. В книге нет «особо умных» примеров, не имеющих аналогов в реальных программах. Везде, где только можно, я отношу в приложение С иллюстрирующие технические детали языка примеры, которые используют идентификаторы x или y для переменных, имена A или B для типов, и обозначения $f()$ и $g()$ для функций.

Везде, где возможно, язык и библиотека рассматриваются скорее в контексте их применения, нежели в стиле «сухого справочника». Представленные в книге языковые элементы и степень детальности их описания отражают мои личные представления о том, что действительно нужно для эффективного программирования на C++. Другая моя книга (в соавторстве с Э. Кенигом — Andrew Koenig) — The Annotated C++ Language Standard, служит дополнением к настоящей книге и содержит полное и всестороннее определение языка C++, а также поясняющие комментарии. Логичным было бы еще одно дополнение — The Annotated C++ Standard Library. Но ввиду ограниченности сил и времени я не могу обещать выход такой книги.

Ссылки на разделы данной книги даются в форме §2.3.4 (глава 2, раздел 3, подраздел 4), §B.5.6 (приложение В, подраздел 5.6) и §6.6[10] (глава 6, упражнение 10).

1.1.2. Упражнения

Упражнения размещены в концах глав данной книги. По большей части они формулируются в стиле «напиши программу». Всегда пишите программы, пригодные для успешной компиляции и нескольких тестовых прогонов. Упражнения существенно разнятся по уровню сложности, так что для каждого из них приводится приблизительная условная оценка. Масштаб оценок экспоненциальный, так что если, например, упражнение, оцененное как (*1), занимает 10 минут, то упражнение с оценкой (*2) может занять час, а (*3) — целый день. Кроме того, время, необходимое для выполнения упражнения, зависит больше от вашего опыта, чем от самого упражнения. Например, упражнение с оценкой (*1) может занять и целый день, если вы при этом параллельно знакомитесь с компьютерной системой и выясняете детали работы с ней. А с другой стороны, упражнение с оценкой (*5) может быть решено кем-нибудь за один час, если у него под рукой набор готовых программ и утилит.

Любую книгу по языку С можно использовать в качестве источника дополнительных упражнений к части I. А любая книга по структурам данных и алгоритмам послужит источником упражнений для частей II и III.

1.1.3. Замечания о конкретных реализациях языка (компиляторах)

В книге используется «чистый C++», как он определен в стандарте [C++, 1998], [C++, 2003]. Поэтому примеры из книги должны компилироваться в любых стандартных реализациях. Большинство из них были проверены на нескольких компиляторах. При этом не все компиляторы справились с новейшими элементами языка C++. Однако я не думаю, что есть необходимость в том, чтобы явно называть такие реализации, ибо разработчики непрерывно работают над их улучшением, и эта информация быстро устареет. В приложении В приведены советы по устранению проблем со старыми компиляторами, а также с кодом, написанным для компиляторов языка С.

1.2. Как изучать C++

Изучая C++, нужно сконцентрироваться на концепциях и не потеряться в технических деталях. Цель изучения — стать квалифицированным программистом, который эффективно проектирует и реализует новые разработки, а также умело поддерживает уже существующие. Для этого важнее понять методологию проектирования и программирования, чем исчерпывающе изучить все детали языка — последнее постепенно приходит вместе с опытом.

C++ поддерживает множество стилей программирования. Все они основаны на строгой статической проверке типов, и большинство из них нацелены на достижение высокого уровня абстракции и непосредственного представления идей программиста. При этом каждый из стилей достигает свою цель без снижения эффективности выполнения программы и заметного увеличения потребляемых компью-

терных ресурсов. Программисты, переходящие с иных языков (например, C, Fortran, Smalltalk, Lisp, ML, Ada, Eiffel, Pascal, Modula-2) должны четко осознать: чтобы реально воспользоваться преимуществами языка C++, нужно потратить время, изучить и на практике овладеть стилями и приемами программирования, характерными для C++. То же относится и к программистам, привыкшим к работе с ранними, менее выразительными версиями C++.

Бездумный перенос на новый язык идей и методов программирования, доказавших свою эффективность для другого языка, в типичном случае приводит к неуклюжему, медленному и сложному в сопровождении коду. К тому же писать такой код крайне утомительно, ибо каждая его строчка и каждое сообщение компилятора об ошибке подчеркивают отличие применяемого языка от «старого». Вы можете программировать в стиле Fortran, C, Smalltalk и т.п. на любом языке, но делать это для языка с иной философией и неэкономно, и неприятно. Любой язык является щедрым источником идей о том, как следует писать программы на C++. Однако чтобы эти идеи стали эффективными в новом контексте, их нужно преобразовать в нечто, что хорошо согласуется с общей структурой и системой типов C++. Над базовой системой типов языка программирования возможна лишь пиррова победа.

C++ поддерживает постепенный подход к обучению. Ваш собственный путь изучения нового языка программирования зависит от того, что вы уже знаете, и чему именно хотите научиться. В этом деле нет единого подхода для всех и каждого. Я предполагаю, что вы изучаете C++, чтобы повысить эффективность в проектировании программ и их реализации. То есть я полагаю, что вы не только хотите изучить новый синтаксис, чтобы с ним писать новые программы старыми методами, но в первую очередь стремитесь научиться новым, более продуктивным способам построения программных систем. Это надо делать постепенно, так как приобретение и закрепление новых навыков требует времени и практики. Подумайте, сколько времени обычно занимает качественное овладение иностранным языком, или обучение игре на музыкальном инструменте. Конечно, стать квалифицированным программным архитектором и разработчиком можно быстрее, но не настолько, как многим хотелось бы.

Отсюда следует, что вы будете использовать C++ (зачастую для построения реальных систем) еще до того, как овладеете всеми его нюансами. Поддерживая несколько парадигм программирования (глава 2), C++ обеспечивает возможность продуктивной работы программистов разной квалификации. Каждый новый стиль программирования добавляет новый штрих в ваш совокупный инструментарий, но каждый стиль ценен и сам по себе, так как увеличивает эффективность вашей работы как программиста. Язык C++ организован таким образом, что вы можете изучать его концепции последовательно, постепенно наращивая свои возможности. Это важно, так как ваши возможности будут расти практически пропорционально затраченным на обучение усилиям.

В непрекращающихся дебатах по поводу необходимости (или отсутствия необходимости) предварительного изучения языка C я решительно на стороне тех, кто считает, что лучше сразу приступить к изучению C++: он безопаснее, он более выразителен и он снижает необходимость в низкоуровневом программировании. Легче понять хитроумные «штучки» на C, компенсирующие отсутствие в нем высокоуровневых конструкций, если предварительно ознакомиться с общими возможностями этих языков и некоторыми высокоуровневыми средствами C++.

Приложение В поможет программистам при переходе от C++ к C, например, с целью поддержки ранее созданного кода.

В настоящее время существует несколько независимо разработанных реализаций C++. Масса инструментов, библиотек, графических сред разработки к вашим услугам. Великое множество учебников, руководств, журналов, бюллетеней, электронных досок объявлений, почтовых рассылок, конференций и курсов призваны снабдить вас самой последней информацией о новых реализациях C++, их инструментах, библиотеках и т.д. Если вы планируете использовать C++ самым серьезным образом, я настоятельно рекомендую получить доступ к указанным источникам информации. Так как у каждого из них свой взгляд на C++, то целесообразно обратиться хотя бы к двум таким источникам. Например, см. [Barton, 1994], [Booch, 1994], [Henricson, 1997], [Koenig, 1997], [Martin, 1995].

1.3. Как проектировался C++

Простота была важным критерием при разработке C++ — там, где стоял выбор между простотой языковых конструкций и простотой реализации компилятора, выбиралось первое. Большое внимание уделялось совместимости с языком C [Koenig, 1989], [Stroustrup, 1994], (приложение В), что, однако, помешало «почистить» синтаксис языка C.

В языке C++ нет встроенных высокоуровневых типов и операций. Например, в C++ нет встроенных матриц с операциями их обращения, нет и строк с операциями конкатенации. Если такие типы нужны, то они могут быть созданы средствами языка. Более того, создание типов общего назначения или типов, специфичных для конкретной программы, и есть основная форма деятельности при программировании на C++. Хорошо спроектированный пользовательский тип отличается от встроенного типа только тем, как он определен, а не тем, как он используется. Описанная в части III стандартная библиотека содержит массу примеров таких типов данных. Для пользователя нет большой разницы между встроенными типами и типами стандартной библиотеки.

При проектировании C++ средства, которые замедляют работу программы и требуют дополнительных затрат памяти во всех случаях (даже когда их напрямую не используют), отвергались. Например, не применяются дополнительные внутренние конструкции, требующие индивидуального хранения вместе с каждым объектом. Таким образом, если, например, объявляется структура с двумя 16-битовыми величинами, то объекты этого типа целиком помещаются в 32-битовый регистр.

C++ разрабатывался для той же самой традиционной среды компиляции и исполнения, что была присуща языку C и операционной системы UNIX. Разумеется, C++ не ограничивается операционной системой UNIX; ему просто присущи те же самые модельные взаимоотношения между языком, библиотеками, компиляторами, компоновщиками, средой исполнения и т.д., что свойственны языку C и UNIX. Эта минимальная модель помогла языку C++ успешно работать практически на любой платформе. Тем не менее, целесообразно использовать C++ и в средах с существенно большей поддержкой. Динамическая загрузка, инкрементная компиляция, базы данных определений типов — все это можно с успехом использовать без обратного влияния на сам язык.

В C++ проверка типов и сокрытие данных полагаются на анализ программы во время компиляции, который позволяет в результате избежать непреднамеренной порчи данных. Эти меры не обеспечивают секретности или защиты от того, кто намеренно нарушает правила. Однако их можно использовать свободно, не опасаясь за эффективность программы. Главная идея состоит в том, что свойства языка должны быть не только элегантными, но и приемлемыми для реальных программ.

Систематическое и подробное описание структуры языка C++ см. в [Stroustrup, 1994].

1.3.1. Эффективность и структура

C++ разрабатывался как расширение языка C и за малыми исключениями содержит C в качестве подмножества. Это подмножество, являясь базовой частью языка, демонстрирует тесную связь между его типами, операциями, операторами и объектами, с которыми компьютер работает непосредственно: числами, символами и адресами. В языке C++ все выражения и операторы, за исключением операций `new`, `delete`, `typeid`, `dynamic_cast`, `throw` и `try`-блока, не требуют никакой дополнительной поддержки со стороны среды выполнения.

C++ использует те же самые схемы вызовов функций и последовательности возвратов, что и язык C — или более эффективные. Когда даже столь эффективные механизмы вызовов слишком накладны, C++ обеспечивает возможность прямой подстановки кода функций в точку вызова, так что удастся совместить удобства функциональной декомпозиции и нулевые накладные расходы на функциональные вызовы.

Одной из целей разработки языка C было намерение устранить необходимость в ассемблерном кодировании наиболее требовательных системных задач. При разработке C++ мы постарались по максимуму сохранить это преимущество. Различия между C и C++ заключаются преимущественно в разной степени акцента на типах и структуре. C — выразителен и гибок. C++ — еще более выразителен. Однако чтобы эту выразительность реализовать в собственном коде, требуется уделять большее внимание типам объектов. Зная типы, компилятор корректно обращается с выражениями, тогда как в противном случае вам самим пришлось бы скрупулезно, в мельчайших деталях, определять каждую операцию. Зная типы, компилятор может обнаруживать такие ошибки, которые в противном случае оставались бы в программе до стадии тестирования — или даже дольше. Заметим, что применение системы типов для проверки аргументов вызова функций, для защиты данных от случайного изменения, для предоставления новых типов, для определения операций над новыми типами и т.п. не приводит к дополнительным накладным расходам во время выполнения программы.

Другой акцент C++ — акцент на структуре программы, отражает тот факт, что после разработки языка C произошел резкий рост среднего размера разрабатываемых программ. Небольшую программу (скажем, не более 1000 строк исходного кода) можно «силовым образом» заставить правильно работать, даже нарушая все правила хорошего стиля написания программ. Для больших программ таких простых решений уже нет. Если структура программы из 100000 строк запутана, то вы столкнетесь с ситуацией, когда новые ошибки возникают с той же скоростью, с какой устраняются старые. C++ разрабатывался так, чтобы обеспечить лучшую

структуру для больших программ и чтобы программист в одиночку мог справиться с большими объемами кода. Кроме того, хотелось, чтобы строка кода на C++ отражала больший объем работы, чем среднестатистическая строка кода на C или Pascal. К настоящему времени C++ перевыполнил поставленные перед ним задачи.

Не каждый фрагмент кода может быть хорошо структурирован, независим от аппаратуры, легко читаем и т.д. В C++ предусмотрены средства для прямого и непосредственного манипулирования аппаратными ресурсами без какого-либо намека на простоту восприятия и безопасность работы. Но такие средства всегда можно аккуратно спрятать за элегантными и безопасными интерфейсами.

Естественно, что применение C++ для написания больших программ сопровождается вовлечением в процесс многих программистов. Здесь-то и приносят дополнительные плоды гибкость языка C++ и его акцент на модульность и строго типизированные интерфейсы. Язык C++ имеет столь же сбалансированный набор вспомогательных средств для написания больших программ, что и многие другие языки. Но дело здесь в том, что по мере увеличения размера программ сложности, связанные с их разработкой и сопровождением, перетекают из области чисто языковых проблем в область создания удобного рабочего инструментария, графических сред разработки и управления проектами. Эти вопросы обсуждаются в части IV.

Настоящая книга акцентирует внимание на методах написании полезных программных средств, общеупотребительных типов, библиотек и т.п. Все это можно использовать и при написании небольших программ, и при разработке больших систем. Более того, поскольку все нетривиальные программы состоят из полунезависимых частей, то методы написания таких частей будут полезны всем программистам.

Наверное, вы полагаете, что написание программ посредством детального описания структуры используемых в ней типов приводит к увеличению совокупного размера исходного кода. Для языка C++ это не так. Программа на C++, которая определяет классы, точно определяет типы функциональных аргументов и т.д., немного короче эквивалентной, но не применяющей этих средств программы на C. Когда же используются библиотеки, программа на C++ уже намного короче своего C-эквивалента (если он вообще может быть создан).

1.3.2. Философские замечания

Язык программирования служит двум целям: он предоставляет программисту средства формулирования подлежащих выполнению действий, и он предоставляет набор концепций, которые программист использует, обдумывая, что нужно сделать. Первая цель в идеале требует языка, «близкого к машине», чтобы все машинные концепции отображались просто и эффективно способом, достаточно очевидным для программиста. Язык C был создан с этой целью. Вторая цель в идеале требует языка, близкого к решаемой задаче, чтобы понятия из логического пространства задачи могли формулироваться в терминах языка программирования кратко и непосредственно. Эту цель и преследуют средства, добавленные к языку C в процессе разработки C++.

Существует очень тесная связь между языком, на котором мы думаем/программируем, и задачами/решениями, которые мы можем себе представить. По этой причине ограничение языка средствами, не позволяющими программисту совер-

шать ошибки, по крайней мере неразумно. Как и в случае естественных языков, знание по крайней мере двух языков чрезвычайно полезно. Язык снабжает программиста набором концептуальных средств, которые, будучи неадекватными задаче, просто игнорируются. Хороший программный дизайн и отсутствие ошибок не могут гарантироваться всего лишь присутствием/отсутствием каких-либо языковых элементов.

Система типов особо полезна в случае нетривиальных задач. Для них концепция классов языка C++ на деле доказала свою чрезвычайную силу.

1.4. Исторические замечания

Я придумал язык C++, зафиксировал связанные с ним определения и выполнил самую первую реализацию. Также я выбрал и сформулировал критерии проектирования языка, разработал все основные элементы и рассматривал предложения по его расширению в комитете по стандартизации C++.

Ясно, что C++ многое заимствует у C [Kernighan, 1978]. За исключением устранимых лазеек в системе типов (см. приложение B), C остается подмножеством C++. Я сохранил все низкоуровневые средства языка C, чтобы можно было справиться с самыми критичными системными задачами. В свою очередь, C взял многое у своего предшественника — языка BCPL [Richards, 1980]; язык C++ позаимствовал у BCPL стиль однострочных комментариев // comment. Другим источником вдохновения был для меня язык Simula67 [Dahl, 1970], [Dahl, 1972]; концепция классов (вместе с производными классами и виртуальными функциями) была позаимствована у этого языка. Возможность перегружать операции и помещать объявления всюду, где может стоять оператор, напоминает Algol68 [Woodward, 1974].

С момента выхода первого издания этой книги язык интенсивно пересматривался и обновлялся. Основной ревизии подверглись правила разрешения перегрузки, компоновка и средства управления памятью. Дополнительно, были внесены небольшие изменения для улучшения совместимости с языком C. Также был сделан ряд обобщений и несколько фундаментальных дополнений: множественное наследование, статические функции-члены классов, константные функции-члены, защищенные члены классов, шаблоны, обработка исключений, RTTI (run-time type identification) и пространства имен (namespaces). Общим мотивом всех этих изменений и расширений было стремление улучшить средства языка для построения и использования библиотек. Эволюция C++ описана в [Stroustrup, 1994].

Шаблоны планировались, главным образом, для статической типизации контейнеров (таких, как списки, вектора и ассоциативные массивы) и для их элегантного и эффективного использования (обобщенное программирование). Ключевая цель заключалась в уменьшении применения макросов и операций явного приведения типов. На разработку шаблонов оказали влияние механизм обобщений языка Ada (своими достоинствами и недостатками) и параметризованные модули языка Clu. Аналогично, механизм исключений C++ испытал влияние со стороны языков Ada [Ichbiah, 1979], Clu [Liskov, 1979] и ML [Wikstrom, 1987]. Остальные нововведения языка C++ в период с 1985 по 1995 годы — множественное наследование, чисто виртуальные функции и пространства имен — проистекали главным образом из опыта применения C++, а не заимствовались из других языков.

Самые ранние версии языка, широко известные как «С с классами» [Stroustrup, 1994], использовались, начиная с 1980 года. Толчком к созданию языка послужило мое желание писать управляемые событиями программы-симуляторы, для которых язык Simula67 был бы идеальным, если бы не проблемы с эффективностью. «С с классами» использовался преимущественно в проектах, на которых это средство разработки особо эффективных (быстродействующих и требующих минимума памяти) программ проходило суровую обкатку и тестирование. В нем тогда отсутствовали перегрузка операций, ссылки, виртуальные функции, шаблоны, исключения и многие другие элементы. Первый выход С++ за стены исследовательской организации произошел в июле 1983 года.

Название С++ (читается «си плюс плюс») было придумано Риком Маскитти (Rick Mascitti) летом 1983 года. Это название отражает эволюцию языка со стороны С; в этом языке «++» означает операцию инкремента. Более короткое «С+» соответствует ошибочному выражению в языке С, и, к тому же, оно уже было занято (для имени другого языка). Язык не был назван D, поскольку он является прямым расширением С и не стремится «лечить» его проблемы отбрасыванием отдельных элементов. Иные интерпретации названия С++ см. в приложении к [Orwell, 1949].

Я начал разрабатывать С++ с тем, чтобы я и мои друзья перестали программировать на ассемблере, С или иных высокоуровневых языках того времени. Главной целью было сделать процесс создания качественных программ более простым и приятным. В первые годы не было никаких спецификаций и планов на бумаге: разработка, документирование и реализация выполнялись одновременно. Не было никакого «С++ проекта», равно как и «комитета по С++». Все это время С++ эволюционировал, помогая решать проблемы, возникающие у пользователей, а также под влиянием дискуссий с друзьями и коллегами.

Последовавший затем взрывной рост практического использования С++ привел к изменению ситуации. Где-то в 1987 году стало ясно, что формальная стандартизация С++ неизбежна, и мы стали готовить почву для этой деятельности [Stroustrup, 1994]. Результатом стали осознанные усилия по поддержанию контактов между производителями компиляторов и основными пользователями посредством обычной и электронной почты, а также на конференциях по С++ и в иных местах.

AT&T Bell Laboratories внесла значительный вклад в этот процесс, позволив мне делиться предварительными версиями спецификаций языка С++ с разработчиками компиляторов и пользователями. Поскольку многие из них работали на конкурентов AT&T, значение этого вклада не стоит недооценивать. Менее цивилизованная компания могла бы существенно попортить стандартизацию языка, просто ничего не делая. Сотни людей из десятков организаций прочли то, что потом стало общепризнанным руководством по языку и базовым документом для стандарта ANSI C++. Их имена перечислены в The Annotated C++ Reference Manual [Ellis, 1989]. В конце концов, в декабре 1989 года по инициативе Hewlett-Packard был создан комитет X3J16 при ANSI. В июне 1991 года эта инициатива ANSI (Американский Национальный Институт Стандартизации) стала частью международной деятельности под эгидой ISO (Международная Организация по Стандартизации). С 1990 года эти комитеты стали основным форумом для обсуждения эволюции и совершенствования языка С++. Я работал в этих комитетах в качестве главы рабочей группы по расширению языка, где непосредственно отвечал за рассмотрение предложений по коренным изменениям и добавлению новых средств. Предварительный вариант

стандарта был выпущен для публичного рассмотрения в апреле 1995 года. Официальный стандарт ISO C++ (ISO/IEC 14882) был принят в 1998 году. Технический документ с исправлениями мельчайших ошибок и неточностей был опубликован в 2003 году [C++, 2003].

Эволюция C++ прошла рука об руку с некоторыми из классов, представленных в данной книге. Например, я разработал классы `complex`, `vector` и `stack`, наряду с механизмом перегрузки операций. Классы `string` и `list` были разработаны Джонатаном Шопиро (Jonathan Shopiro) и мною. Они стали первыми активно использовавшимися библиотечными классами. Современный класс `string` из стандартной библиотеки берет свое начало в этих ранних усилиях. Библиотека задач, рассмотренная в [Stroustrup, 1987] и в §12.7[11], была частью самой первой программы на «C с классами», написанной когда-либо. Она поддерживала стиль моделирования (simulation), присущий языку Simula. Библиотека задач неоднократно пересматривалась и изменялась, преимущественно Джонатаном Шопиро, и по сей день используется активно. Вариант потоковой библиотеки, рассмотренный в первом издании книги, был написан мною. Джерри Шварц (Jerry Schwarz) переработал его в стандартные классы потокового ввода/вывода (глава 21), используя технику манипуляторов Эндрю Кенига (Andrew Koenig) (§21.4.6) и некоторые другие идеи. Классы потокового ввода/вывода подвергались в дальнейшем определенной переработке в процессе стандартизации, и большую часть работы выполнили Джерри Шварц, Натан Майерс (Nathan Myers) и Норихиро Кумагаи (Norihiko Kumagai). Разработка шаблонов испытала влияние шаблонных реализаций `vector`, `map`, `list` и `sort`, выполненных Эндрю Кенигом, Александром Степановым и мною. В свою очередь, работа Степанова по обобщенному программированию на базе шаблонов привела к контейнерам и алгоритмам стандартной библиотеки (§16.3, глава 17, глава 18, §19.2). Предназначенные для численных расчетов классы стандартной библиотеки (например, `valarray` — см. главу 22) были разработаны, главным образом, Кентом Баджем (Kent Budge).

1.5. Применение C++

C++ используется сотнями тысяч программистов практически во всех предметных областях. Эта деятельность подкрепляется наличием десятков независимых реализаций, сотнями библиотек и книг, несколькими специализированными журналами, многими конференциями и бесчисленными консультантами. Широко доступны многочисленные учебные курсы по C++ самого разного уровня.

Ранние приложения на C++ явно тяготели к системным задачам. Несколько важных операционных систем целиком были написаны на C++ [Campbell, 1987], [Rozier, 1988], [Hamilton, 1993], [Berg, 1995], [Parrington, 1995], а еще большее количество имеет важные части, выполненные на C++. Я всегда рассматривал бескомпромиссную низкоуровневую эффективность важной для C++. Это позволяет писать на C++ драйверы и другие программные компоненты, непосредственно оперирующие аппаратурой компьютера в реальном масштабе времени. В таких программах предсказуемость поведения ничуть не менее важна, чем скорость выполнения. Часто и компактность результирующего кода имеет важное значение. C++ был разработан таким образом, что буквально каждое его средство может использоваться в задачах с суровыми требованиями к производительности и размеру [Stroustrup, 1994, §4.5].

Большинство приложений безусловно содержат участки кода, критичные с точки зрения производительности. Однако большая часть программного кода расположена не в них. Для большей части программного кода чрезвычайно важны простота сопровождения, расширения и тестирования. С++ нашел широчайшее применение в областях, где очень важна надежность, и где требования к программам часто меняются. Таковы торговля, банковские и страховые операции, телекоммуникации, а также военные приложения. Многие годы централизованное управление междугородними телефонными звонками в США осуществлялось программой, написанной на С++, и каждый звонок на номер с префиксом 800 обрабатывался этой программой [Kamath, 1993]. Такие приложения велики и эксплуатируются очень долго. Поэтому стабильность, масштабируемость и совместимость были неперенными аспектами разработки С++. Программы, содержащие миллионы строк кода на С++, встречаются не так уж и редко.

Как и С, язык С++ не предполагал его интенсивного применения для решения вычислительных задач. И тем не менее, многие численные, научные и инженерные задачи программируются на С++. Основной причиной этого факта является необходимость комбинировать в программах вычислительную активность и графику, а также необходимость вычислений со сложными структурами данных, которые плохо вписываются в прокрустово ложе языка Fortran [Budge, 1992], [Barton, 1994]. Графика и GUI (Graphics User Interface — графический интерфейс пользователя) — вот области, где С++ используется весьма интенсивно. Любой, кто работал на Apple Macintosh или PC Windows, косвенно использовал С++, поскольку базовые пользовательские интерфейсы этих систем реализованы на С++. Наиболее популярные библиотеки для работы с графическим сервером X-System на платформе UNIX также написаны на С++. Таким образом, С++ широко используется в приложениях, для которых важен развитый пользовательский интерфейс.

Все перечисленное указывает на наиболее сильную сторону С++ — возможность эффективно обслуживать широкий диапазон прикладных областей. Не так уж трудно повстречаться с задачами, для которых одновременно нужны и сетевые взаимодействия (в локальных и глобальных сетях), и вычисления, и графика, и взаимодействие с пользователем, и доступ к базам данных. Традиционно эти области обслуживались раздельно и разными группами технических специалистов, каждая из которых применяла разные языки программирования. Язык же С++ одинаково хорошо применим ко всем этим областям. Более того, он разрешает существовать фрагментам программы, написанным на разных языках.

С++ широко используется для обучения и исследований. Это вызывает удивление у многих людей, которые совершенно справедливо замечают, что это далеко не самый маленький и простой язык. И тем не менее, про С++ можно сказать, что он:

- достаточно ясен для обучения базовым концепциям,
- практичен, эффективен и гибок для создания особо критичных систем,
- доступен для организаций и коллективов, которым необходимы разнообразные среды разработки и выполнения,
- достаточно обширен и содержателен для изучения сложных концепций и методов,
- достаточно технологичен для переноса изученного в коммерческую среду.

В общем, С++ — это язык, вместе с которым можно расти непрерывно.

1.6. Языки С и С++

Язык С был выбран в качестве базы при разработке С++, потому что он:

1. Универсальный, лаконичный и относительно низкоуровневый.
2. Адекватен большей части системных задач.
3. Исполняется везде и на всем.
4. Хорошо подходит для программирования в среде UNIX.

Конечно, С имеет свои проблемы, но их имел бы и любой другой язык, специально разработанный с самого нуля, а проблемы С всем уже хорошо известны. Разработанный поверх С язык «С с классами» стал полезным (хотя и неуклюжим) инструментом практически в первые же месяцы после того, как вообще возникла идея добавить Simula-подобные классы в С. Однако по мере того, как росла популярность языка С++ и все более важными становились его средства, построенные поверх С, постоянно поднимался вопрос о целесообразности сохранения такой совместимости. Конечно, при устранении наследственности часть проблем отпадет сама собой (см., например, [Sethi, 1981]). Но все же мы на это не пошли по следующим причинам:

1. Миллионы строк кода на С могут эффективно инкорпорироваться в С++ программы, пока не требуется их значительной перделки.
2. Миллионы строк библиотечного кода и полезных утилит, написанных на С, могут использоваться из программ на С++, пока эти языки синтаксически близки и совместимы на уровне правил компоновки.
3. Сотни тысяч программистов знают С, так что для перехода на С++ им нужно изучить лишь новые конструкции С++, а не переучивать основы.
4. С и С++ будут долгие годы использоваться на одних и тех же системах одними и теми же людьми, так что разница между этими языками должна быть или минимальной, или очень большой, иначе не избежать путаницы и ошибок.

Определения в С++ были пересмотрены с тем, чтобы любая конструкция, применяемая и в С, и в С++, имела бы одинаковый смысл в обоих языках (с незначительными исключениями; см. §B.2)

Язык С также не стоял на месте и эволюционировал, частично под влиянием С++ [Rosler, 1984]. Стандарт ANSI C [C, 1999] содержит синтаксис объявления функций, позаимствованный у «С с классами». Заимствование шло в обоих направлениях. Например, тип **void*** был впервые предложен для ANSI C, но реализован сначала в С++. Как было обещано в первом издании настоящей книги, определение С++ пересмотрено с целью устранения необоснованных несовместимостей, и теперь С++ более совместим С, чем вначале. В идеале, С++ должен быть близок к С настолько, насколько это возможно (но не более того) [Koenig, 1989]. Стопроцентная совместимость никогда не планировалась, ибо она подрывает типовую безопасность и гармонию между встроенными и пользовательскими типами.

Знание С не является обязательным условием для изучения С++. Программирование на С толкает на применение трюков, совершенно ненужных в свете возможностей С++. В частности, явное приведение типов в С++ используется реже, чем

в С (§1.6.1). В то же время, *хорошие* программы на С, по сути, весьма близки к программам на С++. Таковы, например, все программы из книги Кернигана и Ритчи *The C Programming Language (2nd Edition)* [Kernigan, 1988]. В принципе, знание любого языка со статической системой типов помогает изучению С++.

1.6.1. Информация для С-программистов

Чем лучше вы знакомы с С, тем труднее вам будет удержаться от соблазна писать на С++ в стиле С, теряя тем самым многие преимущества С++. Пожалуйста, ознакомьтесь с приложением В, в котором перечисляются все различия между С и С++. Вот некоторые вещи, с которыми С++ справляется лучше, чем С:

1. Макросы менее необходимы в С++. Используйте **const** (§5.4) или **enum** (§4.8) для объявления констант, **inline** (§7.1.1) — чтобы избежать накладных расходов на вызов функции, **шаблоны** (глава 13) — для определения семейств функций и типов, и **namespace** (§8.2) для преодоления коллизии имен.
2. Объявляйте переменные только тогда, когда они потребуются и тотчас же инициализируйте их. В языке С++ объявление допустимо всюду, где может стоять оператор (§6.3.1), в инициализирующих выражениях в операторе цикла **for** (§6.3.3) и в условиях (§6.3.2.1).
3. Не применяйте **malloc()**. Операция **new** (§6.2.6) работает лучше. Вместо **realloc()** попробуйте тип **vector** (§3.8, §16.3).
4. Старайтесь избегать **void***, арифметики указателей, объединений, явных приведений типа кроме как глубоко внутри реализаций функции или класса. В большинстве случаев явное приведение типов свидетельствует о дефектах проектирования. Если преобразование все же нужно, применяйте одну из новых операций приведения (§6.2.7), которые точнее отражают ваши намерения.
5. Минимизируйте применение массивов и строк в С-стиле — стандартные библиотечные типы **string** (§3.5) и **vector** (§3.7.1) заметно упрощают программирование. В целом, не пытайтесь программировать самостоятельно то, что уже запрограммировано в стандартной библиотеке.

Чтобы обеспечить компоновку функции С++ по правилам языка С, применяйте в ее объявлении соответствующий модификатор (§9.2.4).

И что самое важное, старайтесь мыслить о программе как о наборе взаимосвязанных концепций, представимых классами и объектами, а не о наборе структур данных, чьи биты перерабатываются функциями.

1.6.2. Информация для С++-программистов

Многие программисты используют С++ уже целое десятилетие. Большинство из них работают в одной и той же среде и привыкли к ограничениям, свойственным ранним компиляторам и библиотекам. Нельзя сказать, что опытные программисты не замечали появления новых средств в С++ — скорее, они не отслеживали их взаимоотношений, вследствие которых и стали возможными новые фундаментальные технологии. То, что ранее было неизвестно или казалось непрактичным, теперь может оказаться наилучшим подходом. Выяснить это можно лишь пересмотрев основы.

Читайте главы по порядку. Если содержание главы вам известно, то ее беглый просмотр займет лишь несколько минут. а если нет, то тогда вы наверняка столкнетесь с неожиданностями. Я сам кое-что узнал, пока писал эту книгу, и полагаю, что вряд ли существует такой программист, который знает абсолютно все изложенные здесь средства и технологии. Более того, чтобы правильно пользоваться языком, нужно видеть перспективу, которая и вносит порядок в набор средств и технологий. Данная книга с ее организацией и примерами нацелена на то, чтобы дать вам такую перспективу.

1.7. Размышления о программировании на C++

В идеале вы разрабатываете программу в три этапа. Сначала вы обдумываете исходную постановку задачи (анализ), затем вы формулируете ключевые концепции, связанные с решением задачи (проектирование), и только после этого воплощаете решение в виде программы (программирование). К сожалению, часто бывает так, что и проблема, и методы ее решения становятся предельно ясными лишь в процессе написания и отладки работающей программы. Вот здесь-то и сказывается выбор языка.

В большинстве приложений встречаются понятия, отразить которые в терминах фундаментальных типов или в виде функции без ассоциированных с нею данных весьма затруднительно. Столкнувшись с таким понятием, вводите класс, который и будет отражать это понятие в программе. Класс языка C++ — это тип. Тип специфицирует поведение объектов этого типа: как они создаются, как ими можно манипулировать и как объекты уничтожаются. Класс также определяет представление объектов, хотя на ранней стадии проектирования этот вопрос не является основным. Залогом хорошего проектирования служит четкое соответствие: «одно понятие — один класс». Для этого часто приходится сосредоточиться на вопросах типа: «Как создаются объекты рассматриваемого класса? Могут ли объекты класса копироваться и/или уничтожаться? Какие операции могут выполняться над этими объектами?» Если четких ответов на эти вопросы нет, то скорее всего нет и ясного понимания исходного понятия. Лучше вернуться к осмыслению исходной задачи и переформулировке основных концепций ее решения, а не кидаться тут же программировать с неясными последствиями.

Традиционно, легче всего работать с объектами, имеющими ярко выраженную математическую природу: числами всех типов, множествами, геометрическими фигурами и т.п. Текстовый ввод/вывод, строки, базовые контейнеры, фундаментальные алгоритмы обработки контейнеров и ряд математических классов являются частью стандартной библиотеки C++ (глава 3, §16.1.2). Кроме того, существует огромное количество библиотек, реализующих как самые общие, так и узкоспецифические для конкретных предметных областей понятия.

Понятия не существуют в вакууме; чаще всего они взаимодействуют друг с другом. Часто выявить и точно определить характер взаимодействия между понятиями бывает труднее, чем сформулировать набор понятий самих по себе. Лучше избегать «каши» из классов (понятий), взаимодействующих по принципу «каждый с каждым». Рассмотрим два класса, A и B. Отношения «A вызывает функции из B», «A создает объекты типа B» и «член класса A имеет тип B» редко создают проблемы.

В то же время, отношений «А использует данные из В» в общем случае следует избегать.

Одним из мощнейших интеллектуальных приемов преодоления сложностей, связанных с иерархическими зависимостями типов, является их упорядочение в древовидную структуру с самым общим понятием в корне дерева. В языке C++ такое упорядочение осуществляется с помощью механизма наследования классов. Часто программу можно организовать в виде набора деревьев или ациклических направленных графов классов. При этом программист определяет набор базовых классов, у каждого из которых имеются свои производные классы. Виртуальные функции (§2.5.5, §12.2.6) широко применяются для определения операций с наиболее общими понятиями (базовыми классами). Если необходимо, то эти операции могут уточняться для более специализированных понятий (производных классов).

Но иногда даже ациклических направленных графов не хватает для организаций понятий (концепций) конкретной программы; например, когда часть понятий представляются взаимозависимыми по своей природе. Тогда нужно локализовать циклические зависимости, чтобы они не влияли на структуру программы в целом. Если же не удастся ни избавиться от циклических зависимостей, ни локализовать их, то вы, похоже, находитесь в тупике, выйти из которого вам никакой язык не поможет. В самом общем случае, если не удастся сформулировать относительно простые связи между базовыми понятиями задачи, то программа, скорее всего, будет неуправляемой.

Одним из лучших способов распутывания взаимозависимостей является четкое разделение интерфейса и реализации. Абстрактные классы языка C++ (§2.5.4, §12.3) служат решению этой задачи.

Другая форма общности может быть выражена с помощью шаблонов (§2.7, глава 13). Классовый шаблон определяет целое семейство классов. Например, шаблон списков определяет «список элементов типа Т», где Т может быть любым типом. Таким образом, шаблон — это механизм порождения типа на основе другого типа, передаваемого шаблону в качестве параметра. Наиболее распространенными шаблонами являются контейнеры (такие как списки, вектора и ассоциативные массивы), а также фундаментальные алгоритмы для работы с этими контейнерами. Типовую параметризацию классов и связанных с ними функций всегда лучше выражать с помощью шаблонов, а не через механизм наследования классов.

Помните, что большое количество программ можно просто и ясно написать, используя лишь примитивные типы, структуры данных, традиционные функции и небольшое количество библиотечных классов. Весь мощнейший аппарат определения новых типов следует применять лишь там, где в этом есть серьезная необходимость.

Вопрос «Как писать хорошие программы на C++?» практически полностью аналогичен вопросу «Как писать хорошую английскую прозу?». Здесь есть два совета: «Знай то, что хочешь сказать» и «Практикуйся. Подражай успешным образцам». Оба совета подходят и для C++, и для английского языка, но обоим из них так трудно следовать.

1.8. Советы

Ниже дан набор «правил», которые могут пригодиться при изучении C++. По мере продвижения в этом процессе вы можете переработать их в нечто, более пригодное для ваших задач и вашего индивидуального стиля программирования. Перечисленные правила являются упрощенными и не отражают множества деталей. Не воспринимайте их слишком буквально. Для написания хороших программ требуются ум, вкус и терпение. Вряд ли эти правила помогут вам с самого начала. Экспериментируйте!

1. В процессе программирования вы реализуете конкретное представление концепций решения некоторой проблемы. Постарайтесь, чтобы структура программы отражала эти концепции как можно более непосредственно:
 - [a] Если вы мыслите об «этом» как о некой общей сущности, сделайте «это» классом.
 - [b] Если вы мыслите об «этом» как о конкретном экземпляре сущности, сделайте «это» объектом класса.
 - [c] Если два класса имеют общий интерфейс, оформите его в виде абстрактного класса.
 - [d] Если реализации двух классов имеют что-то общее, вынесите это общее в базовый класс.
 - [e] Если класс является контейнером объектов, сделайте его шаблоном.
 - [f] Если функция реализует алгоритм работы с контейнером, сделайте ее функциональным шаблоном, реализующим алгоритм для работы с семейством контейнеров.
 - [g] Если классы, шаблоны и т.д. логически связаны, поместите их все в единое пространство имен.
2. Определяя класс (если только это не математический класс вроде матриц или комплексных чисел, или низкоуровневый тип вроде связанного списка):
 - [a] Не используйте глобальные данные (применяйте классовые поля данных).
 - [b] Не используйте глобальные функции.
 - [c] Не делайте поля данных открытыми.
 - [d] Не используйте дружественные функции, разве что во избежание [a] и [c].
 - [e] Не используйте в классах «поля типа»; применяйте виртуальные функции.
 - [f] Не используйте встраиваемые функции, разве что для значительной оптимизации.

Более конкретные или подробные правила вы найдете в разделе советов в конце каждой главы. Помните, что это лишь советы, а не непреложные законы. Советами стоит пользоваться только там, где это разумно. Они не заменяют ум, опыт, здравый смысл и хороший вкус.

Я считаю советы вроде «не делайте этого никогда» бесполезными. Поэтому большинство советов формулируются в виде предложений, что желательно делать, а негативные утверждения не формулируются в виде абсолютных запретов. Я не знаю ни одного важного элемента языка C++, примеры полезного применения которого на практике не встречаются. Раздел «Советы» не содержит пояснений. Вместо этого, каждый совет снабжается ссылкой на соответствующий раздел книги. В случае негативных советов, указанный раздел содержит позитивную альтернативу.

1.8.1. Литература

Несколько прямых ссылок есть в тексте, а здесь мы приводим полный список книг и статей, упомянутых прямо или косвенно.

- [Barton, 1994] John J. Barton, and Lee R. Nackman: *Scientific and Engineering C++*, Addison-Wesley. Reading, Mass. 1994. ISBN 1-201-53393-6.
- [Berg, 1995] William Berg, Marshall Cline, and Mike Girou: *Lessons Learned from the OS/400 00 Project*. CACM. Vol. 38 No. 10. October 1995.
- [Booch, 1994] Grady Booch: *Object-Oriented Analysis and Design*, Benjamin/Cummings. Menlo Park, Calif. 1994. ISBN 0-8053-5340-2.
- [Budge, 1992] Kent Budge, J. S. Perry, and A. C. Robinson: *High-Performance Scientific Computation using C++*. Proc. USENIX C++ Conference. Portland, Oregon. August 1992.
- [C, 1990] X3 Secretariat: *Standard — The C Language*. X3J11/90-013. ISO Standard ISO/IEC 9899. Computer and Business Equipment Manufacturers Association. Washington, DC, USA.
- [C++, 1998] X3 Secretariat: *International Standard — The C++ Language* X3J16-14882. Information Technology Council (NSITC)] Washington, DC, USA.
- [C++, 2003] X3 Secretariat: *International Standard — The C++ Language* (including the 2003 Technical Corrigendum). 14882:2003(E). Information Technology Council (NSITC). Washington, DC, USA.
- [Campbell, 1987] Roy Campbell, et al.: *The Design of a Multiprocessor Operating System*. Proc. USENIX C++ Conference. Santa Fe, New Mexico November 1987.
- [Coplien, 1995] James O. Coplien and Douglas C. Schmidt (editors): *Pattern Languages of Program Design*. Addison-Wesley. Reading. Mass, 1995. ISBN 1-201-60734-4
- [Dahl, 1970] O-J. Dahl, B. Myrhaug, and K. Nygaard: *SIMULA Common Base Language*. Norwegian Computing Center S-22. Oslo, Norway. 1970.
- [Dahl, 1972] O-J. Dahl and C A. R. Hoare: *Hierarchical Program Construction in Structured Programming*. Academic Press, New York. 1972.

- [Ellis, 1989] Margaret A. Ellis and Bjarne Stroustrup: *The Annotated C++ Reference Manual*. Addison-Wesley. Reading, Mass. 1990. ISBN 0-201-51459-1.
- [Gamma, 1995] Erich Gamma, et al.: *Design Patterns*. Addison-Wesley. Reading, Mass. 1994. ISBN 0-201-63361-2.
- [Goldberg, 1983] A. Goldberg and D. Robson: *SMALLTALK-80 — The Language and Its Implementation*. Addison-Wesley. Reading, Mass. 1983.
- [Griswold, 1970] R. E. Griswold, et al.: *The Snobol4 Programming Language*. Prentice-Hall. Englewood Cliffs, New Jersey. 1970.
- [Hamilton, 1993] G. Hamilton and P. Kougiouris: *The Spring Nucleus: A Microkernel for Object*. Proc. 1993 Summer USENIX Conference. USENIX.
- [Griswold, 1983] R. E. Griswold and M. T. Griswold: *The ICON Programming Language*. Prentice-Hall. Engewood Cliffs, New Jersey. 1983.
- [Henricson, 1997] Mats Henricson and Erik Nyquist: *Industrial Strength C++: Rules and Recommendations*. Prentice-Hall. Engewood Cliffs, New Jersey. 1997. ISBN 0-13-120965-5.
- [Ichbian, 1979] Jean D. Ichbiah, et al.: *Rationale for the Design of the ADA Programming Language*. SIGPLAN Notices. Vol. 14 No. 6. June 1979.
- [Kamath, 1993] Yogeesh H. Kamath, Ruth E. Smilan, and Jean G. Smith: *Reaping Benefits with Object-Oriented Technology*. AT&T Technocal Journal. Vol. 72 No. 5. September/October 1993.
- [Kernighan, 1978] Brian W. Kernighan and Dennis M. Ritchie: *The C Programming Language*. Prentice-Hall. Englewood Cliffs, New Jersey. 1978.
- [Kernighan, 1988] Brian W. Kernighan and Dennis M. Ritchie: *The C Programming Language (Second Edition)*. Prentice-Hall. Englewood Cliffs, New Jersey. 1978. ISBN 0-13-110362-8.
- [Koenig, 1989] Andrew Koenig and Bjarne Stroustrup: *C++: As close to C as possible — but no closer*. The C++ Report. Vol. 1 No. 7. July 1989.
- [Koenig, 1997] Andrew Koenig and Barbara Moo: *Ruminations on C++*. Addison Wesley Longman. Reading, Mass. 1997. ISBN 1-201-42339-1.
- [Knuth, 1968] Donald Knuth: *The Art of Computer Programming*. Addison-Wesley. Reading, Mass.
- [Liskov, 1979] Barbara Liskov et al.: *Clu Reference Manual*. MIT/LCS/TR-225. MIT Cambridge. Mass. 1979.
- [Martin, 1995] Robert C Martin: *Designing Object-Oriented C++ Applications Using the Booch Method*. Prentice-Hall. Englewood Cliffs, New Jersey. 1995. ISBN 0-13-203837-4.
- [Orwell, 1949] George Orwell: *1984*. Secker and Warburg. London. 1949
- [Parrington, 1995] Graham Parrington et al.: *The Design and Implementation of Arjuna*. Computer Systems. Vol. 8 No. 3. Summer 1995.

- [Richards, 1980] Martin Richards and Colin Whitby-Stevens: *BCPL — The Language and Its Compiler*. Cambridge University Press, Cambridge. England. 1980. ISBN 0-521-21965-5.
- [Rosier, 1984] L. Rosier: *The Evolution of C — Past and Future*. AT&T Bell Laboratories Technical Journal. Vol. 63 No. 8. Part 2. October 1984.
- [Rozier, 1988] M. Rozier, et al.: *CHORUS Distributed Operating Systems*. Computing Systems. Vol. 1 No. 4. Fall 1988.
- [Sethi, 1981] Ravi Sethi: *Uniform Syntax for Type Expressions and Declarations*. Software Practice & Experience. Vol. 11. 1981.
- [Stepanov, 1994] Alexander Stepanov and Meng Lee: *The Standard Template Library*. HP Labs Technical Report HPL-94-34 (R. 1). August, 1994.
- [Stroustrup, 1986] Bjarne Stroustrup: *The C++ Programming Language*. Addison-Wesley. Reading, Mass. 1986. ISBN 0-201-12078-X.
- [Stroustrup, 1987] Bjarne Stroustrup and Jonathan Shopiro: *A Set of C Classes for Co-Routine Style Programming*. Proc. USENIX C++ Conference. Santa Fe, New Mexico. November 1987.
- [Stroustrup, 1991] Bjarne Stroustrup: *The C++ Programming Language (Second Edition)*. Addison-Wesley. Reading, Mass. 1991. ISBN 0-201-53992-6.
- [Stroustrup, 1994] Bjarne Stroustrup: *The Design and Evolution of C++*. Addison-Wesley. Reading, Mass. 1994. ISBN 0-2-1-54330-3.
- [Tarjan, 1983] Robert E. Tarjan: *Data Structures and Network Algorithms*. Society for Industrial and Applied Mathematics. Philadelphia, Penn. 1983. ISBN 0-898-71187-8.
- [Unicode, 1996] The Unicode Consortium: *The Unicode Standard, Version 2.0*. Addison-Wesley Developers Press. Reading, Mass, 1996. ISBN 0-201-48345-9.
- [UNIX, 1985] UNIX Time-Sharing System: *Programmer's Manual. Research Version, Tenth Edition*. AT&T Bell Laboratories, Murray Hill, New Jersey. February 1985.
- [Wilson, 1996] Gregory V. Wilson and Paul Lu (editors): *Parallel Programming Using C++*. The MIT Press. Cambridge. Mass. 1996. ISBN 0-262-73118-5.
- [Wikstrom, 1987] Ake Wikstrom: *Functional Programming Using ML*. Prentice-Hall Englewood Cliffs, New Jersey. 1987.
- [Woodward, 1974] P. M. Woodward and S. G. Bond: *Algol 68-R Users Guide*. Her Majesty's Stationery Office. London. England. 1974.

Русские переводы:

- [Booch, 1994] Г. Буч. *Объектно-ориентированный анализ и проектирование с примерами приложений на C++, 2-е издание*. СПб. «Невский Диалект». 1998.
- [Ellis, 1989] Б. Страуструп, М. А. Эллис. *Справочное руководство по языку C++ с комментариями: проект стандарта ANSI*. М. «Мир». 1992.
- [Kernighan, 1988] Б. Керниган, Д. Ричи. *Язык программирования Си. 3-е издание*. СПб. «Невский Диалект». 2001.
- [Knuth, 1968] Д. Кнут. *Искусство программирования для ЭВМ. т. 1, 2, 3*. М. «Мир». 1976.
- [Orwell, 1949] Дж. Оруэлл. *1984* (см. в сборнике: Дж. Оруэлл. *Проза отчаяния и надежды*. СПб. «Лениздат». 1990).
- [Stroustrup, 1986] Б. Страуструп. *Язык программирования Си++*. М. «Радио и связь». 1991.

Ссылки на литературу, посвященную проектированию и другим проблемам разработки крупных программных проектов, можно найти в конце главы 23.

Обзор языка C++

*В первую очередь, разделимся с защитниками языка.
— Генрих VI, Часть II*

Что такое C++? — парадигмы программирования — процедурное программирование — модульность — отдельная компиляция — обработка исключений — абстракция данных — типы, определяемые пользователем — конкретные типы — абстрактные типы — виртуальные функции — объектно-ориентированное программирование — обобщенное программирование — контейнеры — алгоритмы — язык и программирование — советы.

2.1. Что такое язык C++?

Язык C++ — это универсальный язык программирования с акцентом на системном программировании, и который:

- лучше, чем язык C,
- поддерживает абстракции данных,
- поддерживает объектно-ориентированное программирование,
- поддерживает обобщенное программирование.

В настоящей главе поясняется, что все это значит, но без ознакомления с мельчайшими деталями языковых определений. Дается общий обзор языка C++ и ключевых идей его применения, не отягощенных подробными сведениями, необходимыми для начала реального практического программирования на C++.

Если восприятие отдельных мест настоящей главы покажется вам затруднительным, не обращайтесь внимание и просто идите дальше. В последующих главах все разъясняется в деталях.

Неплохо, однако, потом вернуться к пропущенному еще раз. Детальное знание всех языковых свойств и конструкций, даже их исчерпывающее знание, не заменяет собой и не отменяет необходимость общего понимания роли и места этого языка

программирования, а также знакомства с фундаментальными идеями и приемами его использования.

2.2. Парадигмы программирования

Объектно-ориентированное программирование есть техника программирования, можно сказать, парадигма, для написания «хороших» программ для некоторого класса задач. Термин же «объектно-ориентированный язык программирования» должен в первую очередь означать такой язык, который хорошо поддерживает объектно-ориентированный стиль программирования.

Здесь следует четко различать две ситуации. Мы говорим, что язык *поддерживает* некоторый стиль программирования, если он предоставляет специальные средства, позволяющие этот стиль легко (удобно, надежно, эффективно) реализовывать на практике. Если же язык требует исключительных усилий и огромного опыта для реализации некоторой техники программирования, то мы говорим, что он лишь *допускает* возможность ее применения. Например, на языке Fortran 77 можно писать структурные программы, а на языке C можно писать программы в объектно-ориентированном стиле, но все это требует неадекватно больших усилий, так как эти языки напрямую не поддерживают указанные стили программирования.

Поддержка парадигмы программирования осуществляется не только через специально для этого предназначенные языковые конструкции, но и более тонко — через механизмы автоматического выявления непреднамеренного отклонения от выбранной парадигмы как на этапе компиляции, так и/или на этапе выполнения программы. Строгая проверка типов является очевидным примером такого механизма; выявление неоднозначностей (двусмысленностей — *ambiguities*) и контроль исполнения программы дополняют языковые средства поддержки парадигмы программирования. Внеязыковые средства, такие как библиотеки и среды программирования, еще более расширяют реальную поддержку парадигмы.

В то же время, неправильно говорить, что некоторый язык программирования лучше другого просто потому, что он содержит некоторые конструкции, отсутствующие в другом языке. И тому есть немало реальных примеров. На самом деле, важно не то, какими средствами обладает язык, а то, что средств этих на деле достаточно для реальной поддержки стиля программирования, оптимального для конкретной предметной области:

1. Все эти средства должны естественно и элегантно интегрироваться в язык.
2. Должна иметься возможность комбинировать средства для достижения целей, в противном случае требующих применения дополнительных специализированных средств.
3. Неестественных «средств специального назначения» в языке должно быть как можно меньше.
4. Реализация средств поддержки парадигм не должна обременять не нуждающиеся в них программы.
5. От программиста нельзя требовать знаний тех аспектов языка, которых он явным образом не использует в своих программах.

Первый из перечисленных принципов апеллирует к эстетике и логике. Два последующих выражают концепцию минимализма. А два последних принципа гласят: «нет нужды — нет проблемы».

Язык C++ создавался с учетом перечисленных принципов как расширение языка C для добавления поддержки абстракции данных, объектно-ориентированного и обобщенного стилей программирования. Он *не навязывает* программисту конкретный стиль программирования.

Стили программирования и поддерживающие их механизмы языка C++ рассматриваются в последующих разделах. Изложение материала стартует с рассмотрения процедурного стиля программирования, а заканчивается объектно-ориентированным программированием классовых иерархий и обобщенным программированием, использующим шаблоны языка C++. Показано, что каждая из парадигм строится поверх ее предшественницы, каждая вносит что-то новое в совокупный рабочий инструментарий C++-программиста, и что каждая парадигма основывается на проверенных временем и практикой подходах к проектированию и разработке программ.

Изложение материала в последующих разделах настоящей главы не является исчерпывающим. Акцент делается на вопросах проектирования программ и на их структуре, а вовсе не на тонкостях и деталях самого языка. На данном этапе важнее понять, что именно можно сделать с помощью языка C++, а не то, как этого достичь практически.

2.3. Процедурное программирование

Исходная парадигма процедурного программирования звучит так:

*Решите, какие процедуры нужны;
используйте наилучшие алгоритмы.*

Здесь акцент делается на процессе обработки данных, в первую очередь на алгоритме необходимых вычислений. Языки поддерживают эту парадигму, предоставляя механизмы передачи аргументов функциям и получения их возврата (то есть вычисленного значения функции). Литература по процедурному программированию наполнена обсуждением способов передачи аргументов и различием их типов, классификацией видов функций (процедур, подпрограмм и макросов) и т.п.

Функция извлечения квадратного корня является типичным примером «хорошего стиля» для процедурной парадигмы. Получив аргумент в виде числа с плавающей точкой двойной точности, она вырабатывает искомый результат. Для этого она выполняет хорошо известные математические вычисления:

```
double sqrt(double arg)
{
    // код для вычисления квадратного корня
}
void f()
```

```
{
    double root2 = sqrt (2) ;
    // ...
}
```

Фигурные скобки, {}, в языке C++ предназначены для группировки. Здесь они указывают начало и конец тела функции. Двойная наклонная черта, //, означает начало комментария, который занимает всю оставшуюся часть строки. Ключевое слово `void` указывает здесь на отсутствие у функции возвращаемого значения (возврата).

С точки зрения организаций программ, функции используются для наведения порядка в хаосе алгоритмов. Сами алгоритмы записываются с помощью функциональных вызовов и других средств языка. В следующих подразделах бегло рассматриваются простейшие средства языка C++, предназначенные для организации вычислений.

2.3.1. Переменные и простейшая арифметика

Каждое имя в программе и каждое выражение имеют определенный тип, задающий набор допустимых для них операций. К примеру, следующее объявление

```
int inch;
```

устанавливает, что *inch* относится к типу *int*, то есть *inch* является целочисленной переменной.

Объявление является *оператором (statement)*, вводящим имя в программу. Оно связывает имя с типом. Тип определяет, что можно (допустимо) делать с именем или выражением.

Язык C++ имеет ряд *встроенных типов*, имеющих непосредственное отношение к устройству электронных узлов (процессор и оперативная память) компьютера. Например:

```
bool      // логический, возможные значения - true и false
char      // символьный, например, 'a', 'z', и '9'
int       // целый, например, 1, 42, и 1216
double    // вещественные числа с двойной точностью, например, 3.14 и 299793.0
```

Для заданного компьютера переменная типа *char* имеет естественный размер, используемый для хранения символа (как правило, один байт), а переменная типа *int* — естественный размер, используемый для вычислений с целыми числами (обычно называемый *словом*).

С любой комбинацией этих типов можно выполнять следующие арифметические операции:

```
+      // плюс, унарный и бинарный
-      // минус, унарный и бинарный
*      // умножение
/      // деление
%      // остаток от деления (второй операнд не может иметь тип
      // с плавающей запятой)
```

А также операции сравнения:

```

==      // равно
!=      // не равно
<       // меньше чем
>       // больше чем
<=      // меньше или равно
>=      // больше или равно

```

Для арифметических операций и операций присваивания C++ выполняет имеющие очевидный смысл преобразования типов, так что их можно использовать смешанным образом:

```

void some_function ()    // функция, не возвращающая значение
{
    double d = 2.2;      // инициализировать число с плавающей запятой
    int i = 7;           // инициализировать целое число
    d = d+i;             // присвоить сумму переменной d
    i = d*i;             // присвоить произведение переменной i
}

```

Как и в языке C, знак = используется для операции присваивания, а знак == используется для операции сравнения на равенство.

2.3.2. Операторы ветвления и циклы

Язык C++ обеспечивает общепринятый набор операторов ветвления и цикла. Для примера рассмотрим функцию, выводящую приглашение к вводу и возвращающую булевское (логическое) значение, зависящее от ввода пользователя:

```

bool accept ()
{
    cout<< "Do you want to proceed (y or n) ?\n"; // вывести вопрос
    char answer = 0;
    cin >> answer;                               // считать ответ
    if(answer == 'y') return true;
    return false;
}

```

Операция << («вставить в поток») используется как операция вывода, а *cout* представляет собой стандартный поток вывода. Операция >> («извлечь из потока») используется как операция ввода, а *cin* представляет собой стандартный поток ввода. Правый операнд операции >> принимает введенное значение, причем само это значение зависит от типа операнда. Символ \n в конце выводимой строки означает переход на новую строку.

Приведенный пример можно слегка улучшить, если явно отреагировать на ответ 'n':

```

bool accept2 ()
{
    cout << "Do you want to proceed (y or n) ?\n"; // вывести вопрос
    char answer = 0;
    cin>>answer;                               // считать ответ

```

```

switch (answer)
{
    case 'y' :
        return true;
    case 'n' :
        return false;
    default:
        cout << "I'll take that for a no. \n";
        return false;
}
}

```

Оператор **switch** сравнивает заданное значение с набором констант. Константы в разных **case**-ветвях должны быть разными, и если проверяемое значение не совпадает ни с одной из них, то выбирается **default**-ветвь. Программист, в общем случае, не обязан реализовывать **default**-ветвь.

Редкая программа совсем не использует циклов. В нашем примере мы могли бы дать пользователю несколько попыток ввода:

```

bool accept3 ()
{
    int tries = 1;
    while (tries < 4)
    {
        cout << "Do you want to proceed (y or n) ? \n"; // вывести вопрос
        char answer = 0;
        cin >> answer; // считать ответ

        switch (answer)
        {
            case 'y' :
                return true;
            case 'n' :
                return false;
            default:
                cout << "Sorry, I don't understand that. \n";
                tries = tries + 1;
        }
    }
    cout << "I'll take that for a no. \n";
    return false;
}

```

Оператор цикла **while** выполняется до тех пор, пока его условие не станет равным **false** («ложь»).

2.3.3. Указатели и массивы

Массив можно объявить следующим образом:

```
char v[10]; // массив из 10 символов
```

Указатель объявляется так, как показано ниже:

```
char* p;    // указатель на символ
```

В этих объявлениях `[]` означает «массив», а `*` означает «указатель». Массивы индексируются с нуля, то есть `v` содержит следующие десять элементов: `v[0]` ... `v[9]`. Указатель может содержать адрес объекта соответствующего типа:

```
p = &v[3]; // p указывает на четвертый элемент массива v
```

Унарная операция `&` означает «взятие адреса».

Рассмотрим копирование десяти элементов массива в другой массив:

```
void another_function ()
{
    int v1[10];
    int v2[10];
    // ...
    for (int i=0; i<10; ++i) v1[i]=v2[i];
}
```

Оператор цикла `for` задает следующую последовательность действий: «присвоить `i` значение `0`; пока `i` меньше `10`, копировать `i`-ый элемент и инкрементировать `i`». Операция инкремента `++` увеличивает значение целой переменной на единицу.

2.4. Модульное программирование

С течением времени акцент в разработке программ сместился от разработки процедур в сторону организации данных. Помимо прочего, этот сдвиг вызван увеличением размера программ. Набор связанных процедур и обрабатываемых ими данных часто называют *модулем* (*module*). Парадигма программирования теперь звучит так:

*Решите, какие модули нужны;
сегментируйте программы так, чтобы данные были скрыты в модулях*

Эта парадигма известна также как *принцип сокрытия данных*. В задачах, где группировка процедур и данных не нужна, процедурный стиль программирования по-прежнему актуален. К тому же, принципы построения «хороших процедур» теперь применяются по отношению к отдельным процедурам модуля. Типовым примером модуля может служить определение стека. Вот соответствующий список задач, требующих решения:

1. Предоставить пользовательский интерфейс к стеку (например, функции `push()` и `pop()`).
2. Гарантировать доступ к внутренней структуре стека (реализуемой, например, с помощью массива элементов) исключительно через интерфейс пользователя.
3. Гарантировать автоматическую инициализацию стека до первого обращения пользователя.

Язык C++ предоставляет механизм группировки связанных данных, функций и т.д. в *пространства имен (namespaces)*. Например, пользовательский интерфейс модуля *Stack* может быть объявлен и использован следующим образом:

```
namespace Stack // интерфейс
{
    void push(char);
    char pop();
}

void f()
{
    Stack::push('c');
    if(Stack::pop() != 'c') error("impossible");
}
```

Квалификатор Stack:: означает, что речь идет о функциях *push()* и *pop()*, определенных в пространстве имен *Stack*. Иное использование этих имен не конфликтует с указанным и не вносит никакой путаницы в программу.

Полное определение стека можно разместить в отдельно компилируемой части программы:

```
namespace Stack // реализация
{
    const int max_size = 200;
    char v[max_size];
    int top = 0;

    void push(char c) { /* проверить на переполнение и push c */ }
    char pop() { /* проверить на пустоту стека и pop */ }
}
```

Ключевым в этом определении является тот факт, что пользовательский код изолируется от внутреннего представления модуля *Stack* посредством кода, реализующего функции *Stack::push()* и *Stack::pop()*. Пользователю нет необходимости знать, что *Stack* основан на массиве, и в результате имеется возможность изменять его внутреннее представление без последствий для пользовательского кода. Символы */** маркируют начало в общем случае многострочного комментария, заканчивающегося символами **/*.

Так как данные есть лишь часть того, что бывает желательным «скрыть», то концепция сокрытия данных очевидным образом расширяется до *концепции сокрытия информации*: имена функций, типы и т.д. локально определяются внутри модулей. Для этого язык C++ разрешает поместить в пространство имен любое объявление (§8.2).

Модуль *Stack* является лишь одной из возможных реализаций стека. Иные примеры стека в последующих главах будут иллюстрировать другие стили программирования.

2.4.1. Раздельная компиляция

Язык C++ поддерживает *принцип раздельной компиляции (separate compilation)*, взятый из языка C. Это позволяет организовать программу в виде набора относительно независимых фрагментов.

В типовом случае, мы размещаем части модуля, ответственные за интерфейс взаимодействия с пользователем, в отдельный файл с «говорящим» именем. Так, код

```
namespace Stack // интерфейс
{
    void push(char);
    char pop();
}
```

будет помещен в файл **stack.h**, называемый *заголовочным файлом (header file)*. Пользователи могут *включить* его в свой код следующим образом:

```
#include "stack.h" // включить интерфейс

void f()
{
    Stack::push('c');
    if(Stack::pop() != 'c') error("impossible");
}
```

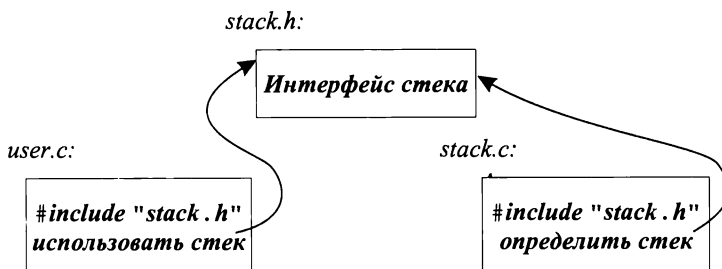
Чтобы компилятор мог гарантировать согласованность кода, в файл с реализацией стека также нужно включить интерфейсный заголовочный файл:

```
#include "stack.h" // включить интерфейс

namespace Stack // представление
{
    const int max_size = 200;
    char v[max_size];
    int top = 0;

    void Stack::push(char c) { /* проверить на переполнение и push c */ }
    char Stack::pop() { /* проверить на пустоту стека и pop */ }
}
```

Пользовательский код может располагаться в третьем файле, скажем, **user.c**. Код файлов **user.c** и **stack.c** разделяет информацию об интерфейсе стека, расположенную в файле **stack.h**. Во всем остальном файлы **user.c** и **stack.c** независимые и могут компилироваться раздельно. Графически, рассмотренные фрагменты программы можно представить следующим образом:



Раздельная компиляция является чрезвычайно полезным инструментом в реальном программировании. Это не узкая материя, с которой сталкиваются лишь программы, реализующие в виде модулей концепции стека и т.п. И хотя раздельная компиляция и не относится, строго говоря, к базовым элементам языка, она наилучшим образом позволяет извлечь максимальную пользу из его конкретных реализаций. Так или иначе, это вопрос большого практического значения. Целесообразно стремиться к достижению максимальной степени модульности программы, логической реализации этой модульности в конструкциях языка, и физической раскладке модулей по файлам для эффективной раздельной компиляции (главы 8, 9).

2.4.2. Обработка исключений

Когда программа выполнена в виде набора модулей, обработка ошибок должна рассматриваться в контексте модулей. Какой модуль отвечает за обработку тех или иных ошибок? Часто модуль, столкнувшийся с ошибочной ситуацией, не знает, что нужно предпринять для ее исправления. Ответные действия зависят от модуля, инициировавшего операцию, приведшую к ошибке, а не от модуля, выполнявшего эти действия. По мере роста размера программ и в связи с интенсивным использованием библиотек, стандарты по обработке ошибок (или шире — «исключительных ситуаций») становятся важными.

Снова рассмотрим пример с модулем *Stack*. Что следует предпринять в ответ на попытку занести в стек слишком много символов? Автор модуля *Stack* не знает, что нужно пользователю в этом случае, а пользователь не может распознать ситуацию переполнения стека (в противном случае он ее вообще не допустил бы). Устранить указанное противоречие можно следующим образом: автор модуля *Stack* выявляет переполнение стека и сообщает об этом (неизвестному) пользователю. В ответ пользователь предпринимает необходимые действия. Например:

```
namespace Stack           // интерфейс
{
    void push (char) ;
    char pop () ;
    class Overflow { } ;   // тип, представляющий исключение, связанное с переполнением
}
```

Обнаружив переполнение, функция *Stack::push()* может косвенно вызвать код обработки данной исключительной ситуации, сгенерировав «исключение типа *Overflow*»:

```
void Stack::push (char c)
{
    if (top == max_size) throw Overflow () ;
    // поместить c в стек
}
```

Оператор *throw* передает управление обработчику исключений типа *Stack::Overflow*, расположенному в теле функции, прямо или косвенно вызвавшей функцию *Stack::push()*. Для этого компилятор самостоятельно создает машинный код, раскручивающий стек вызовов функций назад до состояния, актуального на момент работы указанной функции. Таким образом, оператор *throw* работает как многоуровневый оператор *return*. Вот соответствующий пример:


```

void f()
{
    // ...
    try // возникающие здесь исключения передаются обработчику, определенному ниже
    {
        while (true) Stack : : push ( 'c' );
    }
    catch (Stack : : Overflow)
    {
        // oops: переполнение стека; предпримем надлежащие действия
    }
    // ...
}

```

Здесь цикл **while** в *try*-блоке пытается выполняться вечно. В результате, при некотором обращении к функции **Stack : : push ()** последняя *сгенерирует исключение* типа **Stack : : Overflow**, и управление будет передано в *catch*-блок, предназначенный для обработки исключений этого типа.

Использование механизма исключительных ситуаций языка C++ позволяет сделать обработку ошибок более регулярной, а тексты программ более «читабельными». Более подробно этот вопрос рассматривается в §8.3, главе 14 и приложении Е.

2.5. Абстракция данных

Модульность является фундаментальным аспектом успешных программных проектов большого размера. В настоящей книге она всегда в центре внимания при обсуждении вопросов проектирования программ. Однако способ объявления и использования модулей, рассмотренный нами до сих пор, не является адекватным в случае сложных программных систем. Сейчас я рассмотрю применение модулей для логического формирования того, что можно назвать пользовательскими типами данных, а затем преодолёю связанные с таким подходом недостатки с помощью специальных синтаксических конструкций языка C++, предназначенных для явного создания пользовательских типов.

2.5.1. Модули, определяющие типы

Модульное программирование естественным образом приводит к идее о централизованном управлении всеми данными одного и того же типа с помощью специального управляющего модуля. Например, чтобы работать одновременно со множеством стеков, а не с единственным, представленным выше модулем **Stack**, можно определить одноименный менеджер стеков с интерфейсом следующего вида:

```

namespace Stack
{
    struct Rep; // определение раскладки стека находится в другом месте
    typedef Rep& stack;

    stack create ( ) ; // создать новый стек
    void destroy (stack s) ; // удалить стек s
}

```

```
void push (stack s, char c) ; // поместить c в s
char pop (stack s) ;         // извлечь s из стека (pop s)
}
```

Объявление

```
struct Rep;
```

говорит о том, что **Rep** это имя типа, определение которого будет дано позже (§5.7).
Объявление

```
typedef Rep & stack;
```

дает имя `stack` «ссылкам на **Rep**» (подробности см. в §5.5). Центральная идея состоит в том, что конкретный стек создается как переменная типа **Stack::stack**, а остальные детали от пользователя скрыты.

Переменные типа **Stack::stack** ведут себя почти как переменные встроенных типов:

```
struct Bad_pop { };

void f()
{
    Stack::stack s1 = Stack::create(); // создаем новый стек
    Stack::stack s2 = Stack::create(); // создаем еще один новый стек

    Stack::push(s1, 'c');
    Stack::push(s2, 'k');

    if(Stack::pop(s1) != 'c') throw Bad_pop();
    if(Stack::pop(s2) != 'k') throw Bad_pop();

    Stack::destroy(s1);
    Stack::destroy(s2);
}
```

Реализацию модуля **Stack** можно выполнить разными способами. Важно, что пользователю нет необходимости знать, как именно мы это делаем: при неизменном интерфейсе изменение реализации не влияет на код пользователя.

Реализация модуля **Stack** может заранее разместить в памяти несколько стеков, и тогда функция **Stack::create()** будет просто возвращать ссылку на неиспользуемый стек. Функция **Stack::destroy()**, в свою очередь, будет помечать стек как «свободный», так что функция **Stack::create()** может использовать его повторно:

```
namespace Stack // представление
{
    const int max_size = 200;

    struct Rep
    {
        char v[max_size];
        int top;
    };

    const int max = 16; // максимальное количество стеков
```

```

Rep stacks[max];    // априорная раскладка стеков
bool used[max];     // used[i] == true если stacks[i] используется

typedef Rep& stack;
}

void Stack::push(stack s, char c) { /* проверить s на переполнение и push c */ }
char Stack::pop(stack s) { /* проверить, не пуст ли s и pop */ }

Stack::stack Stack::create()
{
    // выбрать неиспользуемый Rep, пометить как используемый,
    // инициализировать его, и вернуть ссылку на него
}

void Stack::destroy(stack s) { /* пометить s как неиспользуемый */ }

```

Что мы здесь сделали по существу? Мы полностью определили интерфейсные функции, используя тип `stack` из реализации модуля **Stack**. Результирующее поведение созданного таким образом «стекового псевдотипа» зависит от нескольких факторов: от того, как именно мы определили интерфейсные функции; от того, как мы предоставляем тип **Stack::stack** пользователям, а также от деталей определения типов **Stack::stack** и **Stack::Rep** (так называемых типов представления — *representation types*).

Рассмотренное решение далеко не идеальное. Существенной проблемой является зависимость способа представления таких псевдотипов пользователю от деталей устройства внутренних типов представления, а ведь пользователи от таких деталей должны быть изолированы. Действительно, стоит нам применить более сложные структуры для внутреннего представления стеков, как сразу же изменятся правила присваивания и инициализации для **Stack::stack**. Может быть, иногда это и неплохо. Однако ж ясно, что проблема создания удобного типа для стеков просто переместилась в тип представления **Stack::stack**.

Более фундаментально, пользовательские типы, реализованные посредством модуля с доступом ко внутренней реализации, ведут себя не как встроенные типы и имеют меньшую поддержку. Например, время доступа к типу представления **Stack::Rep** контролируется функциями **Stack::create()** и **Stack::destroy()**, а не обычными правилами языка.

2.5.2. Типы, определяемые пользователем

Для преодоления рассмотренной выше проблемы язык C++ позволяет пользователю создавать новые типы, которые ведут себя (почти) так же, как типы встроенные. Их часто называют *абстрактными типами данных*. Однако я предпочитаю называть их *типами, определяемыми пользователем* (*user-defined types*). Строгое определение абстрактных типов данных потребовало бы математически точной «абстрактной» формулировки. При ее наличии то, что мы называем *типами*, было бы конкретными примерами таких истинно абстрактных сущностей. Программная парадигма становится такой:

Решите, какие типы нужны;
обеспечьте полный набор операций для каждого типа.

Если не требуется создавать более одного объекта определенного типа, то достаточно рассмотренного стиля программирования с сокрытием данных на базе модулей.

Математические типы, такие как рациональные или комплексные числа, являются естественными примерами типов, определяемых пользователем. Вот иллюстрация на эту тему:

```
class complex
{
    double re, im;

public:
    complex(double r, double i) { re=r; im=i; } // создать complex из двух скаляров
    complex(double r) { re=r; im=0; }           // создать complex из скаляра
    complex() { re = im = 0; }                  // по умолчанию: (0,0)

    friend complex operator+ (complex, complex);
    friend complex operator- (complex, complex); // бинарная операция
    friend complex operator- (complex);          // унарная операция
    friend complex operator* (complex, complex);
    friend complex operator/ (complex, complex);

    friend bool operator== (complex, complex); // проверка на равенство
    friend bool operator!= (complex, complex); // проверка на неравенство
    // ...
};
```

Объявление класса **complex** (то есть типа, определяемого пользователем) задает как представление комплексных чисел, так и набор операций над комплексными числами. При этом представление является *закрытым* (*private*); доступ к полям **re** и **im** имеют лишь функции, перечисленные в объявлении класса **complex**. Их можно определить примерно так:

```
complex operator+ (complex a1, complex a2)
{
    return complex (a1.re+a2.re, a1.im+a2.im);
}
```

Функции с именем, совпадающим с именем класса, называют *конструкторами*. Конструктор задает способ инициализации объекта класса. Класс **complex** объявляет три конструктора. Один из них преобразует вещественное число типа **double** в *комплексное число*, другой создает комплексное число из пары вещественных, а третий строит комплексное число со значением по умолчанию.

Использовать класс **complex** можно следующим образом:

```
void f (complex z)
{
    complex a = 2.3;
    complex b = 1/a;
    complex c = a+b*complex (1, 2.3);
    // ...
    if (c!=b) c = -(b/a)+2*b;
```

Компилятор преобразует выполняемые над комплексными числами операции в вызовы соответствующих функций из определения класса **complex**. К примеру, для выражения $c != b$ вызывается функция **operator!=** (c, b), а вместо выражения $1/a$ вызывается функция **operator/(complex(1), a)**.

Большинства модулей, хотя и не все из них, лучше формулируются в виде типов, определяемых пользователем.

2.5.3. Конкретные типы

Типы, определяемые пользователем, разрабатываются с разными целями. Рассмотрим определяемый пользователем тип **Stack** по аналогии с рассмотренным выше типом **complex**. Пример станет более реалистичным, если мы будем передавать стеку в момент его создания аргумент, фиксирующий предельную емкость (размер) стека:

```
class Stack
{
    char* v;
    int top;
    int max_size;

public:
    class Underflow { }; // используется как исключение
    class Overflow { }; // используется как исключение
    class Bad_size { }; // используется как исключение

    Stack(int s); // конструктор
    ~Stack(); // деструктор

    void push(char c);
    char pop();
};
```

Конструктор **Stack(int)** вызывается автоматически, когда создается объект класса. Он берет на себя заботу об инициализации. Если требуется очистка памяти (cleanup) в момент выхода объекта из области видимости, можно объявить функцию, называемую *деструктором* (*destructor*, по цели своих действий противоположна конструктору):

```
Stack::Stack(int s) // конструктор
{
    top = 0;
    if( s < 0 || 10000 < s ) throw Bad_size(); // || означает ИЛИ
    max_size = s;
    v = new char[s]; // расположить элементы в свободной памяти (куча, динам. память)
}

Stack::~~Stack() // деструктор
{
    delete[] v; // уничтожить элементы для дальнейшего
                // использования памяти (§6.2.6)
```

Конструктор инициализирует вновь создаваемый объект класса *Stack*. Для этого он выделяет некоторое количество памяти из свободного пула (называемого *кучей* или *динамической памятью* — *heap* or *dynamic store*) с помощью операции *new*. Деструктор же освобождает эту память. Пользователи класса *Stack* не предпринимают для этого никаких специальных мер. Они лишь создают объекты типа *Stack* и используют их приблизительно так же, как и переменные встроенных типов. Например:

```
Stack s_var1 (10);           // глобальный стек из 10 элементов
void f(Stack& s_ref, int i)   // ссылка на Stack
{
    Stack s_var2 (i);         // локальный стек из i элементов
    Stack* s_ptr= new Stack (20); // указатель на Stack в свободной памяти

    s_var1.push ('a');         // доступ через переменную (§5.7)
    s_var2.push ('b');         //
    s_ref.push ('c');          // доступ по ссылке (§5.5, §5.7)
    s_ptr->push ('d');          // доступ по указателю (§5.7)
    // ...
}
```

Для типа *Stack* действуют те же правила именования, области видимости, выделения памяти, времени жизни и т.д., что и для встроенных типов, таких как, например, *int* или *char*. Подробно о том, как можно управлять временем жизни классовых объектов, рассказано в §10.4.

Естественно, функции *push()* и *pop()*, объявленные в классе *Stack*, должны быть где-то определены:

```
void Stack::push(char c)
{
    if(top == max_size) throw Overflow();
    v[top] = c;
    top = top + 1;
}

char Stack::pop()
{
    if(top == 0) throw Underflow();
    top = top - 1;
    return v[top];
}
```

Такие типы, как *complex* или *Stack*, называют *конкретными типами* (*concrete types*), в противовес *абстрактным типам* (*abstract types*), интерфейс которых еще сильнее изолирует пользователя от деталей реализации.

2.5.4. Абстрактные типы

В процессе перехода от реализованного в виде модуля псевдотипа *Stack* (§2.5.1) к соответствующему правильному типу (§2.5.3), одно свойство было утеряно. Мы не отделили представление от пользовательского интерфейса; скорее оно является частью того, что включается в программные фрагменты, использующие тип *Stack*.

Представление является закрытым, так что к нему могут обращаться лишь объявленные в классе функции. Но оно все же присутствует в явном виде и в том же самом месте. Поэтому, если представление хоть сколько-нибудь значительно изменится, пользовательский код подлежит перекомпиляции. Это плата за то, что пользовательские типы ведут себя так же, как встроенные типы. В частности, у нас не будет настоящих локальных переменных пользовательских типов, если не будет в точности известен размер их представления.

Для редко меняющихся пользовательских типов, и в случаях, когда локальные переменные обеспечивают необходимую ясность и эффективность, все это допустимо, если не идеально. Однако если же мы хотим полностью изолировать пользователей от изменений внутренних реализаций стека, предыдущее определение типа **Stack** не годится. В таком случае приходится идти на полное отделение интерфейса от представления (реализации) и на отказ от настоящих локальных переменных этого типа.

Сначала определяем *интерфейс (interface)*:

```
class Stack
{
public:
    class Underflow { };           // используется как исключение
    class Overflow { };           // используется как исключение
    virtual void push (char c) = 0;
    virtual char pop () = 0;
};
```

Слово **virtual** в языках Simula и C++ означает «позднее может быть замещено (переопределено — redefined) в классах, производных от данного». Производный класс призван обеспечить реализацию интерфейса базового класса **Stack**. Необычный синтаксис `=0` означает, что производный класс *обязан* определить эту функцию. Таким образом, класс **Stack** служит интерфейсом для любого производного от него класса, реализующего функции **push()** и **pop()**.

Класс **Stack** может быть использован следующим образом:

```
void f(Stack& s_ref)
{
    s_ref.push('c');
    if(s_ref.pop() != 'c') throw Bad_pop();
};
```

Обратите внимание на то, как пользовательская функция **f()** использует интерфейс класса **Stack** при полном неведении относительно деталей реализации. Класс, обеспечивающий интерфейс для множества иных классов, часто называют *полиморфным типом (polymorphic type)*.

Неудивительно, что реализация может содержать все то из рассмотренного в предыдущем разделе конкретного типа **Stack**, что осталось за бортом интерфейсного класса **Stack**:

```
class Array_stack : public Stack // Array_stack реализует Stack
{
    char* p;
    int max_size;
    int top;
```

```
public:
    Array_stack (int s) ;
    ~Array_stack () ;

    void push (char c) ;
    char pop () ;
};
```

Здесь «**public**» можно читать как «производный от», «предоставляет реализацию для» или «является подтипом».

Чтобы функция вроде $f()$ могла использовать тип **Stack** при полном игнорировании деталей реализации, некоторая другая пользовательская функция должна создать объект, с которым функция $f()$ и будет реально работать:

```
void g ()
{
    Array_stack as (200) ;
    f(as) ;
}
```

Так как функция $f()$ ничего не знает о классе **Array_stack**, а лишь об интерфейсе **Stack**, то она будет столь же успешно работать и с иными реализациями интерфейса **Stack**. Например:

```
class List_stack : public Stack // List_stack реализует Stack
{
    list<char> lc; // (стандартная биб-ка) список символов (§3.7.3)

public:
    List_stack () {}
    void push (char c) { lc.push_front(c) ; }
    char pop () ;
};

char List_stack::pop ()
{
    char x = lc.front () ; // получить первый элемент
    lc.pop_front () ; // удалить первый элемент
    return x;
}
```

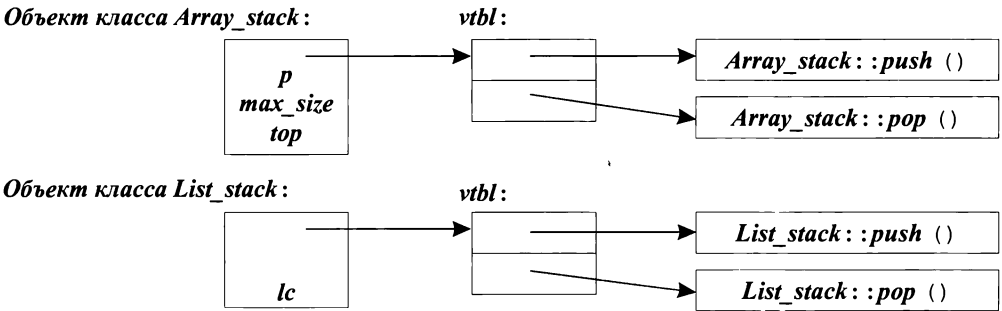
Данное представление базируется на списке символов. Вызов $lc.push_front(c)$ добавляет c в качестве первого элемента списка lc , вызов $lc.pop_front()$ удаляет первый элемент, а $lc.front()$ возвращает значение первого элемента списка.

Любая функция может создать объект класса **List_stack** и передать его функции $f()$:

```
void h ()
{
    List_stack ls;
    f(ls) ;
}
```


2.5.5. Виртуальные функции

Зададимся вопросом, как же вызов `s_ref.pop()` в теле функции `f()` связывается с надлежащим определением функции? Ведь когда функция `f()` вызывается из функции `h()`, должна работать функция `List_stack::pop()`. А когда `f()` вызывается из `g()`, должна работать иная функция — функция `Array_stack::pop()`. Для правильного разрешения вызова функций объекты классов, производных от `Stack`, должны содержать дополнительную информацию, которая прямо указывает, какая функция должна быть вызвана во время выполнения программы. Обычно, компиляторы транслируют имя виртуальной функции в соответствующий индекс в *таблице, содержащей указатели на функции*. Такую таблицу принято называть «*таблицей виртуальных функций*», или просто *vtbl*. Каждый класс с виртуальными функциями имеет свою собственную таблицу *vtbl*, идентифицирующую эти функции. Графически это можно отобразить следующим образом:



Таблицы виртуальных функций позволяют корректно использовать объекты даже в тех случаях, когда на момент вызова функции неизвестны ни размеры объекта, ни местоположение его данных. Единственное, что нужно знать вызывающей стороне — это расположение таблицы *vtbl* класса и индексы в этой таблице для вызова виртуальных функций. Механизм вызова виртуальных функций по сути столь же эффективен, как и вызов обычных функций. Дополнительные затраты памяти сводятся к одному указателю на каждый объект класса с виртуальными функциями и на таблицу *vtbl* для самого класса.

2.6. Объектно-ориентированное программирование

Абстракция данных чрезвычайно важна для выполнения качественного проектирования и она будет в центре нашего внимания на протяжении всей книги. Однако типы, определяемые пользователем, сами по себе не настолько гибкие, чтобы удовлетворить все наши потребности. В настоящем разделе сначала демонстрируется проблема, связанная с простым пользовательским типом, а затем показывается, как ее преодолеть с помощью классовых иерархий.

2.6.1. Проблемы, связанные с конкретными типами

Конкретный тип, вроде «псевдотипа», реализованного в виде модуля, представляется таким черным ящиком. Будучи определенным, черный ящик практически не взаимодействует с остальными частями программы. Приспособить его для других целей можно лишь, модифицировав его определение. С одной стороны, можно считать такое свойство конкретных типов как идеальное, а с другой стороны — это источник чрезвычайной негибкости. Рассмотрим сейчас определение типа *Shape*, который будет использоваться в графической системе. Для начала считаем, что графическая система должна поддерживать окружности (circles), треугольники (triangles) и квадраты (squares). Кроме того, предполагаем, что у нас есть

```
class Point { /* ... */ };
class Color { /* ... */ };
```

Здесь */** и **/* маркируют начало и конец комментариев. Такая нотация подходит и для многострочных комментариев, и для комментариев, оканчивающихся где-то в середине строки (так что после них в той же строке располагается незакомментированный код).

Мы можем определить тип *Shape* приблизительно так:

```
enum Kind { circle, triangle, square }; // перечисление (§4.8)

class Shape
{
    Kind k; // поле muna
    Point center;
    Color col;
    // ...

public:
    void draw ();
    void rotate (int);
    // ...
};
```

Поле *k*, которое можно назвать *полем muna* (*type field*), требуется, чтобы операции, вроде *draw()* или *rotate()*, могли установить, с каким типом фигуры они имеют дело. При этом код функции *draw()* может быть устроен следующим образом:

```
void Shape::draw ()
{
    switch (k)
    {
        case circle:
            // рисуем окружность
            break;
        case triangle:
            // рисуем треугольник
            break;
        case square:
            // рисуем квадрат
            break;
```

Это просто несъедобное варево. Получается, что функции вроде *draw()*, обязаны знать обо всех типах фигур, с которыми система имеет дело. Код таких функций растет по мере добавления в систему новых видов обрабатываемых фигур. Каждый раз, когда мы добавляем новую фигуру, нужно тщательно изучить код каждой операции и (возможно) изменить его. Если у нас нет доступа к исходному коду каждой операции, то мы вообще не можем добавлять новые фигуры. Поскольку добавление новых фигур означает вторжение в код важных операций, оно требует мастерства и потенциально чревато внесением ошибок в код, обрабатывающий старые фигуры. Выбор представления для некоторых фигур может оказаться слишком тесным, если потребуется втиснуть его в типично жесткие рамки представления общего типа *Shape*.

2.6.2. Классовые иерархии

Проблема заключается в том, что не видно разницы между *общими свойствами* всех фигур (любая фигура обладает определенным цветом, может быть нарисована и т.д.) и *особыми свойствами* отдельных их типов (окружность характеризуется радиусом и рисуется специфическим способом). *Объектно-ориентированное программирование* призвано выявлять такую разницу и извлекать из этого максимальную пользу. Языки, которые располагают средствами, позволяющими явно выразить указанную разницу и использовать этот факт, поддерживают объектно-ориентированное программирование. А остальные языки — нет.

Механизм наследования (заимствованный в C++ из Simula) решает проблему. Сначала мы определяем класс, который задает общие свойства всех фигур:

```
class Shape
{
    Point center;
    Color col;
    // ...

public:
    Point where() {return center;}
    void move(Point to) {center = to; /* ... */ draw();}

    virtual void draw() = 0;
    virtual void rotate(int angle) = 0;
    // ...
};
```

Аналогично рассмотренному в §2.5.4 абстрактному типу *Stack*, функции, для которых интерфейс вызова может быть сформулирован, а реализация — нет, объявляются *виртуальными*. В частности, функции *draw()* и *rotate()* могут определяться лишь для конкретных типов фигур. Поэтому они и объявлены виртуальными.

Отталкиваясь от определения класса *Shape*, мы можем писать *универсальные* функции, которые манипулируют векторами указателей на фигуры:

```
void rotate_all(vector<Shape*> &v, int angle)    // поворачиваем элементы v
                                                // на angle градусов
{
    for (int i = 0; i < v.size(); ++i) v[i] -> rotate(angle);
}
```

Чтобы задать конкретную фигуру, мы должны, во-первых, указать, что это фигура вообще, а во-вторых, определить дополнительную специфику (включая виртуальные функции):

```
class Circle : public Shape
{
    int radius;

public:
    void draw() { /* ... */ }
    void rotate(in) {} // да, функция ничего не делает
};
```

В языке C++ говорят, что класс **Circle** *наследуется от* (*is derived from*) класса **Shape**, а класс **Base** называется *базовым* (*base*) для класса **Circle**. Альтернативная терминология называет **Circle** и **Shape** *подклассом* и *суперклассом* (*subclass* и *superclass*), соответственно. Производный класс наследует все члены базового класса, а потому эту тему принято называть *наследованием* (*inheritance*) классов. Новая парадигма программирования звучит так:

*Решите, какие классы нужны;
обеспечьте полный набор операций для каждого класса,
явно выразите общность посредством наследования.*

Там, где нет общности типов, достаточно одной лишь абстракции данных. Степень общности между типами, которую можно выявить и отразить с помощью наследования и виртуальных функций, является лакмусовой бумажкой для определения того, насколько применим объектно-ориентированный подход к данной задаче. В некоторых предметных областях, таких как интерактивная графика, объектно-ориентированный подход применим широчайшим образом. В других предметных областях, например в области математических вычислений, достаточно абстракции данных, а применение средств объектно-ориентированного программирования представляется ненужным или избыточным.

Выявление общности между типами не является тривиальной задачей. Степень общности, которую можно найти, зависит от способа проектирования программной системы. Начинать искать общность типов нужно уже на начальной стадии проектирования, когда имеются лишь технические требования к системе. Классы могут специально проектироваться как строительные блоки для других классов. Уже существующие классы можно подвергнуть анализу на предмет выявления у них общих черт, которые можно выделить в общий базовый класс.

Попытки рассмотреть вопрос о том, что такое объектно-ориентированное программирование, не обращая при этом к конкретным конструкциям языков программирования, предприняты в работах [Кеп, 1987], и [Booch, 1994] (см. §23.6).

Иерархии классов и абстрактные классы (§2.5.4) дополняют, а не исключают друг друга (§12.5). Вообще, перечисленные здесь парадигмы имеют тенденцию к комплиментарности и взаимоподдержке. Например, и классы, и модули содержат функции, в то время как модули содержат и классы, и функции. Опытный разработчик применяет различные парадигмы по мере необходимости.

2.7. Обобщенное программирование

Тому, кто захочет использовать стек, вряд ли всегда будет требоваться именно стек символов. Стек это более общее понятие, не зависящее от символов. Соответственно, его желательно и определять независимым образом.

Вообще, если алгоритм можно выразить независимо от деталей его реализации, причем приемлемым образом и без логических искажений, то так и надо делать.

Соответствующая программная парадигма звучит так:

*Решите, какие нужны алгоритмы;
параметризируйте их так, чтобы они могли работать
со множеством подходящих типов и структур данных.*

2.7.1. Контейнеры

Мы можем *обобщить* стек символов до *стека элементов любого типа*, если объявим его *шаблоном*, а тип элементов *char* заменим на *параметр шаблона*. Вот соответствующий пример:

```
template<class T> class Stack
{
    T* v;
    int max_size;
    int top;

public:
    class Underflow { };
    class Overflow { };

    Stack (int s);           // конструктор
    ~Stack ();               // деструктор

    void push (T);
    T pop ();
};
```

Префикс *template<class T>* сообщает следующему за ним объявлению, что *T* — это *параметр типа*.

Функции-члены определяются аналогичным образом:

```
template<class T> void Stack<T>::push (T c)
{
    if (top == max_size) throw Overflow ();
    v[top] = c;
    top = top + 1;
}

template<class T> T Stack<T>::pop ()
{
    if (top == 0) throw Underflow ();
    top = top - 1;
    return v[top];
}
```

Теперь приведем пример кода, использующего новое определение стека:

```
Stack<char> sc (200) ;           // стек для 200 символов
Stack<complex> scplx (30) ;      // стек для 30 комплексных чисел
Stack< list<int> > sli (45) ;    // стек для 45 списков целых

void f()
{
    sc.push ( 'c' ) ;
    if (sc.pop () != 'c' ) throw Bad_pop () ;

    scplx.push (complex (1, 2) ) ;
    if (scplx.pop () != complex (1, 2) ) throw Bad_pop () ;
}
```

С помощью шаблонов можно также определять списки, вектора, ассоциативные массивы и т.д. Классы, предназначенные для хранения набора элементов некоторого типа, принято называть *контейнерными классами* (*container classes*), или просто *контейнерами*.

Механизм шаблонов автоматически обрабатывается на стадии компиляции программы и не вносит дополнительных накладных расходов на стадии ее выполнения (что типично для ручного кодирования).

2.7.2. Обобщенные алгоритмы

Стандартная библиотека языка C++ содержит богатый набор контейнеров, кроме того, пользователи могут разрабатывать собственные контейнеры (главы 3, 17, 18). В результате мы сталкиваемся с ситуацией, когда желательно еще раз применить парадигму обобщенного программирования и *параметризовать алгоритмы по типам контейнеров*. Например, мы хотим сортировать, копировать элементы и осуществлять поиск в векторах, списках и массивах, но не хотим писать отдельные варианты функций *sort()*, *copy()* и *search()* для каждого из типов контейнеров. Кроме того, вряд ли мы захотим жестко фиксировать структуру данных, принимаемую единственными вариантами функций. Поэтому возникает задача обобщенного доступа к элементам контейнера, который не зависит от типа обрабатываемого контейнера.

Один из подходов, а именно подход к контейнерам и нечисловым алгоритмам, принятый в стандартной библиотеке языка C++ (§3.8, глава 18), фокусируется на понятии *последовательностей элементов* (*sequences*) и на манипулировании ими с помощью *итераторов* (*iterators*).

Понятие последовательности элементов графически иллюстрируется следующим образом:



У последовательности элементов (*elements*) есть начало (*begin*) и конец (*end*). Итератор ссылается на элемент последовательности, и для него существует операция, позволяющая ему перейти к следующему элементу последовательности. Конец последовательности представляется итератором, ссылающимся на нечто, расположенное за последним элементом последовательности. Физическое представление

«конца» жестко не фиксируется; им может быть специальный «элемент-часовой» (*sentinel element*), или что-то другое. Рассмотренное понятие последовательности элементов одинаково применимо и к спискам, и к массивам.

Нам требуются стандартные обозначения для операций «доступ к элементам посредством итератора» и «перевод итератора к следующему элементу». Очевидным выбором (при понимании общей идеи) является выбор стандартных знаков операции *** (операция разыменования — *dereference operator*) и операции *++* (операция инкремента — *increment operator*), соответственно.

Теперь мы можем писать следующий код:

```
template<class In, class Out> void copy (In from, In too_far, Out to)
{
    while (from != too_far)
    {
        *to = *from;           // копируем элемент указуемый итератором
        ++to;                  // следующий выходной элемент
        ++from;                 // следующий входной элемент
    }
}
```

Этот обобщенный код копирует элементы любых контейнеров, для которых можно определить итераторы с правильным синтаксисом и семантикой.

Наш обобщенный копирующий алгоритм можно также применить и ко встроенным в C++ низкоуровневым массивам и указателям, поскольку требующиеся операции для них определены:

```
char vc1[200];           // массив из 200 символов
char vc2[500];           // массив из 500 символов

void f()
{
    copy (&vc1[0], &vc1[200], &vc2[0]);
}
```

Здесь все элементы массива *vc1* копируются в начало массива *vc2*.

Все контейнеры стандартной библиотеки (§16.3, глава 17) поддерживают концепции последовательностей и итераторов.

У нас в шаблоне используются два параметра типа, а не один; параметр *In* относится к типу контейнера-источника, а параметр *Out* — к типу контейнера приемника. Это позволяет копировать элементы контейнера одного типа в контейнер другого типа. Например:

```
complex ac[200];

void g (vector<complex>& vc, list<complex>& lc)
{
    copy (&ac[0], &ac[200], lc.begin());
    copy (lc.begin(), lc.end(), vc.begin());
}
```

Этот код копирует массив в контейнер типа *list*, а затем последний копируется в контейнер типа *vector*. Для любого стандартного контейнера функциональный вызов *begin()* возвращает итератор, указывающий на первый элемент последовательности.

2.8. Заключение

Не существует идеальных языков программирования. К счастью, язык и не обязан быть идеальным, чтобы быть подходящим инструментом для создания больших программных систем. На самом деле, язык программирования общего назначения не может быть идеальным одновременно для всех решаемых с его помощью задач. То, что идеально для одной задачи, в другой является скорее недостатком, ибо достижение совершенства предполагает специализацию. Язык C++ разрабатывался так, чтобы он служил хорошим инструментом для решения широкого круга задач, и чтобы на нем можно было явно выразить широкий круг идей (идиом программирования).

Не все можно выразить непосредственно, пользуясь встроенными средствами языка. Но это и не нужно. Средства языка существуют для поддержки разнообразных технологий и стилей программирования. Как следствие, в процессе изучения языка нужно фокусироваться на освоении стилей программирования, для него родных и естественных, а вовсе не на подробнейшем изучении мельчайших деталей языковых конструкций.

В практическом программировании мало толку от знания «потайных мест» языка или от применения максимально возможных средств. Отдельные средства языка сами по себе не представляют большого интереса. Только во взаимодействии с другими средствами и в контексте применяемой технологии они приобретают смысл и интерес. Так что, читая последующие главы, помните, что истинной целью изучения деталей языка является стремление научиться использовать их так, чтобы поддерживать выбранный стиль программирования в контексте основательного плана построения программной системы.

2.9. Советы

1. Не паникуйте раньше времени! Все со временем станет понятным; §2.1.
2. Для написания хороших программ вы не обязаны знать каждую деталь языка C++; §1.7.
3. Сосредоточьте внимание на технологиях программирования, а не на деталях языка; §2.1.

Обзор стандартной библиотеки

*Зачем тратить время на обучение,
если невежество достигается мгновенно?*
— Хоббс

Стандартная библиотека — вывод — строки — ввод — вектора — проверка диапазона — списки — ассоциативные массивы — обзор контейнеров — алгоритмы — итераторы — итераторы ввода/вывода — предикаты — алгоритмы, использующие функции-члены классов — обзор алгоритмов — комплексные числа — векторная арифметика — обзор стандартной библиотеки — советы.

3.1. Введение

Ни одна значительная программа не написана исключительно лишь на «голых» конструкциях языка программирования. Сначала разрабатываются библиотеки поддержки. Затем они становятся основой для дальнейшей работы.

В продолжении главы 2 здесь дается краткий обзор основных библиотечных средств, дающий представление о том, что в итоге может быть создано с помощью языка C++ и его стандартной библиотеки. Рассматриваются такие полезные типы стандартной библиотеки, как *string*, *vector*, *list* и *map*, а также наиболее употребительные варианты их использования. Это позволяет мне приводить более осмысленные примеры и давать более интересные упражнения в последующих главах книги. Как и в главе 2, здесь также настоятельно рекомендуется не отвлекаться и не огорчаться из-за неполного понимания деталей излагаемого материала. Цель настоящей главы состоит в том, чтобы дать вам почувствовать, с чем придется столкнуться при дальнейшем обучении, а также помочь овладеть простейшими способами использования наиболее полезных средств библиотеки. Подробное введение в стандартную библиотеку дано в §16.1.2.

Элементы стандартной библиотеки, которые изучаются в настоящей книге, входят составной частью в любое достаточно полное средство разработки программ на языке C++ (именуемое также средой разработки). В дополнение к стандартной библиотеке

языка C++ эти среды часто содержат средства реализации графического интерфейса пользователя (оконных систем или GUI — Graphical User Interface), предназначенного для удобного визуального взаимодействия пользователя с работающей программой. Кроме того, многие среды поставляются еще и с разного рода «фундаментальными библиотеками», призванными поддерживать корпоративные или промышленные стандарты разработки и/или выполнения программ. Я не рассматриваю такие дополнительные средства и библиотеки. Я просто хочу дать полное описание языка C++ в соответствии со стандартом, а также обеспечить переносимость всех примеров кода, за исключением особо оговариваемых случаев. Естественно, программист может сам, в качестве самостоятельных упражнений, ознакомиться и овладеть дополнительными средствами и библиотеками, поставляемыми с конкретной средой разработки.

3.2. Hello, world! (Здравствуй, мир!)

Вот пример минимальной программы на языке C++:

```
int main () { }
```

Она определяет функцию *main* (), не принимающую аргументов и ничего не делающую.

Каждая программа на C++ должна иметь функцию с именем *main*. Это точка входа в программу, с которой и начинается ее выполнение. Значение типа *int*, возвращаемое программой (если оно вообще имеется, конечно), предназначено для системы, запустившей программу. Если возврат отсутствует, то система получит сообщение об успешном завершении программы. Ненулевой возврат означает аварийное завершение.

В типичном случае программы выводят некоторую информацию о своей работе. Например, следующая программа выводит строку *Hello, world!*:

```
#include <iostream>

int main ()
{
    std::cout << "Hello, world!\n";
}
```

Строка *#include <iostream>* заставляет компилятор *включить* сюда все объявления стандартных потоковых средств ввода-вывода (I/O facilities) из файла *iostream*. Без этих объявлений выражение

```
std::cout << "Hello, world!\n"
```

не имело бы смысла. Операция << («вставить в поток») осуществляет вывод своего правого аргумента в левый аргумент. В данном случае, строковый литерал *"Hello, world!\n"* записывается в стандартный поток вывода *std::cout*. *Строковый литерал* (*string literal*) является последовательностью символов, заключенных в двойные кавычки. В строковых литералах символ \ (обратная наклонная черта) и следующий за ним в совокупности означают один специальный символ. В данном случае, \n означает специальный символ «перевода строки», так что сначала выводится текст *Hello, world!*, а затем осуществляется переход на новую строку (на новую строку устройства вывода).

3.3. Пространство имен стандартной библиотеки

Стандартная библиотека определена в пространстве имен *std* (§2.4, §8.2). Вот почему я пишу *std::cout*, а не просто *cout*. То есть я указываю явным образом, что используется именно *cout* из стандартной библиотеки, а не какой-то другой *cout*.

Любое средство стандартной библиотеки объявлено в каком-либо *стандартном заголовочном файле*, аналогичном файлу *iostream*. Например:

```
#include <string>
#include <list>
```

Теперь доступны такие типы стандартной библиотеки, как *string* и *list*. Для этого нужно явно использовать префикс *std::*, как в следующем примере:

```
std::string s = "Four legs Good; two legs Baaad!";
std::list<std::string> slogans;
```

Однако для упрощения записи примеров в дальнейшем я буду редко использовать префикс *std::*, и также редко буду явно включать в код примеров строки *#include* с необходимыми стандартными заголовочными файлами. Чтобы скомпилировать и запустить на выполнение код примеров, вы сами должны включить все необходимые заголовочные файлы (перечислены в §3.7.5, §3.8.6 и §16.1.2). Кроме того, вы должны или явно добавлять префикс *std::*, или делать все имена из *std* глобально доступными (§8.2.3). Например:

```
#include <string> // включает объявления стандартных средств работы со строками
using namespace std; // делает имена из std доступными без префикса std::
string s = "Ignorance is bliss!"; // ok: string означает std::string
```

В общем случае, «сброс» имен из *std* в одно глобальное пространство имен является дурным тоном. И, тем не менее, чтобы укоротить тексты примеров, описывающих применение средств C++ и библиотеки, я часто опускаю повторяющиеся строки *#include* и *std::* префиксы. В настоящей книге я использую почти исключительно средства стандартной библиотеки языка C++, так что, если в примере встречается имя из стандартной библиотеки, то это либо демонстрация его стандартного применения, либо фрагмент, поясняющий, как стандартное библиотечное решение может быть определено.

3.4. Вывод

В стандартной библиотеке с помощью заголовочного файла *iostream* определяются *средства вывода* для каждого встроенного типа. Кроме того, несложно расширить эти средства и на пользовательские типы. По умолчанию, величины, выводимые в поток *cout*, автоматически преобразуются в *последовательный набор символов*. Например, следующий код

```
void f()
{
    cout<< 10;
```

поместит в стандартный поток вывода сначала символ **1**, а затем символ **0**. То же самое происходит и в следующем примере:

```
void g ()
{
    int i = 10;
    cout<< i;
}
```

Данные разных типов смешиваются очевидным образом:

```
void h (int i)
{
    cout<< "the value of i is ";
    cout<< i;
    cout<< '\n' ;
}
```

Если *i* равно 10, то в поток вывода попадет следующая последовательность символов:

the value of i is 10

Символьной константой (character constant) называется символ, заключенный в одиночные кавычки. Отметим, что символьные константы выводятся в поток именно как символы, а не как соответствующие им числовые значения. Например, код

```
void k ()
{
    cout<< 'a' ;
    cout<< 'b' ;
    cout<< 'c' ;
}
```

поместит в выходной поток последовательность *abc*.

Утомительно то и дело повторять имя стандартного потока вывода при последовательном выводе нескольких величин. К счастью, результат операции `<<` можно использовать для последующей операции вывода. Например:

```
void h2 (int i)
{
    cout<< "the value of i is "<<i<< '\n' ;
}
```

По своему действию эта функция эквивалентна представленной ранее функции *h()*. Подробнее потоки рассматриваются в главе 21.

3.5. Строки

В стандартной библиотеке определен тип *string*, дополняющий рассмотренные ранее строковые литералы. Тип *string* обеспечивает множество полезных операций, таких как *конкатенация (concatenation)*. Например:

```
string s1 = "Hello";
string s2 = "world";

void m1 ()
{
    string s3 = s1 + " " + s2 + "!\n";
    cout<<s3;
}
```

Здесь строка *s3* инициализируется значением (текстом)

Hello, world!

со специальным знаком «перевод строки» на конце. Сложение строк означает их конкатенацию. Строки можно складывать со строками, строковыми литералами и символами.

Во многих приложениях конкатенация реализуется в форме добавления чего-либо к концу заданной строки, что напрямую поддерживается операцией `+=`. Например:

```
void m2 (string& s1, string& s2)
{
    s1 = s1 + '\n'; // добавить к концу содержимого символ перевода строки
    s2 += '\n';     // добавить к концу содержимого символ перевода строки
}
```

Показанные способы добавления к концу строки семантически эквивалентны, но я предпочитаю второй из них, так как он короче и, к тому же, реализуется эффективнее в машинном коде.

Естественно, что строки можно сравнивать между собой и со строковыми литералами. Например:

```
string incantation;

void respond (const string& answer)
{
    if (answer==incantation)
    {
        // сотворим чудо (incantation - колдовство, заклинание)
    }
    else if (answer=="yes")
    {
        // ...
    }
    // ...
}
```

Класс **string** из стандартной библиотеки описывается в главе 20. Помимо прочих полезных вещей он предоставляет возможность работы с подстроками. Например:

```
string name="Niels Stroustrup";

void m3 ()
{
    string s = name.substr (6, 10); // s = "Stroustrup"
```

```
name.replace (0, 5, "Nicholas") ; // name становится "Nicholas Stroustrup"
}
```

Операция¹ **substr**() возвращает копию подстроки, задаваемой аргументами. Первый аргумент означает индекс (позицию) начала подстроки, а второй — ее длину. Так как начальным значением индекса является ноль, то строка *s* принимает значение **Stroustrup**.

Операция **replace**() замещает исходное значение подстроки на новое. В данном случае, подстрока начинается с индекса 0 и имеет длину 5, так что это **Niels**; она замещается на **Nicholas**; таким образом, окончательное значение строки *name* есть **Nicholas Stroustrup**. Дополнительно отметим, что длина замещаемой подстроки и нового значения не обязаны совпадать.

3.5.1. С-строки

Как известно, С-строки (или строки языка С) — это массивы символов с терминальным нулем (§5.2.2). С-строки легко преобразуются в значения типа **string**. Но может потребоваться и обратное преобразование, например, когда имеется значение типа **string**, а нужно вызвать функцию, принимающую аргумент типа С-строки. Его-то и нужно тогда извлечь из значения типа **string** с помощью функции **c_str**() (§20.3.7). Например, мы можем вывести значение строки *name* библиотечной функцией **printf**() языка С (§21.8):

```
void f()
{
    printf( "name: %s\n", name.c_str() );
}
```

3.6. Ввод

В стандартной библиотеке **istream** управляет посимвольным вводом встроенных типов данных точно так же, как рассмотренный выше тип **ostream** управляет их выводом. Действие типа **istream** несложно распространить и на пользовательские типы.

Для обозначения операции ввода используется знак >> («извлечь из потока»), а **cin** обозначает *стандартный поток ввода*. Тип правого операнда операции >> определяет, что именно вводится и куда записывается результат ввода. Например, следующий код

```
void f()
{
    int i;
    cin >> i; // считать целое значение в i
    double d;
```

¹ Здесь, и во многих местах далее, автор использует слово «операция» как синоним слова «действие» в самом широком смысле. В строгом же смысле, здесь имеет место функциональный вызов. — *Прим. ред.*

```
cin>>d;    // считать значение с плавающей запятой двойной точности в d
}
```

читает число, вроде **1234**, из потока ввода в целую переменную **i** и значение с плавающей запятой, вроде **12.34e5**, в переменную **d** типа **double** (числа с плавающей запятой двойной точности).

Ниже дан пример программы, осуществляющей преобразования из дюймов в сантиметры и обратно. Вы вводите число и символ, обозначающий единицу измерения: сантиметры (символ '**c**') или дюймы (символ '**i**'). Программа выведет соответствующее значение в других единицах измерения:

```
int main ()
{
    const float factor = 2.54;    // 1 дюйм равен 2.54 cm
    float x, in, cm;
    char ch = 0;

    cout<<"enter length: ";

    cin>>x;                        // считать число с плавающей запятой
    cin>>ch;                       // считать суффикс

    switch (ch)
    {
        case 'i':                 // дюймы
            in=x;
            cm=x*factor;
            break;
        case 'c':                 // сантиметры
            in=x/factor;
            cm=x;
            break;
        default:
            in=cm=0;
            break;
    }

    cout<<in<<" in= "<<cm<<" cm\n";
}
```

Оператор **switch** сравнивает значение с набором констант. Оператор **break** осуществляет выход из оператора **switch**. Константы в разных **case**-ветвях должны быть разными. Если проверяемое значение не совпадает ни с одной из них, то выбирается **default**-ветвь. Программист, в общем случае, не обязан реализовывать **default**-ветвь.

Часто требуется прочитать последовательность символов. В этом случае удобно помещать результат ввода в объект типа **string**. Вот соответствующий пример:

```
int main ()
{
    string str;

    cout<<"Please, enter your name\n";
    cin >> str;
```

```
cout<<"Hello, "<<str<<"!\n";
}
```

Если вы введете

Eric

то вывод будет таким:

Hello, Eric!

По умолчанию, символ-разделитель (§5.2.2), например, пробел, прерывает ввод, так что если вы введете

Eric Bloodaxe

притворяясь несчастным королем Йорка, то все равно на выходе получите то же самое:

Hello, Eric!

Считать целиком всю строку можно с помощью функции *getline()*, например:

```
int main ()
{
    string str;

    cout<<"Please, enter your name\n";
    getline (cin, str);
    cout<<"Hello, "<<str<<"!\n";
}
```

Теперь при вводе

Eric Bloodaxe

на выходе мы получим желаемое:

Hello, Eric Bloodaxe!

Тип *string* имеет чудесное свойство принимать на хранение тексты любого размера, так что если вы вдруг введете пару мегабайт двоеточий, то программа выведет вам в ответ всю эту кучу знаков — если только компьютер или операционная система не исчерпают при этом какой-либо критический ресурс.

3.7. Контейнеры

Часто в процессе решения самых разных задач удобно создать коллекцию (набор) объектов некоторого типа, чтобы потом такой коллекцией манипулировать. Простейшим примером является процесс чтения символов в строку и последующий вывод строки на дисплей (или принтер). Класс, *основной целью* которого является *хранение объектов*, называется *контейнером*. Выбор подходящих контейнеров и реализация фундаментальных методов работы с ними, достаточных и удобных для решения конкретной задачи, являются важным этапом разработки программ.

Для иллюстрации наиболее важных контейнеров стандартной библиотеки рассмотрим простую программу, хранящую имена и номера телефонов. Это задача,

различные подходы к решению которой хорошо понятны специалистам разных профилей.

3.7.1. Контейнер *vector*

Для большинства программистов на С встроенный массив пар значений (имя, телефонный номер) является естественной точкой отсчета:

```
struct Entry                // запись телефонной книги
{
    string name;
    int number;
};

Entry phone_book [1000];    // телефонная книга на 1000 записей

void print_entry (int i)     // использование (вывод записи)
{
    cout<<phone_book [i].name<<' ' << phone_book [i].number<<' \n' ;
}
```

Однако встроенные массивы имеют жестко фиксированные размеры. Если мы выберем избыточный размер, то впустую израсходуем память; если же размер будет недостаточным, то произойдет переполнение. В любом случае, придется писать низкоуровневый код по управлению выделением памяти. Контейнер *vector* из стандартной библиотеки (§16.3) берет на себя эту заботу:

```
vector<Entry> phone_book (1000) ;

void print_entry (int i)      // используем как раньше
{
    cout<<phone_book [i].name <<' ' <<phone_book [i].number<<' \n' ;
}

void add_entries (int n)      // увеличиваем размер книги на n записей
{
    phone_book.resize (phone_book.size () +n) ;
}
```

Функция *size* (), член класса *vector*, возвращает количество размещенных элементов.

Обратите внимание на круглые скобки в определении объекта *phone_book*. Мы создали единственный объект типа *vector*<*Entry*> и проинициализировали его начальным размером. Это разительно отличается от объявления встроенного массива:

```
vector<Entry> book (1000) ;    // vector из 1000 элементов
vector<Entry> books [1000] ;   // 1000 пустых векторов
```

Если в объявлении объекта вы по ошибке используете квадратные скобки вместо круглых, то компилятор наверняка обнаружит ошибку в тот момент, когда вы попытаетесь использовать объект.

Объекты типа *vector* могут участвовать в операциях присваивания:

```
void f (vector<Entry> & v)
{
    vector<Entry> v2 = phone_book ;
}
```

```

v=v2;
// ...
}

```

При этом копируются все элементы векторов. Таким образом, после инициализации и операции присваивания в функции *f()* векторы *v* и *v2* содержат независимые копии объектов типа *Entry* из телефонной книги. Когда вектора содержат много элементов, невинно выглядящие инициализация и присваивание могут оказаться недопустимо дорогостоящими операциями. В таких случаях лучше использовать ссылки или указатели.

3.7.2. Проверка диапазона индексов

Стандартный тип *vector* по умолчанию никаких проверок индексов не производит (§16.3.3). Например:

```

void f()
{
    int i=phone_book[1001].number;           // 1001 вне диапазона индексов
    // ...
}

```

Вероятнее всего, такое присваивание поместит в переменную *i* некоторое неконтролируемое значение, а не породит сообщение об ошибке. Это нежелательно, и поэтому в следующих разделах я буду использовать контейнер *Vec* — простую адаптацию класса *vector*, проверяющую, попадает ли индекс в допустимый диапазон значений. Тип *Vec* во всем эквивалентен типу *vector*, но он генерирует исключение типа *out_of_range* (определенное в файле *<stdexcept>*), когда индекс выходит за границы допустимого диапазона.

Технологии создания типов, подобных *Vec*, и эффективного использования исключений рассматриваются в §11.12, §8.3, главе 14 и Приложении Е. Однако для иллюстрирующих примеров в настоящей книге вполне достаточно следующего определения:

```

template<class T> class Vec : public vector<T>
{
public:
    Vec() : vector<T>() {}
    Vec(int s) : vector<T>(s) {}

    T& operator[] (int i) {return at(i); }           // с проверкой диапазона
    const T& operator[] (int i) const {return at(i); } // с проверкой диапазона
};

```

Для доступа к элементам векторов по индексу класс *vector* определяет функцию *at()*, генерирующую исключение типа *out_of_range* при выходе индекса за границы допустимого диапазона (§16.3.3). При необходимости, доступ к базовому типу *vector<T>* может быть заблокирован; см. §15.3.2.

Возвращаясь к задаче о хранении имен и телефонных номеров, мы можем теперь смело применить класс *Vec*, будучи полностью уверенными в том, что выход индексов за границу допустимого диапазона будет надежно перехвачен. Например:

```
Vec<Entry> phone_book (1000) ;

void print_entry (int i)                                // используем как раньше
{
    cout<<phone_book[i].name<<' ' <<phone_book[i].number<<' \n' ;
}
```

Выход индекса за границу допустимого диапазона вызовет генерацию исключения, которое пользователь может перехватить. Например:

```
void f()
{
    try
    {
        for (int i=0; i<10000; i++) print_entry (i) ;
    }
    catch (out_of_range)
    {
        cout<<"range error\n" ;
    }
}
```

Исключение будет сгенерировано и перехвачено при попытке вычисления выражения *phone_book[i]* в момент, когда выполнится условие *i == 1000*. Если пользователь не перехватит это исключение, программа будет принудительно снята с выполнения стандартным образом, а не продолжит выполнение с непредсказуемыми последствиями. Для минимизации сюрпризов, связанных с исключениями, в качестве тела стартовой функции *main()* можно использовать *try*-блок:

```
int main ()
try
{
    //ваш код
}
catch (out_of_range)
{
    cerr<<"range error\n" ;
}
catch (...)
{
    cerr<<"unknown exception thrown\n" ;
}
```

Если мы не перехватим исключение, связанное с выходом за границу допустимого диапазона индексов, или иные исключения, то сообщение об ошибке будет записано в стандартный поток ошибок *cerr* (§21.2.1).

3.7.3. Контейнер list

Включение новых абонентов в телефонный справочник и удаление из него ненужных записей являются распространенными операциями. Из-за этого, для телефонного справочника больше подходит *список (list)*, а не вектор. Например:

```
list<Entry> phone_book ;
```

Если мы используем список, то лишаемся доступа к элементам по индексу. Вместо этого, мы осуществляем поиск элемента с заданным значением. При этом мы опираемся на идею о представлении контейнеров в виде последовательности элементов (см. §3.8):

```
void print_entry (const string& s)
{
    typedef list<Entry> : : const_iterator LI;
    for (LI i=phone_book.begin() ; i != phone_book.end() ; ++i)
    {
        const Entry& e = *i;           // ссылка используется ради краткости записи
        if (s==e.name)
        {
            cout<<e.name<<' ' <<e.number<<' \n' ;
            return ;
        }
    }
}
```

Поиск строки *s* начинается с начала списка и продолжается либо до ее обнаружения в некотором элементе, либо завершается достижением конца списка. Любой контейнер стандартной библиотеки предоставляет функции ***begin()*** и ***end()***, возвращающие итераторы, которые настроены, соответственно, на первый элемент и на элемент, следующий «за последним» (§16.3.2). Если итератор *i* ссылается на некоторый элемент контейнера, то ++*i* ссылается на следующий элемент. Доступ к элементам контейнера осуществляется с помощью выражения ****i***.

Пользователям нет необходимости знать точный тип итераторов стандартных контейнеров. Тип итератора определяется в рамках определения контейнера и на него можно ссылаться по имени. Когда не нужно модифицировать элементы контейнера, целесообразно использовать итераторы типа ***const_iterator***. В противном случае нужно применять итераторы типа ***iterator*** (§16.3.1).

Добавлять элементы к списку и удалять их из него очень просто:

```
void f(const Entry& e, list<Entry> : : iterator i, list<Entry> : : iterator p)
{
    phone_book.push_front(e) ; // добавить в начало
    phone_book.push_back(e) ;  // добавить в конец
    phone_book.insert(i, e) ;   // добавить перед элементом, указуемым через i
    Phone_book.erase(p) ;      // удалить элемент, указуемый через p
}
```

Детальное описание функций ***insert()*** и ***erase()*** дано в §16.3.6.

3.7.4. Контейнер map

Писать программы для поиска имен в списках пар значений (имя, номер телефона) довольно утомительно. Кроме того, последовательный поиск эффективен лишь для коротких списков. В то же время, существуют специальные структуры данных, которые напрямую поддерживают внедрение элементов, их удаление и поиск по значению. В стандартной библиотеке, например, имеется контейнерный тип

map (§17.4.1). Тип **map** является контейнером для хранения пар величин. Например:

```
map<string, int> phone_book;
```

Тип **map** часто называют *ассоциативным массивом* (*associative array*) или *словарем* (*dictionary*).

Будучи индексирован по значению его первого типа (называемого *ключом* — *key*), **map** возвращает соответствующее значение его второго типа (называемого просто *значением* или *отображаемым типом* — *value* или *mapped type*). Например:

```
void print_entry(const string& s)
{
    if (int i=phone_book[s]) cout<<s<<' '<<i<<'\\n';
}
```

Если вхождение ключа *s* не обнаружено, возвращается некоторое значение «по умолчанию». Для целочисленных типов данных в контейнерах типа **map** таковым служит *нуль*. А в нашем случае *нуль* не является допустимым телефонным номером.

3.7.5. Контейнеры стандартной библиотеки

Контейнеры типов **map**, **list** и **vector** могут использоваться для представления телефонной книги. У каждого из них есть свои *сильные и слабые стороны*. Например, индексирование векторов является простой и эффективной операцией. Но с другой стороны, внедрение элемента между существующими элементами *вектора* — весьма затратная операция. Для *списков* ситуация во всем противоположная. Ассоциативный массив (**map**) подобен *списку пар* (ключ, значение), но он оптимизирован под поиск значений по заданному ключу.

В стандартной библиотеке реализовано множество полезных типов контейнеров, из которых программист всегда может выбрать тот, который больше всего подходит к особенностям конкретной задачи:

Стандартные контейнеры	
vector < <i>T</i> >	вектор переменного размера (§16.3)
list < <i>T</i> >	двусвязный список (§17.2.2)
queue < <i>T</i> >	очередь (§17.3.2)
stack < <i>T</i> >	стек (§17.3.1)
deque < <i>T</i> >	двусторонняя очередь (§17.2.3)
priority_queue < <i>T</i> >	очередь с приоритетом (§17.3.3.)
set < <i>T</i> >	множество (§17.4.3)
multiset < <i>T</i> >	множество, в котором ключи могут встречаться несколько раз (§17.4.4)
map < <i>key</i> , <i>val</i> >	ассоциативный массив (§17.4.1.)
multimap < <i>key</i> , <i>val</i> >	ассоциативный массив, в котором ключ (<i>key</i>) может встречаться несколько раз (§17.4.2.)

Подробнее стандартные контейнеры рассматриваются в §16.2, §16.3 и главе 17. Они объявляются в пространстве имен *std* и реализуются в заголовочных файлах *<vector>*, *<list>*, *<map>* и так далее (§16.2).

Стандартные контейнеры и их базовые операции разрабатывались так, чтобы с понятийной точки зрения они были очевидными. Более того, одни и те же операции для разных контейнеров имеют один и тот же смысл. В общем случае, базовые операции применимы ко всем типам контейнеров. Например, операция *push_back()* может использоваться (с разумной эффективностью) для добавления элементов в конец *векторов* и *списков*, а операция *size()* сообщает количество элементов для любого контейнера.

Такая *выразительная и семантическая однородность* помогает программистам создавать новые типы контейнеров, которые можно использовать так же, как и стандартные типы контейнеров. Примером может служить рассмотренный выше тип *Vec* (§3.7.2), проверяющий допустимость значений индексов. В главе 17 будет рассмотрен контейнерный тип *hash_map*. Однородность контейнерных интерфейсов *позволяет разрабатывать алгоритмы независимо от конкретных типов контейнеров*.

3.8. Алгоритмы

Структуры данных, вроде списка или вектора, сами по себе не очень-то и полезны. Чтобы их можно было использовать, требуются такие базовые операции, как добавление и удаление элементов. Более того, мы редко используем контейнеры исключительно с целью хранения объектов. Чаще всего, мы объекты сортируем, печатаем, удаляем, извлекаем их подмножества, выполняем поиск и т.д. Как следствие, в стандартной библиотеке наряду с общеупотребительными контейнерами содержатся и *общеупотребительные алгоритмы*. Например, определив операции *==* (равно) и *<* (меньше) для типа *Entry*, мы можем отсортировать вектор *vector<Entry>* и положить копии всех его уникальных элементов в список:

```
bool operator==(const Entry& a, const Entry& b) { return a.name==b.name; }
bool operator<(const Entry& a, const Entry& b) { return a.name<b.name; }

void f(vector<Entry>& ve, list<Entry>& le)
{
    sort( ve.begin(), ve.end() );
    unique_copy( ve.begin(), ve.end(), le.begin() );
}
```

Стандартные алгоритмы рассматриваются в главе 18. Они формулируются в терминах *последовательностей элементов* (*sequences of elements*) (§2.7.2). Последовательность специфицируется *парой итераторов*, указывающих на ее первый элемент, и на «элемент, следующий за последним». В примере функция *sort()* сортирует последовательность элементов, начиная с *ve.begin()* и заканчивая *ve.end()* — здесь это соответствует всем элементам вектора *ve*. Для операции записи достаточно указать лишь первый из элементов, подлежащих перезаписи. Если в операции участвует не один элемент, то будут перезаписаны все элементы целевого объекта, следующие за указанным.

Можно также добавить новые элементы в конец контейнера:

```
void f(vector<Entry>& ve, list<Entry>& le)
{
    sort(ve.begin(), ve.end());
    unique_copy(ve.begin(), ve.end(), back_inserter(le));    // добавить в конец le
}
```

С помощью **back_inserter()** элементы добавляются в *конец* контейнера, при этом емкость контейнера автоматически расширяется до необходимых размеров (§19.2.4). Таким образом, стандартные контейнеры и **back_inserter()** позволяют полностью избавиться от потенциально опасного управления памятью с помощью библиотечной функции **realloc()** языка C (§16.3.5).

Отметим, что если вы забудете про **back_inserter()** при добавлении элементов, то придете к разного рода ошибочным ситуациям. Например:

```
void f(vector<Entry>& ve, list<Entry>& le)
{
    copy(ve.begin(), ve.end(), le);           // ошибка: le не итератор
    copy(ve.begin(), ve.end(), le.end());      // очень плохо: просто пишем за конец
    copy(ve.begin(), ve.end(), le.begin());    // перезаписывает элементы
}
```

3.8.1. Использование итераторов

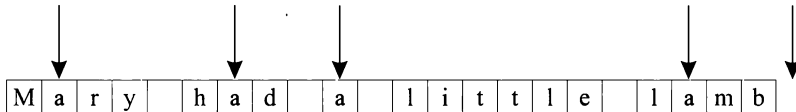
Располагая контейнером, легко получить итераторы, указывающие на характерные элементы контейнера, например, вызвав функции **begin()** и **end()**. Кроме того, многие алгоритмы возвращают итераторы. Например, *стандартный алгоритм find()* ищет значение в последовательности и возвращает итератор, указывающий на найденный элемент. Используя **find()**, можно найти число вхождений символа в строку:

```
int count(const string& s, char c)
{
    int n = 0;
    string::const_iterator i = find(s.begin(), s.end(), c);
    while (i != s.end())
    {
        ++n;
        i = find(i+1, s.end(), c);
    }
    return n;
}
```

Функция **find()** возвращает либо итератор, указывающий на найденный элемент, либо итератор, указывающий на «элемент, следующий за последним». Рассмотрим, что происходит, когда вызывается функция **count()**:

```
void f()
{
    string m = "Mary had a little lamb";
    int a_count = count(m, 'a');
```

Первый вызов алгоритма *find*() найдет символ 'a' в слове *Mary*. В результате мы входим в цикл, так как возвращаемый при этом итератор указывает на внутренний символ строки и, естественно, не равен *s.end()*. В цикле мы вызываем *find()*, используя *i+1* для начального итератора последовательности, то есть продолжаем поиск с символа, следующего за ранее найденным символом 'a'. В результате циклических вызовов алгоритма *find()* мы находим три остальных символа 'a'. В конце концов, *find()* возвратит значение, равное *s.end()*, так что условие *i != s.end()* станет ложным и цикл завершится. Все это удобно проиллюстрировать рисунком:



Здесь стрелки отображают начальное, промежуточные и конечное значения итератора *i*.

Естественно, алгоритм *find()* будет работать аналогично с любым другим стандартным контейнером. Поэтому мы могли бы обобщить функцию *count()* следующим образом:

```
template<class C, class T> int count(const C& v, T val)
{
    typename C::const_iterator i=find(v.begin(), v.end(), val); // typename см. §C.13.5
    int n = 0;

    while(i != v.end())
    {
        ++n;
        ++i; // пропустить только что найденный элемент
        i=find(i, v.end(), val);
    }
    return n;
}
```

Вот пример, использующий новый вариант функции *count()* (*mun complex* определен в файле `<complex>`; §3.9.1, §22.5):

```
void f(list<complex<double>> &lc, vector<string> &vs, string s)
{
    int i1 = count(lc, complex<double>(1, 3));
    int i2 = count(vs, "Chrysippus");
    int i3 = count(s, 'x');
}
```

На самом деле, нам нет нужды определять собственный функциональный шаблон *count()*. Задача подсчета числа вхождений элемента настолько важна, что стандартная библиотека предоставляет соответствующий алгоритм. Для большей универсальности алгоритм *count()* из стандартной библиотеки принимает последовательность в качестве аргумента, а не контейнер. Так что клиентскую функцию *f()* следует переписать в следующем виде:

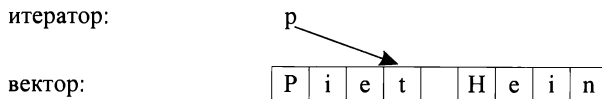

```
void f(list<complex<double> >& lc, vector<string>& vs, string s)
{
    int i1=count(lc.begin(), lc.end(), complex<double>(1, 3));
    int i2=count(vs.begin(), vs.end(), "Diogenes");
    int i3=count(s.begin(), s.end(), 'x');
}
```

Стандартный алгоритм `count()` может работать и со встроенными массивами, и с отдельными частями контейнеров. Например:

```
void g(char cs[], int sz)
{
    int i1=count(&cs[0], &cs[sz], 'z'); // число символов z в массиве
    int i2=count(&cs[0], &cs[sz/2], 'z'); // число символов z в первой половине массива
}
```

3.8.2. Типы итераторов

Чем на самом деле являются итераторы? В общем случае, про них можно сказать, что это объекты некоторого типа. Фактически же, *типы у разных итераторов разные*, так как конкретный итератор должен хранить информацию, позволяющую ему работать с конкретным типом контейнера. Типы итераторов могут отличаться в той же степени, в какой различаются типы контейнеров и цели, для достижения которых итераторы предназначены. Например, итераторы для контейнера **vector** скорее всего есть просто встроенные указатели, так как их удобно использовать для ссылок на индивидуальные элементы вектора:

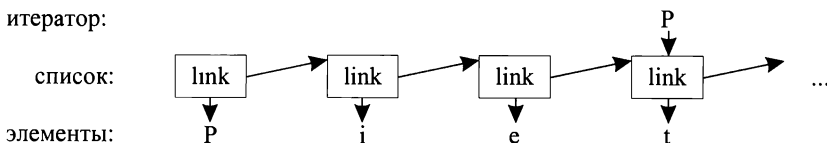


В качестве альтернативы можно реализовать итераторы в виде пары — указатель на начало **вектора** плюс целочисленное смещение:



Последнее решение позволяет реализовать проверку индексов (§19.3).

Итераторы для списков должны быть более сложными, чем просто встроенные указатели, так как элементы списков, в общем случае, не знают, где находится следующий элемент списка. Итератор для списка, гипотетически, мог бы быть указателем на связь (link) между элементами:



Общим для всех типов итераторов является их *семантика* и *обозначения операций*. Например, если к любому итератору применить операцию ++, то она вернет значение итератора, указывающего на следующий элемент. Аналогично, для любого итератора операция * возвращает элемент, на который настроен итератор. На самом деле, любой объект, подчиняющийся некоторым требованиям, вроде рассмотренной семантики операций, и будет фактически итератором (§19.2.1). Более того, пользователям в редких случаях нужно знать истинные типы итераторов; каждый контейнер сам «знает» типы своих итераторов и предоставляет их пользователям под стандартными именами *iterator* и *const_iterator*. Например, для *list<Entry>* общим типом итераторов является тип *list<Entry>::iterator*. Мне редко доводилось интересоваться деталями устройства этого типа.

3.8.3. Итераторы и ввод/вывод

Итераторы являются очень общей и полезной концепцией для манипулирования последовательностями элементов в контейнерах. Тем не менее, последовательности элементов встречаются *не только* в контейнерах. Например, *входной поток образует последовательность значений*; мы записываем последовательность значений в поток вывода. Как следствие, концепция итераторов естественным образом применяется ко вводу/выводу.

Для создания итераторного объекта типа *ostream_iterator* мы должны указать, какой поток будет использоваться и каков тип выводимых в него объектов. Например, мы можем определить *итератор, ссылающийся на стандартный поток вывода, cout*:

```
ostream_iterator<string> oo (cout) ;
```

Если мы что-то присваиваем выражению **oo*, то смыслом такого присваивания является запись присваиваемого значения в поток *cout*. Например:

```
int main ()
{
    *oo = "Hello, " ;           // означает cout <<?"Hello, "
    ++oo ;
    *oo = "world! \n" ;       // означает cout <<?"world! \n"
}
```

Можно сказать, что это еще один способ осуществить стандартный вывод «канонического текста» *Hello, world!*. Выражение *++oo* имитирует запись в массив с помощью указателей. Конечно, я не стал бы на практике использовать показанный код для решения столь простой задачи, но заманчиво сразу же показать работу с потоками вывода как с *контейнерами «только для записи» (write-only container)*; вскоре эта идея должна стать очевидной (если не уже стала очевидной).

Аналогично, *istream_iterator* есть нечто, что позволяет нам работать с потоком ввода как с *контейнером «только для чтения» (read-only container)*. Мы должны указать используемый поток ввода и тип ожидаемых объектов:

```
istream_iterator<string> ii (cin) ;
```

Так как *входные итераторы (input iterators)* неизменно используются *парами*, специфицирующими последовательность элементов, мы должны еще предоставить

итератор, указывающий на конец ввода. Им является итератор типа *istream_iterator*, создаваемый «конструктором по умолчанию»:

```
istream_iterator<string> eos;
```

Теперь мы можем ввести фразу *Hello, world!*, а затем вывести ее:

```
int main ()
{
    string s1=*ii;
    ++ii;
    string s2=*ii;

    cout<<s1<<' '<<s2<<'\n';
}
```

Вообще-то, итераторы типов *ostream_iterator* и *istream_iterator* не предназначены для непосредственного использования. Они, как правило, используются в качестве параметров алгоритмов. Например, следующая простая программа читает файл, сортирует прочитанные слова, устраняет дубликаты и выводит результат в другой файл:

```
int main ()
{
    string from, to;
    cin>> from>> to;                                     // получить имена исходного и целевого файлов

    ifstream is (from.c_str() );                          // поток ввода (c_str(); см §3.5.1 и §20.3.7)
    istream_iterator<string> ii (is);                     // итератор ввода для потока
    istream_iterator<string> eos;                          // страж ввода

    vector<string> b (ii, eos);                             // b это вектор, инициализируемый вводом
    sort (b.begin(), b.end());                             // сортируем буфер

    ofstream os (to.c_str() );                             // поток вывода
    ostream_iterator<string> oo (os, "\n");               // итератор вывода для потока

    unique_copy (b.begin(), b.end(), oo);                 // копировать буфер в поток вывода,
                                                            // удаляя повторяющиеся значения
    return !is.eof() || !os;                             // возврат состояния ошибки (§3.2, §21.3.3)
}
```

Тип *ifstream* — это поток ввода *istream*, который может быть связан с файлом, а *ofstream* — это поток вывода *ostream*, который также может связываться с файлом. Второй аргумент конструктора, использованного для создания итератора *oo* типа *ostream_iterator*, предназначен для отделения друг от друга выводимых значений (*delimiter output values*).

3.8.4. Алгоритм *for_each* и предикаты

Для перебора всех элементов последовательностей приходится использовать итераторы в операторах циклов. Это может быть довольно утомительной задачей, так что стандартная библиотека предоставляет иные способы вызова функций для каждого элемента последовательности.

Рассмотрим программу для чтения слов из потока и вычисления частоты их встречаемости. Естественным типом данных, предназначенным для хранения слов вместе с их частотой, является *map*:

```
map<string, int> histogram;
```

Для подсчета частоты встречаемости слов достаточно следующего очевидного кода:

```
void record (const string& s)
{
    histogram [s] ++;           // частота вхождения s
}
```

Когда все данные введены и обработаны, нужно вывести результаты подсчетов. Контейнеры типа **map** содержат последовательности пар значений (*string*,*int*), имеющих *mun pair*, так что для вывода нужно будет вызвать функцию

```
void print (const pair<const string, int>& r)
{
    cout<<r.first<< ' ' <<r.second<< '\n' ;
}
```

для каждого элемента контейнера (первое поле типа *pair* называется *first*, а второе — *second*). Первое поле в *pair* является скорее *const string*, чем просто *string*, так как все ключи в контейнерах **map** есть константы.

Вот клиентский код для выполнения намеченной работы:

```
int main ()
{
    istream_iterator<string> ii (cin) ;
    istream_iterator<string> eos ;

    for_each (ii, eos, record) ;
    for_each (histogram.begin () , histogram.end () , print) ;
}
```

Нет необходимости сортировать контейнер **map** для получения упорядоченного вывода. Ассоциативные массивы *самостоятельно сортируют свои элементы*, так что в нашей программе перебор элементов автоматически осуществляется в возрастающем порядке.

Во многих задачах требуется найти что-либо в контейнере, а не просто выполнить заданное действие для каждого его элемента. Например, алгоритм *find*() (§18.5.2) позволяет отыскивать в контейнере специфическое значение. Более общим вариантом этой задачи является поиск элемента, удовлетворяющего заданному требованию. Например, нам может потребоваться разыскать элемент, значение которого больше 42. Так как контейнеры типа **map** предоставляют свои элементы в виде пар значений (*key,value*), то нам нужно найти среди последовательности элементов контейнера **map**<*string*, *int*> такую пару *pair*<**const string**, *int*>, для которой целочисленное поле больше 42:

```
bool gt_42 (const pair<const string, int>& r)
{
    return r.second>42;
}

void f (map<string, int>& m)
```

```
typedef map<string, int> : const_iterator MI;
MI=find_if(m.begin(), m.end(), gt_42);
// ...
}
```

Кроме того, мы могли бы подсчитать число слов с частотой встречаемости, большей 42:

```
void g(const map<string, int>& m)
{
    int c42 = count_if(m.begin(), m.end(), gt_42);
    // ...
}
```

Функции типа `gt_42()` используются для управления алгоритмами и называются *предикатами*. Предикат *вызывается для каждого элемента и возвращает логическое значение*, которое алгоритм использует для принятия решения — выполнять или не выполнять заданное действие над элементом. Например, алгоритм `find_if()` работает до тех пор, пока его предикат не вернет значение *true*, означающее, что искомый элемент найден. Аналогичным образом, алгоритм `count_if()` подсчитывает число элементов контейнера, для которых предикат возвращает значение *true*.

В стандартной библиотеке имеется ряд готовых предикатов и несколько полезных шаблонов, позволяющих получать новые предикаты (§18.4.2).

3.8.5. Алгоритмы, использующие функции-члены классов

Многие алгоритмы вызывают функции для каждого элемента последовательно. Например, в §3.8.4 алгоритм

```
for_each(ii, eos, record);
```

вызывает функцию `record()` для каждой строки, прочитанной из потока ввода.

Часто приходится работать с контейнерами, содержащими указатели на объекты, для которых по этим указателям нужно вызвать функции-члены классов, а не глобальные функции. Например, нам может потребоваться вызвать функцию-член `Shape::draw()` для каждого элемента контейнера типа `list<Shape*>`. Для решения данной конкретной задачи мы можем просто написать глобальную функцию, вызывающую функцию-член класса:

```
void draw(Shape* p)
{
    p->draw();
}

void f(list<Shape*>& sh)
{
    for_each(sh.begin(), sh.end(), draw);
}
```

Обобщением этой идеи является решение, опирающееся на шаблон `mem_fun()` из стандартной библиотеки:

```
void g (list<Shape*>& sh)
{
    for_each (sh.begin () , sh.end () , mem_fun (&Shape::draw) ) ;
}
```

Шаблон *mem_fun()* (§18.4.4.2) в качестве параметра принимает указатель на функцию-член класса (§15.5) и возвращает нечто, что может быть вызвано по указателю на класс. Результат *mem_fun (&Shape::draw)* берет аргумент типа *Shape** и возвращает то, что возвращает функция *Shape::draw()*.

Шаблон *mem_fun* очень важен на практике, так как позволяет использовать стандартные алгоритмы для контейнеров, содержащих объекты полиморфных типов.

3.8.6. Алгоритмы стандартной библиотеки

Что такое алгоритм? Общее определение алгоритма гласит, что это «конечный набор правил, определяющих последовательность операций для решения конкретного множества задач и удовлетворяющих пяти принципам: конечность ... определенность ... ввод ... вывод ... эффективность» [Knuth, 1968, §1.1]. В контексте стандартной библиотеки *алгоритм есть набор шаблонов для работы с последовательностями элементов.*

Стандартная библиотека содержит десятки алгоритмов. Алгоритмы объявляются в пространстве имен *std* и реализуются в заголовочном файле *<algorithm>*. Вот некоторые из наиболее полезных алгоритмов:

Нет перевода заголовка	
<i>for_each()</i>	Вызвать функцию для каждого элемента (§18.5 1)
<i>find()</i>	Найти первое вхождение аргументов (§18 5.2)
<i>find_if()</i>	Найти первое соответствие предикату (§18.5 2)
<i>count()</i>	Сосчитать число вхождений элемента (§18.5.3)
<i>count_if()</i>	Сосчитать число соответствий предикату (§18 5 3)
<i>replace()</i>	Заменить элемент новым значением (§18.6.4)
<i>reptace_if()</i>	Заменить элемент, соответствующий предикату, новым значением (§18 6.4)
<i>copy()</i>	Скопировать элементы (§18.6.1)
<i>unique_copy()</i>	Скопировать только различные элементы (§18.6.1)
<i>sort()</i>	Отсортировать элементы (§18.7 1)
<i>equal_range()</i>	Найти диапазон всех элементов с одинаковыми значениями (§18 7.2)
<i>merge()</i>	Слияние отсортированных последовательностей (§18.7 3)

Все эти и многие другие алгоритмы (см. главу 18) можно применять к стандартным контейнерам, строкам типа *string* и встроенным массивам.

3.9. Математические вычисления

Как и язык С, С++ не разрабатывался специально для решения вычислительных задач. И тем не менее, математическая проблематика реально присутствует в программах на языке С++, и стандартная библиотека этот факт отражает.

3.9.1. Комплексные числа

Стандартная библиотека поддерживает семейство комплекснозначных типов по линии класса **complex**, рассмотренного в §2.5.2. Чтобы предоставить возможность скалярным элементам комплексных чисел иметь типы **float**, **double** и так далее, класс **complex** стандартной библиотеки объявлен *шаблоном*:

```
template<class scalar> class complex
{
public:
    complex(scalar re, scalar im) ;
    // ...
};
```

Для комплексных чисел поддерживаются обычные арифметические операции и наиболее общие математические функции. Например:

```
// Стандартная функция возведения в степень из <complex>:
template<class C> complex<C> pow(const complex<C>&, int) ;

void f(complex<float> fl, complex<double> db)
{
    complex<long double> ld = fl+sqrt(db) ;
    db += fl*3 ;
    fl=pow(1/fl, 2) ;
    // ...
}
```

Подробности см. в §22.5.

3.9.2. Векторная арифметика

Класс **vector**, рассмотренный в §3.7.1, разрабатывался так, чтобы обеспечить общий и гибкий механизм для хранения объектов, который согласуется с архитектурой контейнеров, итераторов и алгоритмов. Этот класс не поддерживает математических операций с векторами. Добавить в класс **vector** такие операции довольно просто, но его общность и гибкость препятствуют достижению высокой эффективности, которой придается важное значение в серьезных вычислительных задачах. Как следствие, в стандартной библиотеке имеется еще один *векторный тип*, который называется **valarray**. Он не настолько общий, зато хорошо поддается оптимизации при вычислениях:

```
template<class T> class valarray
{
    // ...
    T& operator[] (size_t) ;
    ..
```

Tun size_t есть беззнаковый целочисленный тип, который используется в конкретных системах для индексации массивов.

Для класса *valarray* реализованы обычные арифметические операции и наиболее общие математические функции. Например:

// Стандартная функция из <valarray>, вычисляющая модуль:

```
template<class T> valarray<T> abs (const valarray<T>& ) ;
```

```
void f(valarray<double>& a1, valarray<double>& a2)
```

```
{  
    valarray<double> a = a1*3.14+a2/a1;  
    a2 += a1*3.14;  
    a = abs (a) ;  
    double d = a2[7] ;  
    // ...  
}
```

Подробности см. в §22.4.

3.9.3. Поддержка базовых вычислительных операций

Естественно, стандартная библиотека содержит элементарные математические функции — такие как *log()*, *pow()* и *cos()* для типов с плавающей запятой; см. §22.3. Для всех встроенных типов предусмотрены классы, описывающие их характерные свойства, например, наибольший показатель степени для чисел с плавающей запятой; см. §22.2.

3.10. Основные средства стандартной библиотеки

Средства стандартной библиотеки можно классифицировать следующим образом:

1. Базовая поддержка языка программирования (выделение памяти; определение типа на стадии выполнения); см. §16.1.3.
2. Стандартная библиотека языка C (с минимальнейшими изменениями для предотвращения нарушений в системе типов); см. §16.1.2.
3. Строки и потоки ввода/вывода (с поддержкой национальных алфавитов и локализации); см. главы 20 и 21.
4. Набор контейнеров (таких как *vector*, *list* и *map*) и алгоритмов для работы с ними (прохода по всем элементам, сортировки, слияния и т.д.); см. главы 16, 17, 18 и 19.
5. Поддержка численных расчетов (комплексные числа, вектора для математических расчетов, BLAS-подобные и обобщенные срезы, семантика оптимизации); см. главу 22.

Главными критериями для включения класса в стандартную библиотеку являлись следующие соображения: будет ли он использоваться любыми C++-программистами (экспертами и новичками); может ли он быть представлен в общей форме, не влекущей за собой заметных накладных расходов по сравнению с более простыми средствами; можно ли легко научиться его простейшему применению. По сути

дела, *стандартная библиотека* предоставляет наиболее *фундаментальные реализации основных структур данных* вкупе с *фундаментальными алгоритмами их обработки*.

Любой алгоритм работает с любым контейнером без каких-либо преобразований. Такая структура библиотеки, обычно называемая STL (Standard Template Library; Stepanov, 1994), позволяет пользователям легко создавать свои собственные контейнеры и алгоритмы помимо имеющихся стандартных, и заставлять их согласованно работать с последними.

3.11. Советы

1. Не изобретайте колесо заново; используйте библиотеки.
2. Не верьте в чудеса; разберитесь, что делают ваши библиотеки, как они это делают и какова цена их деятельности.
3. Когда есть выбор, отдавайте предпочтение стандартной библиотеке.
4. Не думайте, что стандартная библиотека идеально подходит для всех задач.
5. Не забывайте про директиву **#include** для включения заголовочных файлов с определением средств стандартной библиотеки; §3.3.
6. Помните, что все средства стандартной библиотеки определены в пространстве имен **std**; §3.3.
7. Вместо строк типа **char*** пользуйтесь строками типа **string**; §3.5, §3.6.
8. Если сомневаетесь — используйте вектора с проверкой диапазона индексов (такие как **Vec**); §3.7.2.
9. Используйте **vector<T>**, **list<T>**, **map<key,value>** вместо **T[]**; §3.7.1, §3.7.3, §3.7.4.
10. При добавлении элементов в контейнер используйте **push_back()** или **back_inserter()**; §3.7.3, §3.8.
11. Лучше использовать **push_back()** для векторов, чем **realloc()** для встроенных массивов; §3.8.
12. Самые общие исключения перехватывайте в стартовой функции **main()**; §3.7.2.

